

Fanis Prodromou

- Google Developer Expert (GDE) in Angular
- Author of Mastering Angular Reactive Forms
- Educator & Technical Content Creator
- Senior Angular Developer @ ASI
- Co-organizer of Angular Athens Meetup

Code. Teach. Community. Angular.

@prodromouf /prodromouf https://blog.profanis.me

House of Angular Services

HOUSE OF ANGULAR



TEAM WORKSHOPS



AI IMPLEMENTATION



AUDIT & CONSULTING



ANGULAR DEVELOPMENT



UX/UI DESIGN FOR ANGULAR



MIGRATE TO ANGULAR

Scalable Architecture & Modern Reactivity

Target Audience & Prerequisites

Target Audience

- Experienced Angular Developers

  1+ years of professional experience with the core framework.

- Architecture Seekers

  Familiar with Core-Feature-Shared but ready for a more scalable approach.

- Forward-Thinking Engineers

  Ready to modernize workflows and move away from "spaghetti" monoliths.

Target Audience & Prerequisites

What You DON'T Need to Know

- **No Prior Nx Experience Required**

  We'll learn the CLI and workspace structure from the ground up.

- **No Prior Signals Knowledge Needed**

  We will cover the mental shift from Zone.js to fine-grained reactivity.

- **No Mastery of RxJS Required**

  Learn how to simplify your code and reduce boilerplate with Signal patterns.

👨‍💻 Nx Foundation & Architecture	60 mins	(09:00 – 10:00)
☕ Short Break	15 mins	(10:00 - 10:15)
👨‍💻 Nx Architecture & the Intelligent Graph	90 mins	(10:15 – 11:45)
👨‍💻 Context is King: Supercharging AI with Nx	20 mins	(11:45 – 12:05)
🍌 Short Break	15 mins	(12:05 – 12:20)
👨‍💻 Introducing Signals	90 mins	(12:20 – 13:50)
👨‍🍳 Lunch Break	45 mins	(13:50 – 14:35)
👨‍💻 Advanced Reactivity	90 mins	(14:35 – 16:05)
🍌 Short Break	15 mins	(16:05 – 16:20)
👋 Wrap up and Q&A	30 mins	(16:20 – 16:50)

Nx Foundation & Architecture

Building Scalable "Moduliths" with Angular

In this lesson

- The "Monolith" Problem
- Why Nx?
- What is "The Modulith"?
- Explore the folder structure
- Project Types
- The Golden Rule: The Barrel File
- Tooling: CLI & Nx Console
- Hands-on Overview

The Goal

GreenHeaven Plant Shop

GreenHeaven

Home Catalogue

Filters

Additional filters coming soon...

Plant Catalogue

Discover our beautiful collection of indoor plants

Search plants

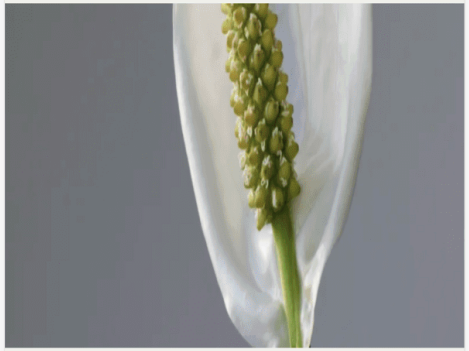
Showing 9 of 70 plants

Snake Plant



$123

Peace Lily



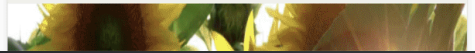
$123

Lavender

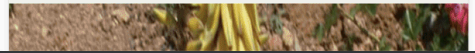


$123

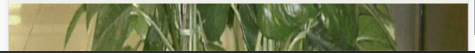
Sunflower

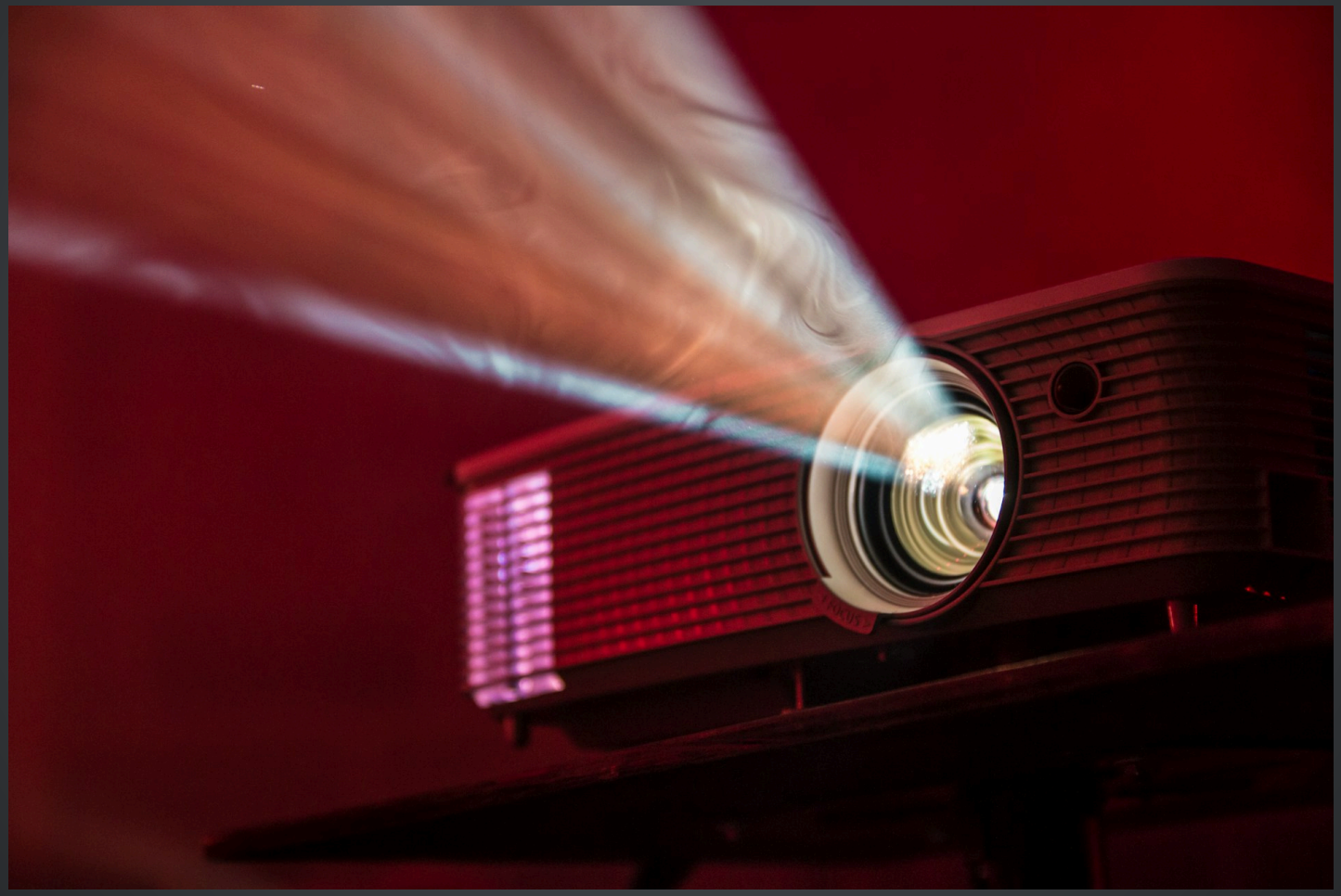


Aloe Vera



Pothos





“ We aren't here to just learn a new library.

We are here to fix the way we work.

The Angular Developer’s "Wall

Daily frustrations we all face

- The "Spaghetti" Fear

  Fearing circular dependencies and accidental coupling when sharing code.

- Architecture Debt

  Debating folder structures because the standard CLI lacks a roadmap for scale.

- Wasted Effort

  Running 1,000 tests when you only touched two files.

Why Nx?

A smart build system (like Angular CLI on steroids)

- Computation Caching
- Smart Rebuilds ("Affected")
- Architectural Guardrails
- Modern Tooling

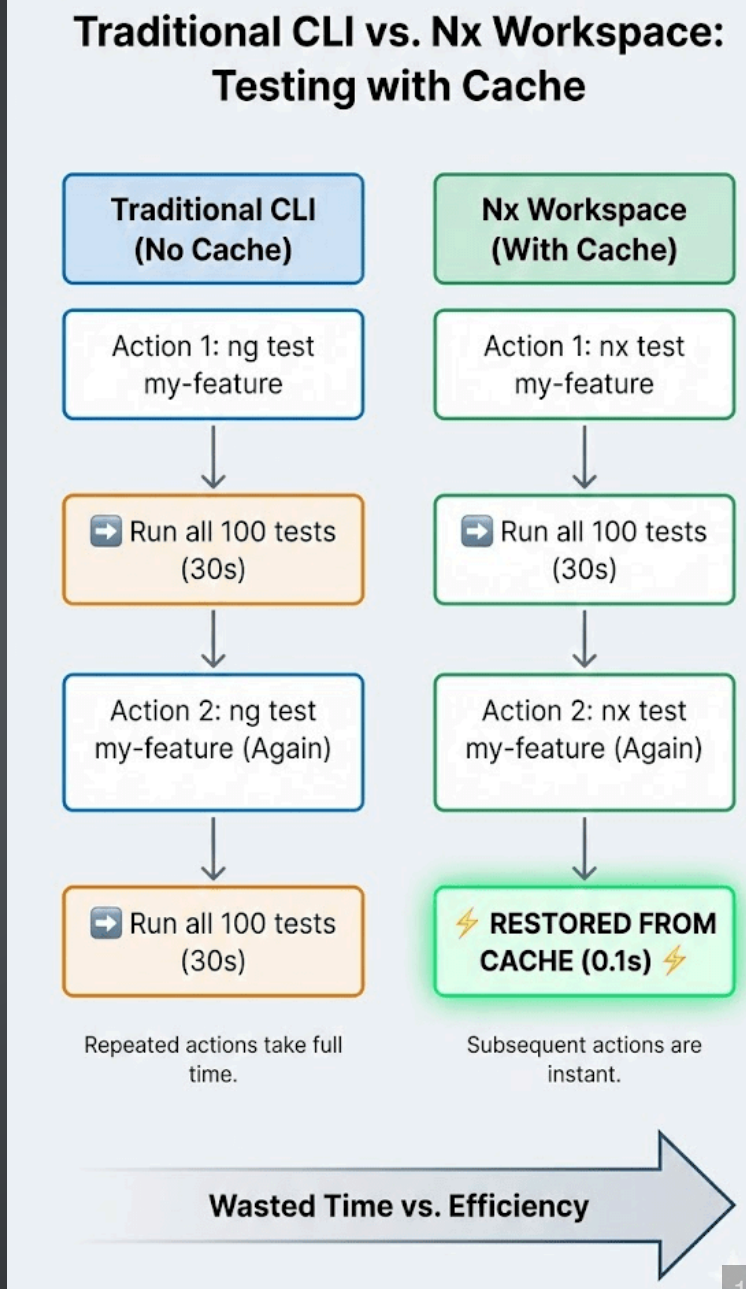


Why Nx?

A smart build system (like Angular CLI on steroids)

- Computation Caching

```
nx test
```

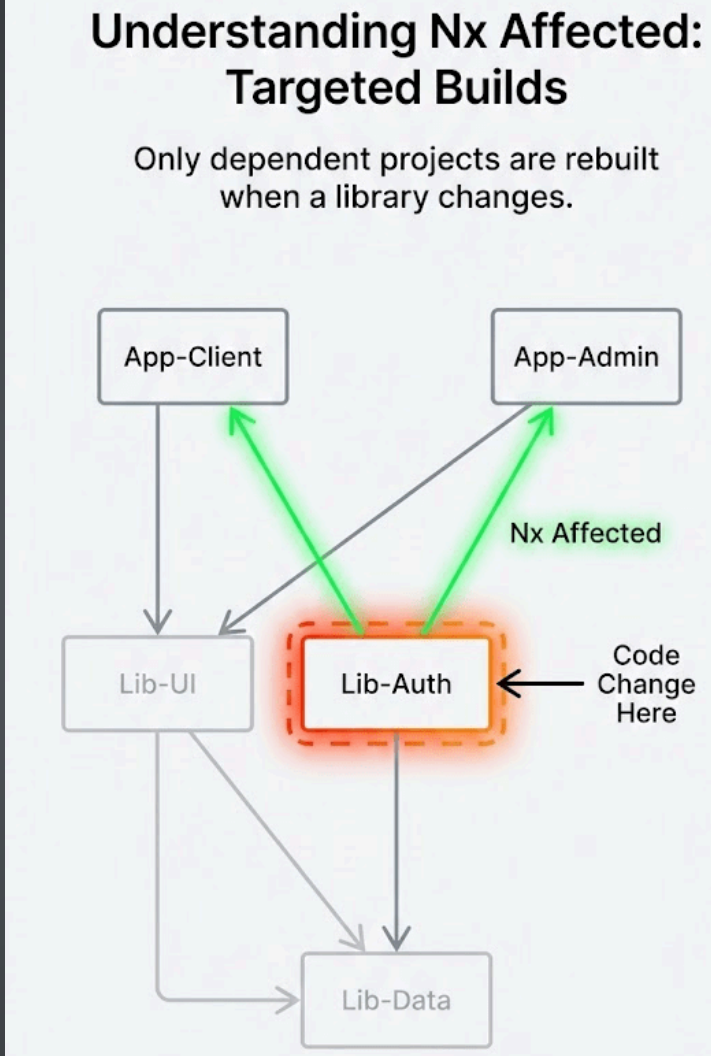


Why Nx?

A smart build system (like Angular CLI on steroids)

- Smart Rebuilds ("Affected")

```
nx build --affected
```

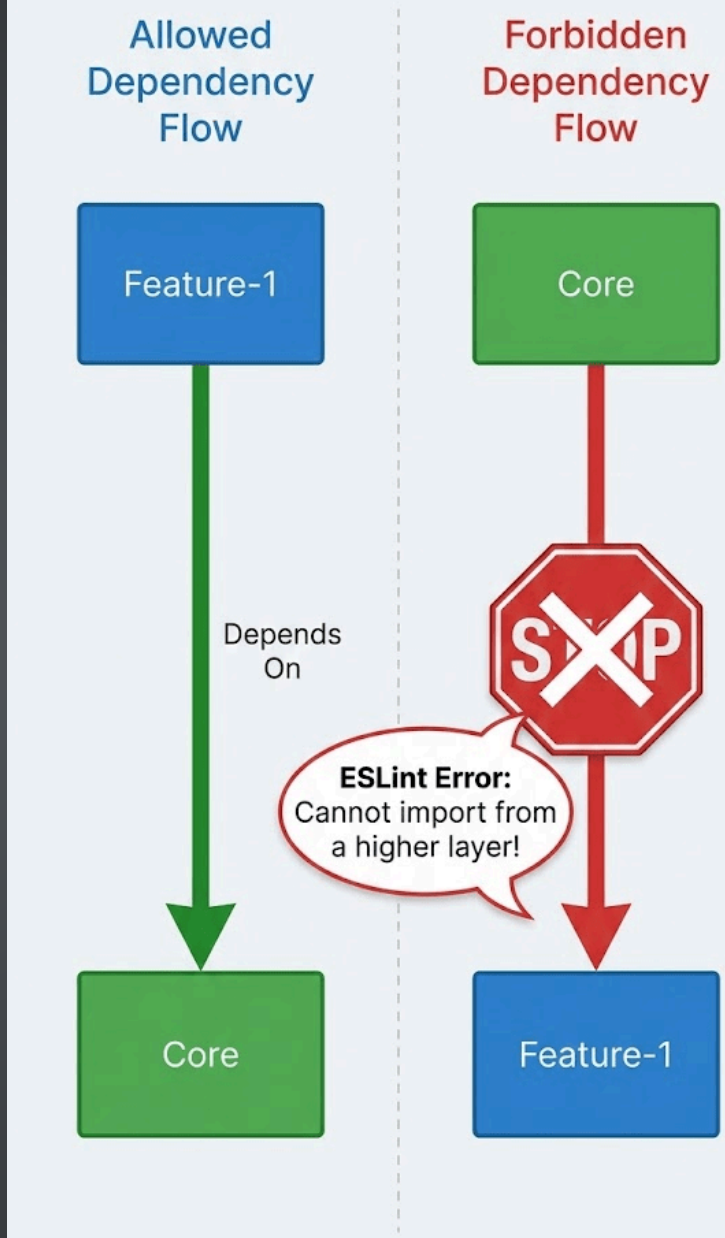


Why Nx?

A smart build system (like Angular CLI on steroids)

- Architectural Guardrails

```
nx lint
```



Enforce Architecture with ESLint Rules

profanis

Why Nx?

A smart build system (like Angular CLI on steroids)

- Modern Tooling

Batteries Included. No Manual Configuration Required.


Jest


Cypress


Playwright


ESLint


Vite


Esbuild

The "Monolith" Problem

- Dump everything into `src/app`
- Tight Coupling

  Features accidentally depend on each other
- Project Centric

  Compiles one project at a time
- Hard to Share

  Reusing code between a Customer App and an Admin App is difficult.



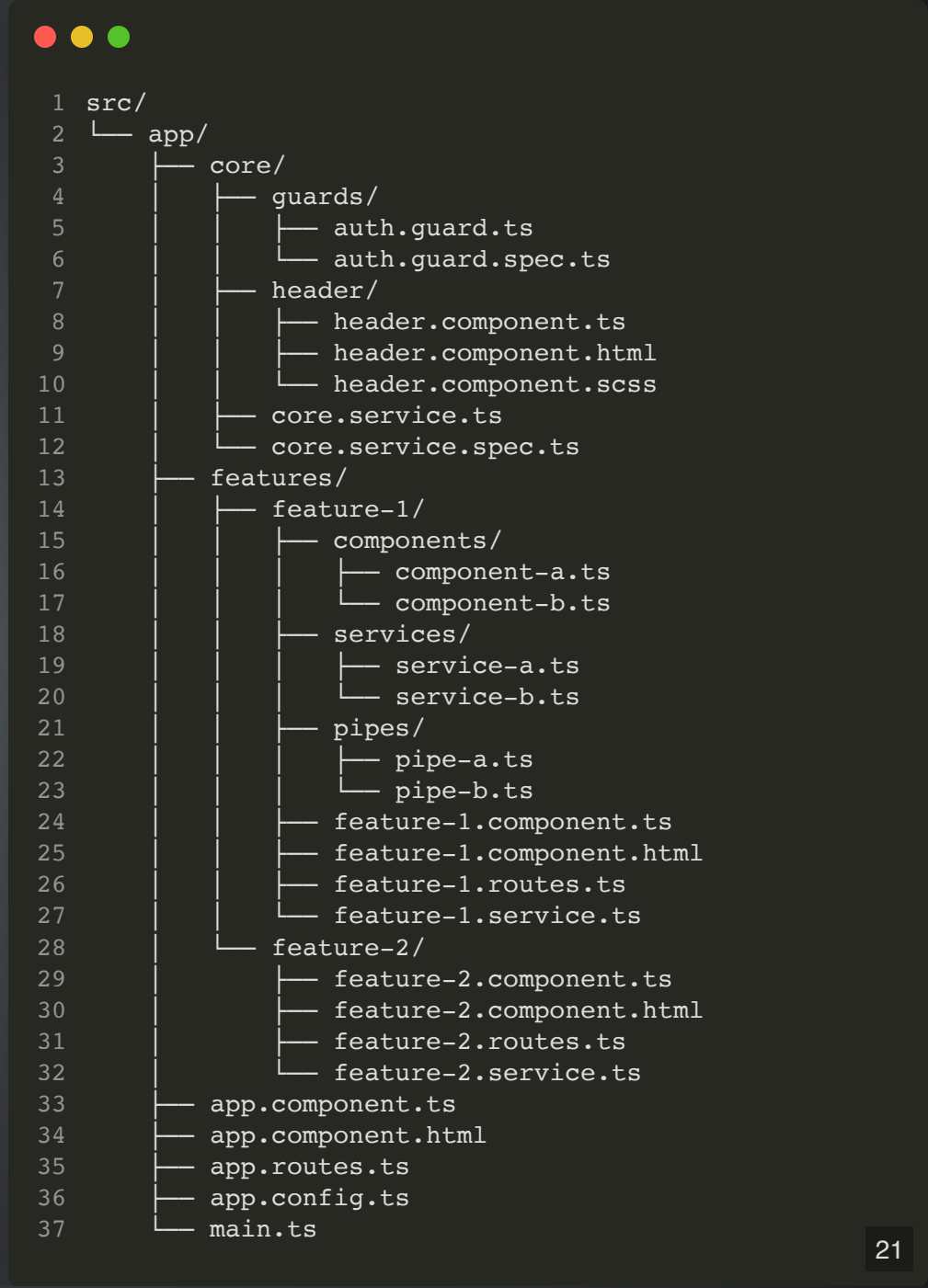
The "Monolith" Problem

- dump everything into `src/app`

```
src/
└── app/
    ├── core/
    │   ├── guards/
    │   │   ├── auth.guard.ts
    │   │   └── auth.guard.spec.ts
    │   ├── header/
    │   │   ├── header.component.ts
    │   │   ├── header.component.html
    │   │   └── header.component.scss
    │   ├── core.service.ts
    │   └── core.service.spec.ts
    ├── features/
    │   ├── feature-1/
    │   │   ├── feature-1.component.ts
    │   │   ├── feature-1.component.html
    │   │   ├── feature-1.routes.ts
    │   │   └── feature-1.service.ts
    │   └── feature-2/
    │       ├── feature-2.component.ts
    │       ├── feature-2.component.html
    │       ├── feature-2.routes.ts
    │       └── feature-2.service.ts
    ├── shared/
    │   ├── components/
    │   │   ├── button/
    │   │   │   ├── button.component.ts
    │   │   │   ├── button.component.html
    │   │   │   └── button.component.scss
    │   │   └── input/
    │   │       ├── input.component.ts
    │   │       ├── input.component.html
    │   │       └── input.component.scss
    │   ├── guards/
    │   │   ├── can-leave.guard.ts
    │   │   └── can-leave.guard.spec.ts
    │   └── pipes/
    │       ├── format-date.pipe.ts
    │       └── format-date.pipe.spec.ts
    ├── app.component.ts
    ├── app.component.html
    ├── app.routes.ts
    ├── app.config.ts
    └── main.ts
```

The "Monolith" Problem

- dump everything into `src/app`



```
src/
└── app/
    ├── core/
    │   ├── guards/
    │   │   ├── auth.guard.ts
    │   │   └── auth.guard.spec.ts
    │   ├── header/
    │   │   ├── header.component.ts
    │   │   ├── header.component.html
    │   │   └── header.component.scss
    │   ├── core.service.ts
    │   └── core.service.spec.ts
    ├── features/
    │   ├── feature-1/
    │   │   ├── components/
    │   │   │   ├── component-a.ts
    │   │   │   └── component-b.ts
    │   │   ├── services/
    │   │   │   ├── service-a.ts
    │   │   │   └── service-b.ts
    │   │   ├── pipes/
    │   │   │   ├── pipe-a.ts
    │   │   │   └── pipe-b.ts
    │   │   ├── feature-1.component.ts
    │   │   ├── feature-1.component.html
    │   │   ├── feature-1.routes.ts
    │   │   └── feature-1.service.ts
    │   └── feature-2/
    │       ├── feature-2.component.ts
    │       ├── feature-2.component.html
    │       ├── feature-2.routes.ts
    │       └── feature-2.service.ts
    ├── app.component.ts
    ├── app.component.html
    ├── app.routes.ts
    ├── app.config.ts
    └── main.ts
```

The "Monolith" Problem

- Tight Coupling

  Features accidentally depend on each other

```ts
// src/app/core/logger.service.ts

import { FeatureAServiceService }
    from '../features/feature-a.service'; // <-- Oops

@Injectable({ providedIn: 'root' })
export class LoggerService {
  private featureService = inject(FeatureAService)
}
```

```ts
// src/app/feature-1.service.ts
import { Feature2Service }
    from '../feature-2.service'; // Dependency A -> B

@Injectable()
export class Feature1Service {
}

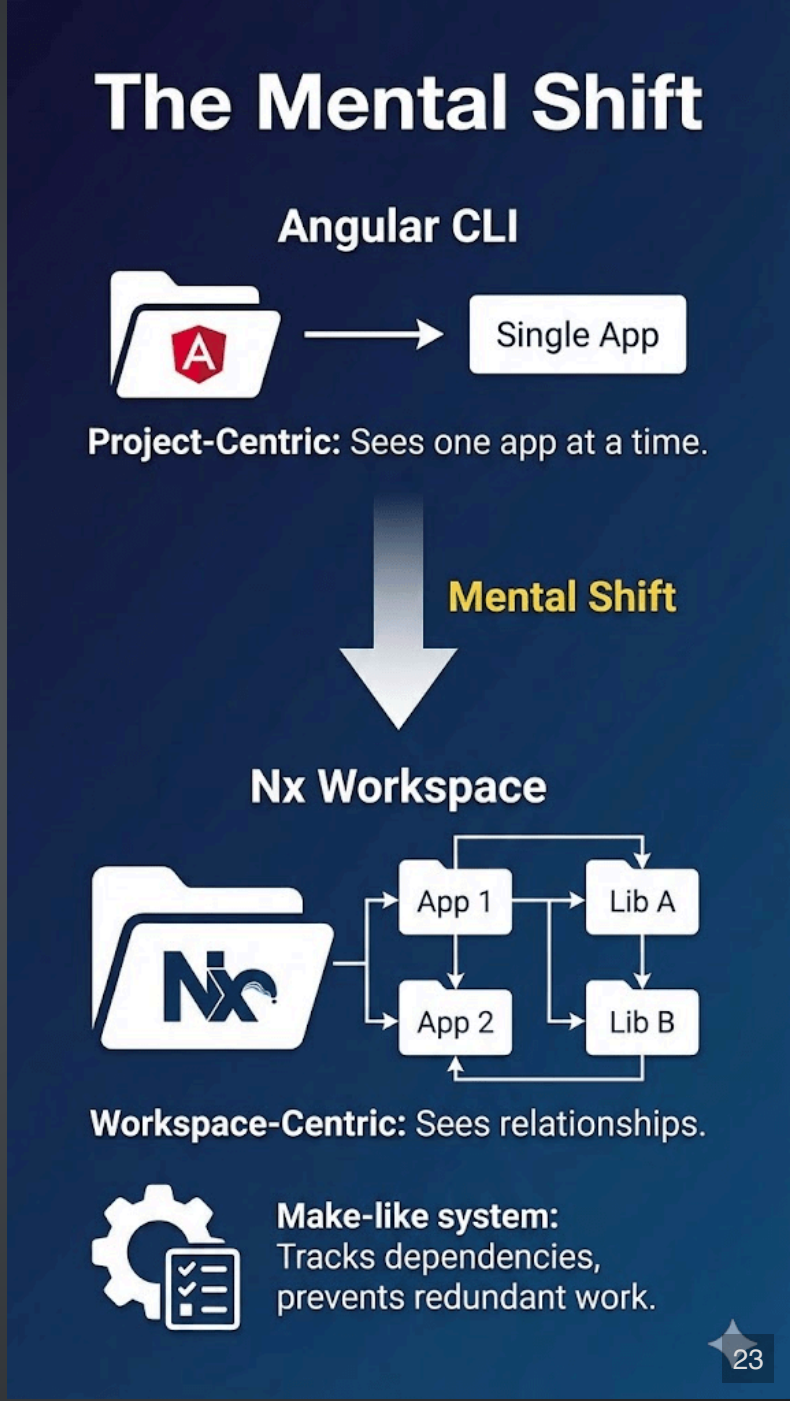
// src/app/feature-2.service.ts
import { Feature1Service }
    from '../feature-1.service'; // Dependency B -> A

export class Feature2Service {
}
```

The "Monolith" Problem

- Project Centric

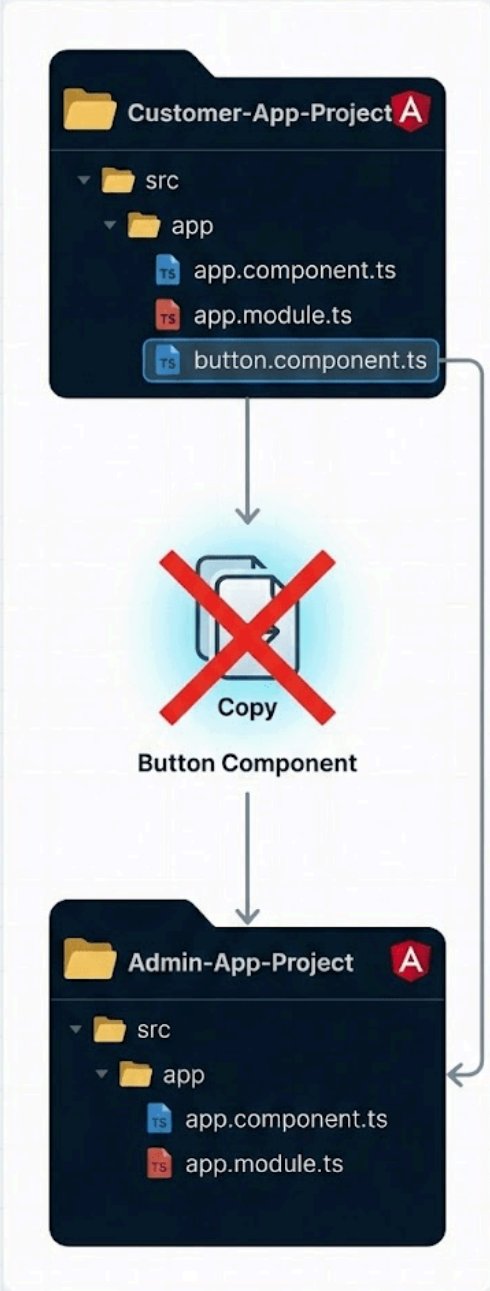
  Compiles one project at a time



The "Monolith" Problem

- **Hard to Share**

  Reusing code between a Customer App and an Admin App is difficult.



The modulith

You don't need 50 apps to use Nx

- Concept

  - We build a **Single Application**

  Monolith deployment

  - We structure it like **Microservices**

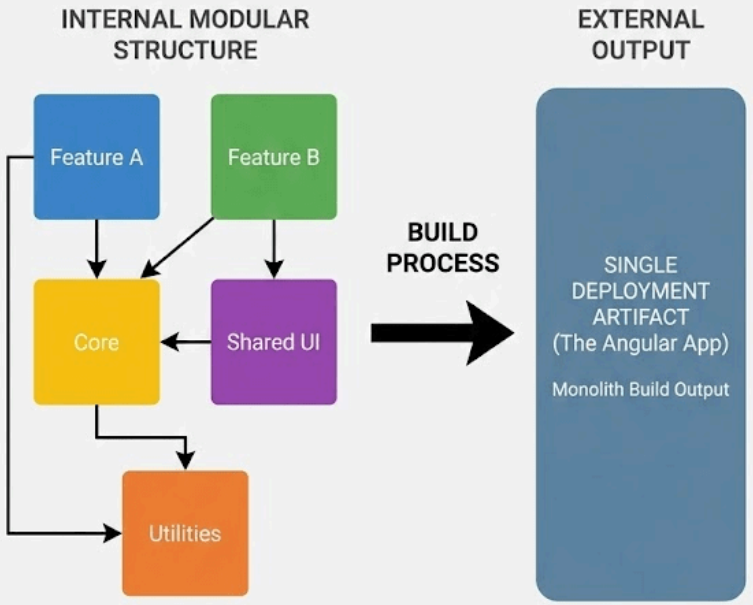
  Modular development

```
my-workspace/
├── apps/
│   └── main-app/
│       ├── app.component.ts
│       ├── app.config.ts
│       ├── app.routes.ts
│       └── main.ts
└── libs/
    ├── auth/
    │   └── guards/
    │       ├── auth.guard.ts
    │       └── auth.guard.spec.ts
    ├── shared/
    │   ├── ui/
    │   │   └── header/
    │   │       ├── header.component.ts
    │   │       ├── header.component.html
    │   │       └── header.component.scss
    │   └── data-access/
    │       ├── core.service.ts
    │       └── core.service.spec.ts
    ├── feature-1/
    │   ├── components/
    │   ├── services/
    │   ├── pipes/
    │   ├── feature-1.component.ts
    │   ├── feature-1.routes.ts
    │   └── index.ts
    └── feature-2/
        ├── feature-2.component.ts
        ├── feature-2.routes.ts
        └── index.ts
```

The modulith

You don't need 50 apps to use Nx

- Strategy
  - Treat your `libs` folder like internal npm packages
  - Strict boundaries between features
  - Clear public APIs

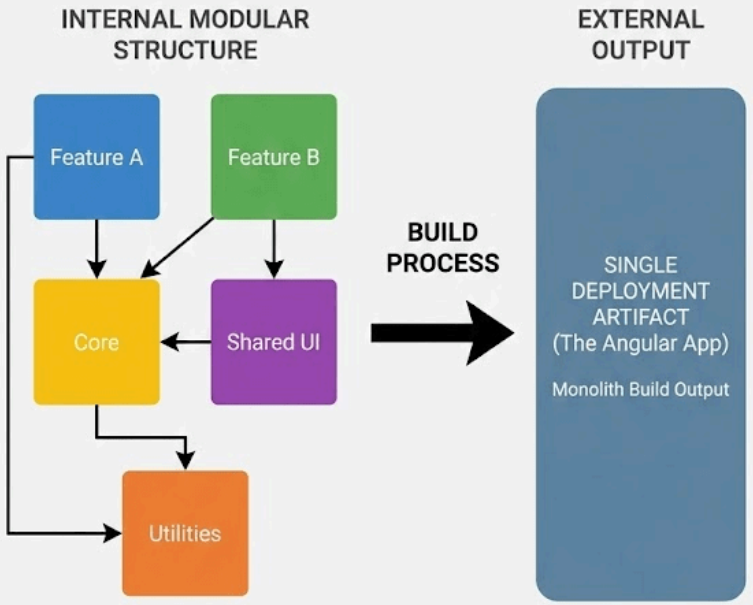


The modulith

You don't need 50 apps to use Nx

- Why

  Lack of organization slows down engineers



Folder Structure

- /apps
  - The shell
  - Entry point that wires everything up
- /libs
  - business logic
  - components
  - UI components

```
workspace-root/
├── apps/
│   └── customer-portal/  <-- App Shell
└── libs/
    ├── data-access/      <-- Shared Logic
    ├── ui-components/    <-- Shared UI
    └── feature-orders/   <-- Feature Logic
```

Folder Structure

- /apps 20%
- /libs 80%

```
workspace-root/
├── apps/
│   └── customer-portal/  <-- App Shell
└── libs/
    ├── data-access/      <-- Shared Logic
    ├── ui-components/    <-- Shared UI
    └── feature-orders/   <-- Feature Logic
```

Project Types

The structure is the solution

- Feature
  - Like our Catalogue Page
- UI
  - Like our Product Card
- Data-Access
  - Like our Product API Service
- Utility
  - Like Date Formatters

```
my-workspace/
├── apps/
│   └── main-app/
│       ├── app.component.ts
│       ├── app.config.ts
│       ├── app.routes.ts
│       └── main.ts
└── libs/
    ├── auth/
    │   └── guards/
    │       ├── auth.guard.ts
    │       └── auth.guard.spec.ts
    ├── shared/
    │   ├── ui/
    │   │   └── header/
    │   │       ├── header.component.ts
    │   │       ├── header.component.html
    │   │       └── header.component.scss
    │   └── data-access/
    │       ├── core.service.ts
    │       └── core.service.spec.ts
    ├── feature-1/
    │   ├── components/
    │   ├── services/
    │   ├── pipes/
    │   ├── feature-1.component.ts
    │   ├── feature-1.routes.ts
    │   └── index.ts
    └── feature-2/
        ├── feature-2.component.ts
        ├── feature-2.routes.ts
        └── index.ts
```

profanis

The Barrel File

Only Export What You Mean To Share

- The Problem

  - developers can import any file deep within another library

```
1 import { InternalHelper }
2     from 'libs/data-access/src/lib/internal-folder/helper.service'
```

The Barrel File

Only Export What You Mean To Share

- The Solution

  - Every Nx library has a barrel file, typically located at **libs/my-lib/src/index.ts**

  - Anything you don't export from this file is considered "private" to that library.

The Barrel File

Only Export What You Mean To Share

- Example

```
📂 libs/data-access/
├── 📂 src/lib/
│   ├── internal-helper.service.ts  <-- Private file
│   └── public-product.service.ts   <-- Public file
└── index.ts                        <-- Public API
```

```
// index.ts content:
export * from './src/lib/public-product.service';

// We do NOT export internal-helper.service.ts here!
```

The How

NX Tooling

- Powerful CLI

```
# Create a feature library
npx nx g @nx/angular:library --name=feature-home
                             --directory=libs/feature-home

# Create a UI library
npx nx g @nx/angular:library --name=ui-header
                             --directory=libs/shared/ui/header
```

Basic NX Commands

CLI

Create a workspace

```
npx create-nx-workspace@latest
```

Create a Library

```
nx generate @nx/angular:library [name]
```

Generate a Component

```
nx generate @nx/angular:component [name] --project=[project-name]
```

more: https://nx.dev/docs/technologies/angular/guides/nx-and-angular

https://nx.dev/docs/reference/nx-commands

The How

NX Tooling

- NX Console

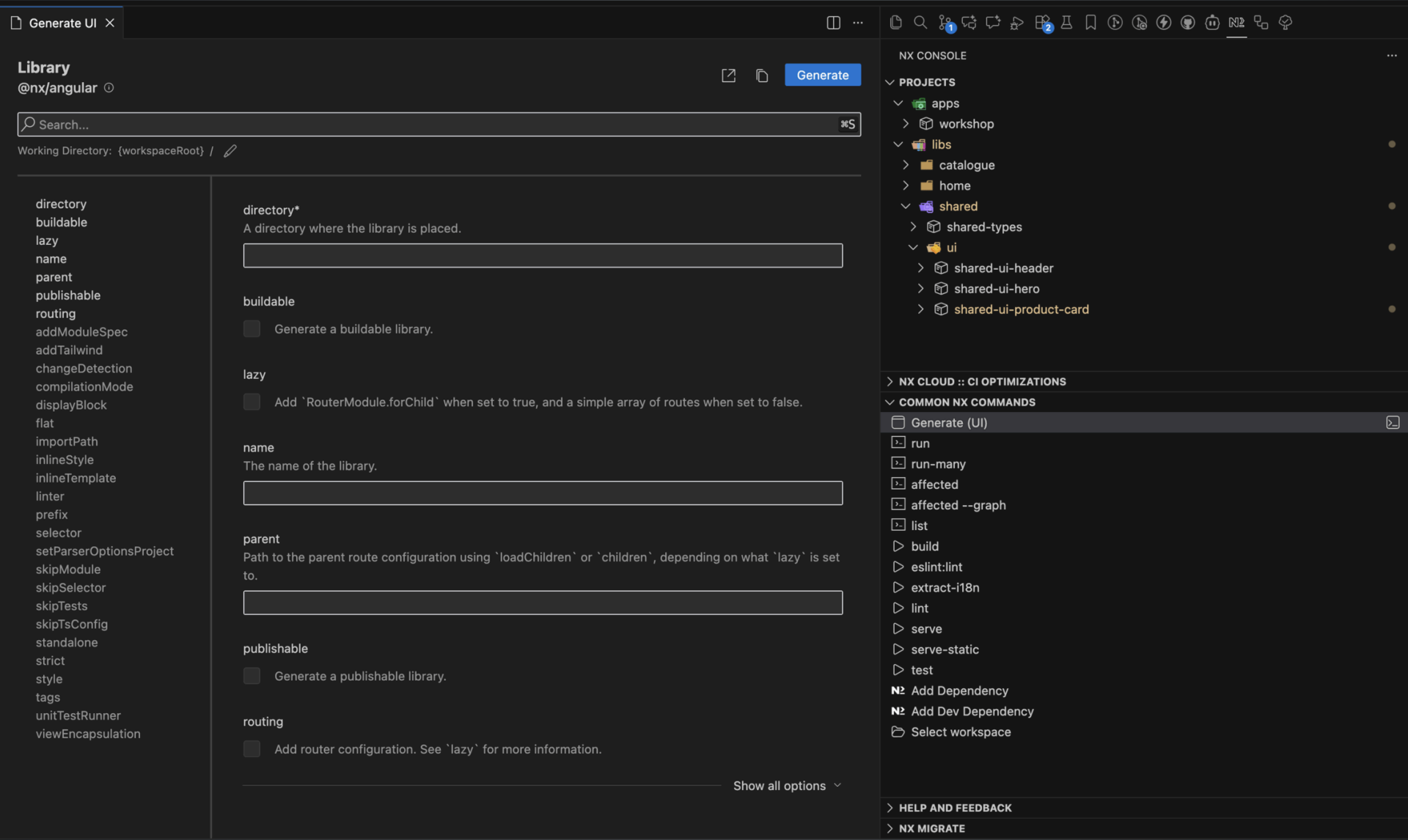


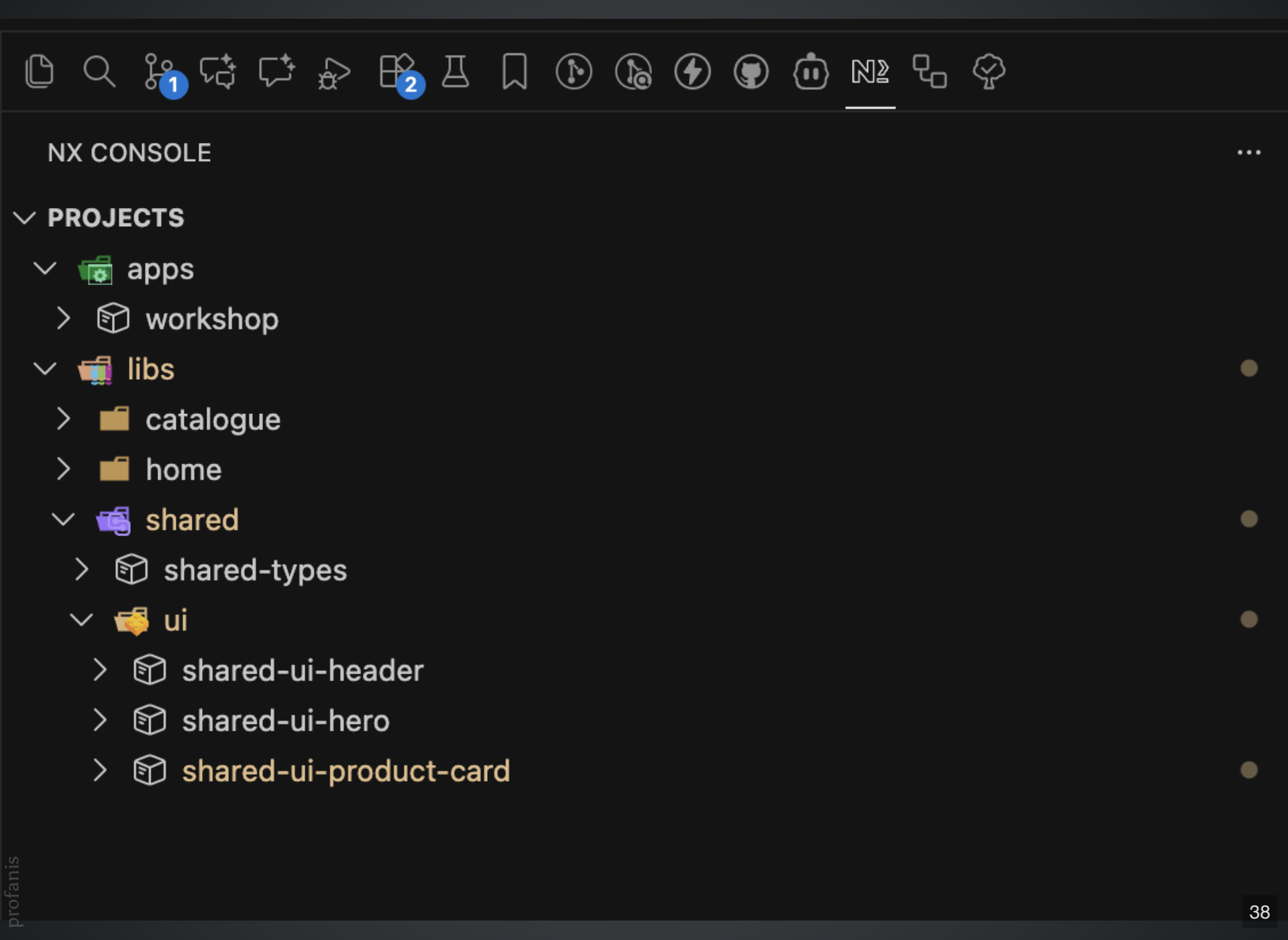
Nx Console

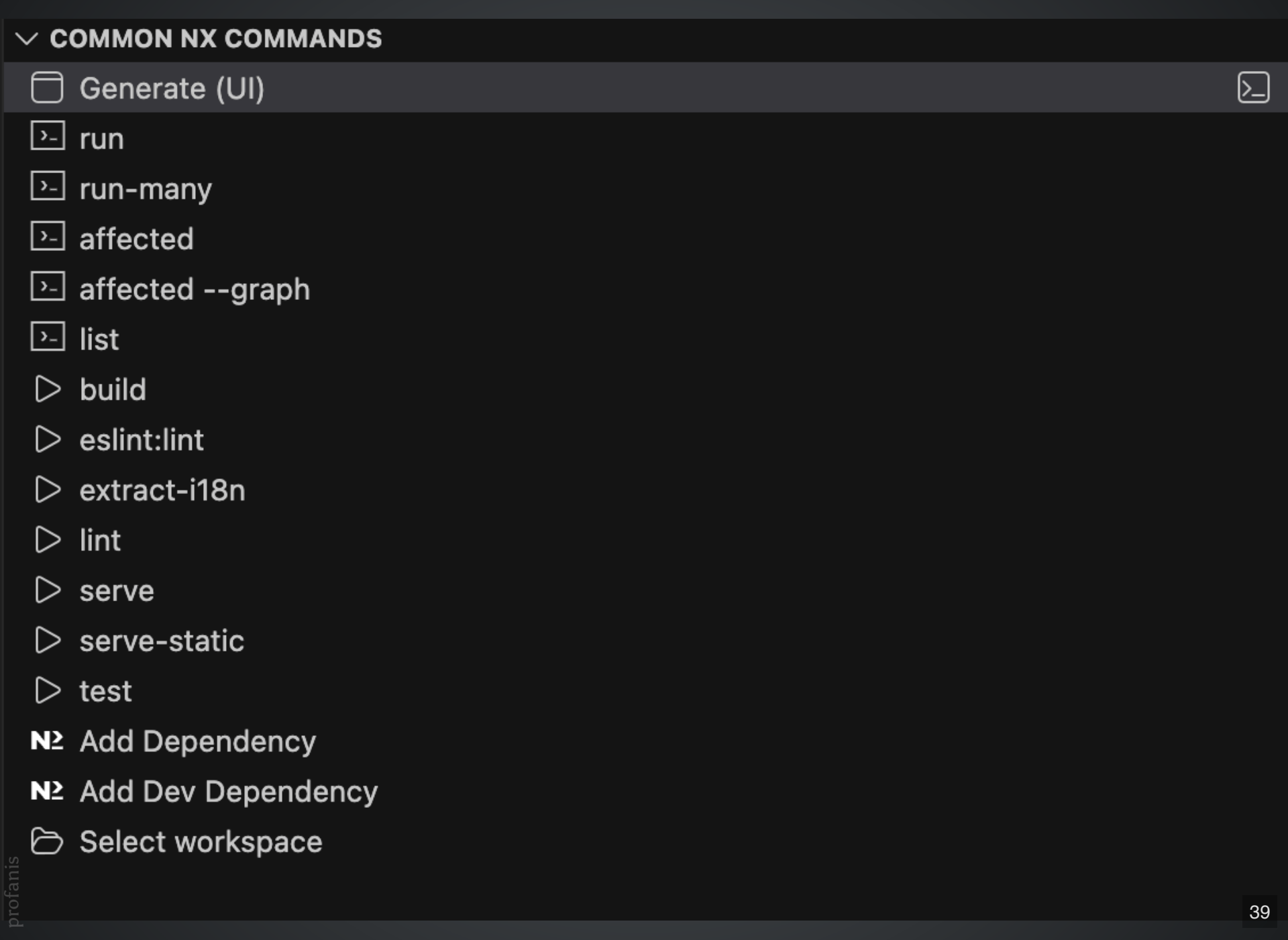
nrwl nx.dev | 2,170,810 | (60)

The UI for Monorepos, providing visual workflows and enriching your AI Chat with deep insights

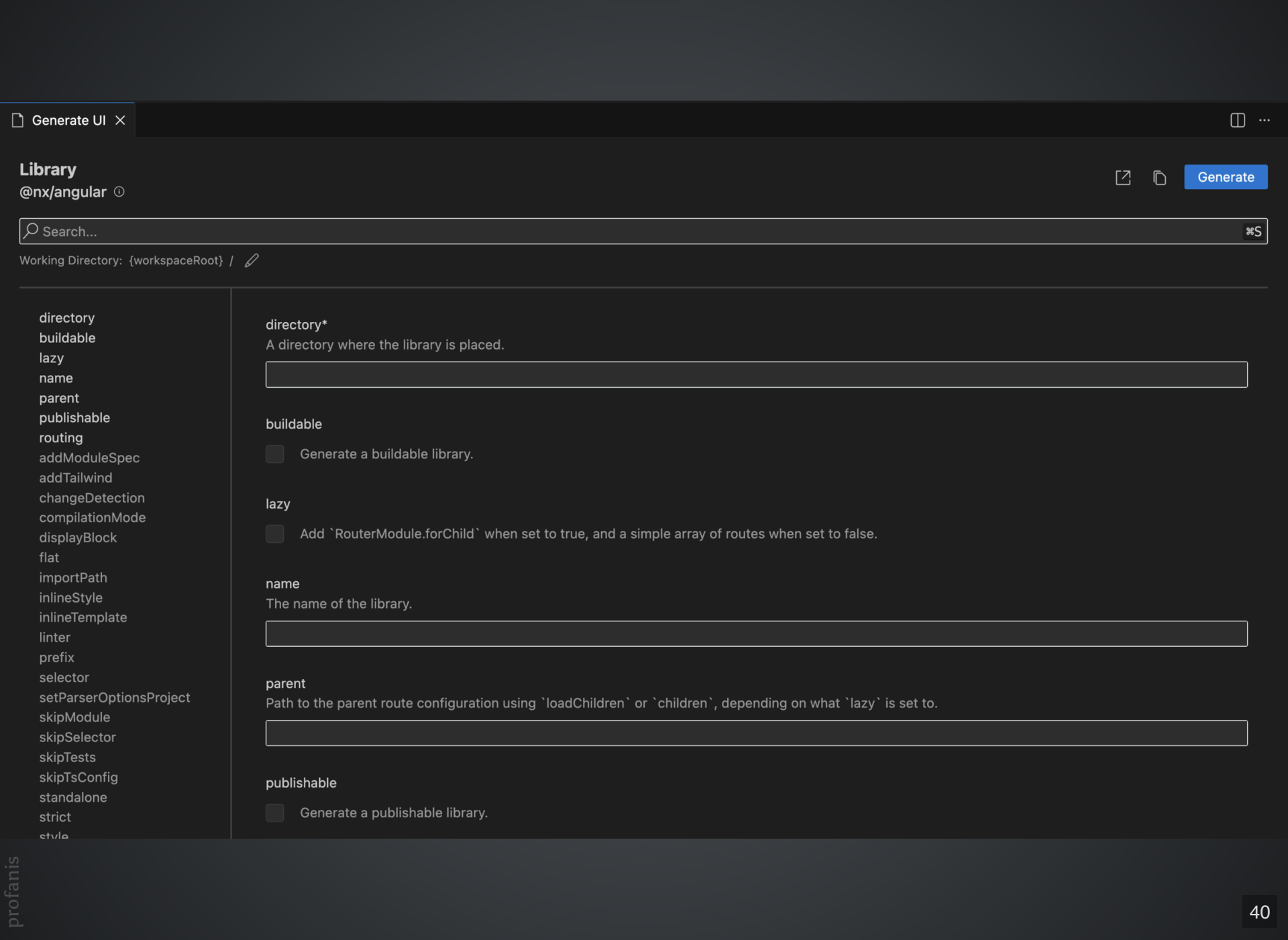
Disable | Uninstall | Auto Update


Generate UI
Library
@nx/angular
Generate
Search...
⌘S
Working Directory: {workspaceRoot} /
directory
buildable
lazy
name
parent
publishable
routing
addModuleSpec
addTailwind
changeDetection
compilationMode
displayBlock
flat
importPath
inlineStyle
inlineTemplate
linter
prefix
selector
setParserOptionsProject
skipModule
skipSelector
skipTests
skipTsConfig
standalone
strict
style
tags
unitTestRunner
viewEncapsulation
directory*
A directory where the library is placed.
buildable
Generate a buildable library.
lazy
Add `RouterModule.forChild` when set to true, and a simple array of routes when set to false.
name
The name of the library.
parent
Path to the parent route configuration using `loadChildren` or `children`, depending on what `lazy` is set to.
publishable
Generate a publishable library.
routing
Add router configuration. See `lazy` for more information.
Show all options
NX CONSOLE
PROJECTS
apps
workshop
libs
catalogue
home
shared
shared-types
ui
shared-ui-header
shared-ui-hero
shared-ui-product-card
NX CLOUD :: CI OPTIMIZATIONS
COMMON NX COMMANDS
Generate (UI)
run
run-many
affected
affected --graph
list
build
eslint:lint
extract-i18n
lint
serve
serve-static
test
Add Dependency
Add Dev Dependency
Select workspace
HELP AND FEEDBACK
NX MIGRATE


NX CONSOLE
PROJECTS
apps
workshop
libs
catalogue
home
shared
shared-types
ui
shared-ui-header
shared-ui-hero
shared-ui-product-card


COMMON NX COMMANDS
Generate (UI)
run
run-many
affected
affected --graph
list
build
eslint:lint
extract-i18n
lint
serve
serve-static
test
Add Dependency
Add Dev Dependency
Select workspace

profanis


Generate UI
Library
@nx/angular
Generate
Search...
⌘S
Working Directory: {workspaceRoot} /
directory
buildable
lazy
name
parent
publishable
routing
addModuleSpec
addTailwind
changeDetection
compilationMode
displayBlock
flat
importPath
inlineStyle
inlineTemplate
linter
prefix
selector
setParserOptionsProject
skipModule
skipSelector
skipTests
skipTsConfig
standalone
strict
style
directory*
A directory where the library is placed.
buildable
Generate a buildable library.
lazy
Add `RouterModule.forChild` when set to true, and a simple array of routes when set to false.
name
The name of the library.
parent
Path to the parent route configuration using `loadChildren` or `children`, depending on what `lazy` is set to.
publishable
Generate a publishable library.

Hands-on



Where we're going

The shape you will have built by the end of this section

```
workshop/
├── apps/
│   └── workshop              the shell — routes and nothing else
└── libs/
    ├── feature-home            @workshop/feature-home
    └── feature-catalogue       @workshop/feature-catalogue
```

- The app wires. The libraries own.

- Every library is a lazy route boundary.

- Every library has exactly one public entry point: index.ts

Hands-on

Five steps

1. Create the Nx workspace
2. Install the Nx Console plugin
3. Generate feature-catalogue with the Nx CLI
4. Generate feature-home with the Nx Console
5. Wire the router and verify

You are building this one from nothing. Section 2 starts from a repo that already has it — falling behind here costs you nothing later.

Hands-on

Create the Nx workspace

TASK

```
npx create-nx-workspace@latest --preset angular-monorepo
```

Why the monorepo preset: it gives you an apps/ and a libs/ from minute one. The standalone preset does not, and retrofitting the split later is the expensive version of this decision.

The installer will ask you eleven questions. Only four of them matter — next slide.

Hands-on

STEP 1 / 5

Create the Nx workspace

HINT

Workspace / application name	workshop
Bundler	esbuild
Stylesheet format	scss
SSR / SSG	No

Everything else — test runner, e2e, CI provider, Nx Cloud — press enter or skip. None of it changes the architecture we are building.

Hands-on

STEP 1 / 5 · CHECK

Two files decide everything

CHECKPOINT

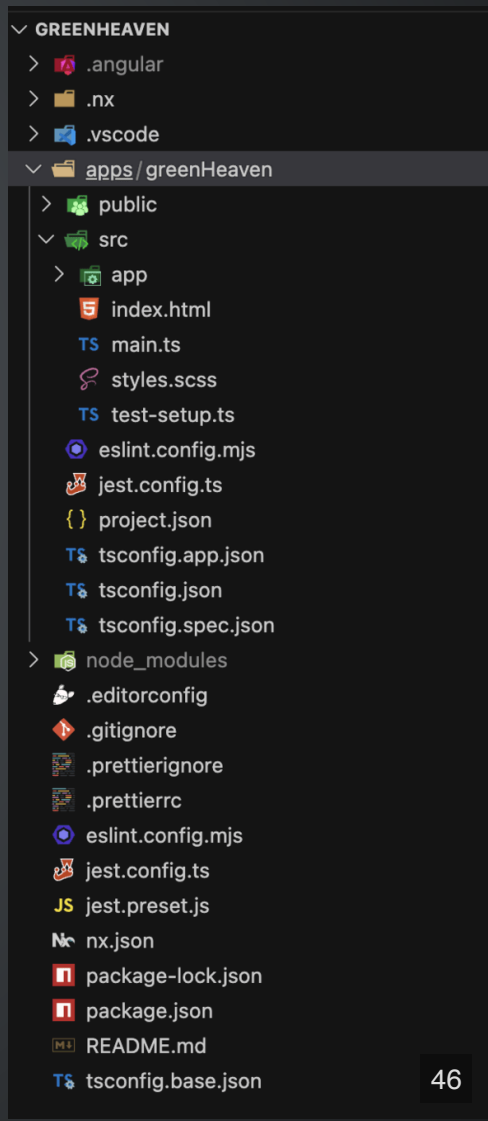
apps/workshop/project.json

you should see `"projectType": "application"`

nx.json

you should see plugins, targetDefaults, namedInputs

nx.json is workspace-wide policy. project.json is the per-project exception. If you are editing the second one often, the first one is wrong.



Hands-on

Install the Nx Console plugin

TASK

Extensions → search

nrwl.angular-console

WebStorm: same name in Plugins

Hands-on

Generate feature-catalogue with the Nx CLI

TASK

```
npx nx g @nx/angular:library \
  --name=feature-catalogue \
  --directory=libs/feature-catalogue
```

A library, not a folder. A folder gives you a path. A library gives you a name, a public API and a boundary the tooling can enforce.

The feature- prefix is not decoration — it is the type of the library, and later it becomes the tag we lint against.

Hands-on

STEP 3 / 5

What is this library actually called?

CHECKPOINT

```
// tsconfig.base.json
"paths": {
  "@workshop/feature-catalogue": ["libs/feature-catalogue/src/index.ts"]
}
```

The alias was generated for you, and it points at index.ts — never at the folder. That file is the library's front door, and this is the only place the real name is written down.

Open the file. Whatever your paths say is what you import in step 5 — not what is on my slide.

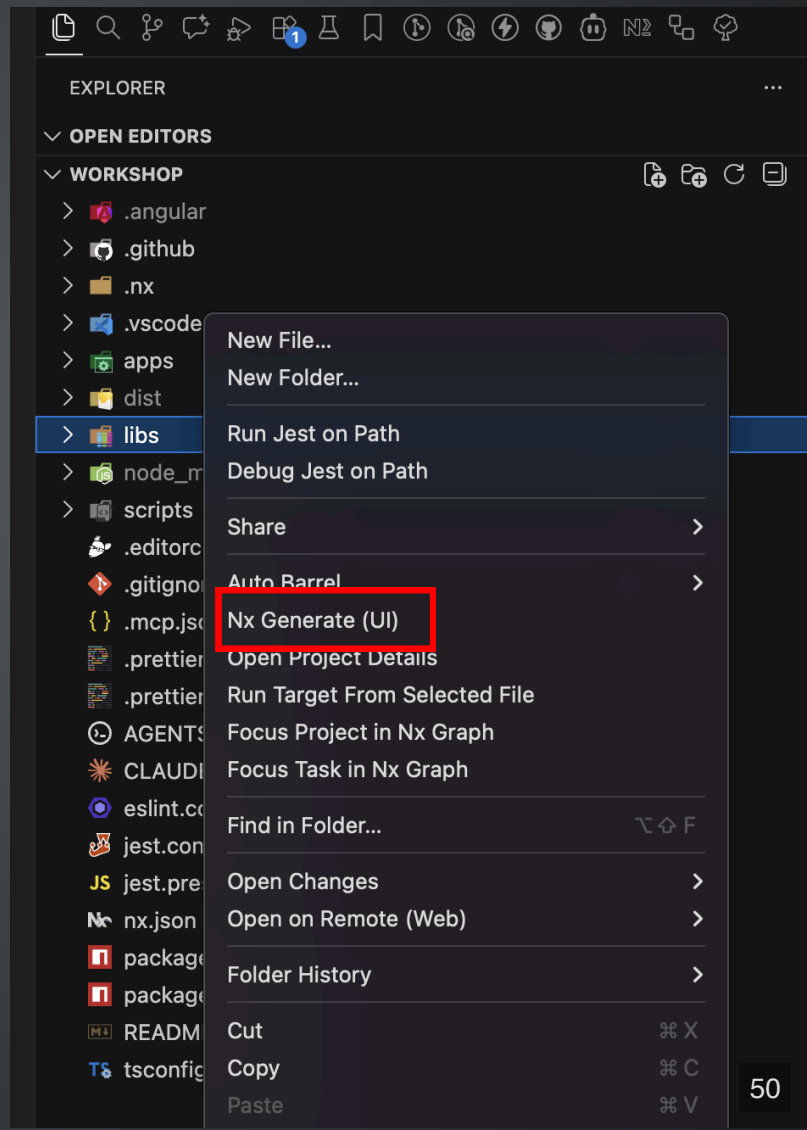
Hands-on

STEP 4 / 5

Generate feature-home with the Nx Console

TASK

1. right-click the libs folder
2. Nx Generate (UI)
3. pick @nx/angular - library



Four entries match "library". @angular-eslint and @schematics/angular are not the ones you want

Hands-on

Three fields, then Generate

HINT

Working Directory	{workspaceRoot} / libs
directory	feature-home
name	feature-home
everything else	leave as-is

Before you hit Generate, read the field list on the left of the form. That is the full contract of the generator.

Same result as step 3. Two doors into the same room — the CLI for speed, the UI for discovery.

Library
@nx/angular

Generate

Search... ⌘S

Working Directory: {workspaceRoot} / libs

directory
buildable
lazy
name
parent
publishable
routing
addModuleSpec
addTailwind
changeDetection
compilationMode
displayBlock
flat
importPath
inlineStyle
inlineTemplate
linter
prefix
selector
setParserOptionsProject
skipModule
skipSelector
skipTests
skipTsConfig
standalone
strict
style
tags
unitTestRunner
viewEncapsulation

directory*
A directory where the library is placed.
feature-home

buildable
Generate a buildable library.

lazy
Add `RouterModule.forChild` when set to true, and a simple array of routes when set to false.

name
The name of the library.
feature-home

parent
Path to the parent route configuration using `loadChildren` or `children`, depending on what `lazy` is set to.

publishable
Generate a publishable library.

routing
Add router configuration. See `lazy` for more information.

Hands-on

Two libraries, two public APIs

CHECKPOINT

```
// libs/feature-home/src/index.ts
export * from './lib/feature-home/feature-home';

// libs/feature-home/project.json
"projectType": "library"
```

If a file is not exported from index.ts, no other project can reach it.

Same check on feature-catalogue. Both should look identical apart from the name.

Hands-on

STEP 5 / 5

Wire the router

TASK

```ts
// apps/workshop/src/app/app.routes.ts
export const appRoutes: Route[] = [
  {
    path: '',
    loadComponent: () =>
      import('@workshop/feature-home').then((m) => m.FeatureHome),
    pathMatch: 'full',
  },
  {
    path: 'catalogue',
    loadComponent: () =>
      import('@workshop/feature-catalogue').then((m) => m.FeatureCatalogue),
  },
];
```

loadComponent means feature-catalogue never enters the home bundle.

Hands-on

STEP 5 / 5

Give yourself something to click

HINT

```
<!-- apps/workshop/src/app/app.html -->
<nav>
  <a routerLink="/">Home</a>
  <a routerLink="/catalogue">Catalogue</a>
</nav>
<router-outlet />

// app.ts  —  the line everyone forgets
imports: [RouterLink, RouterOutlet]
```

npx nx serve workshop

Delete the Nx welcome markup from app.html first, so there is nothing to squint at.

Hands-on

STEP 5 / 5 · CHECK

You are done when all three are true

CHECKPOINT

- ✓ / renders FeatureHome works!
- ✓ /catalogue renders FeatureCatalogue works!
- ✓ DevTools → Network: a new chunk arrives only when you click Catalogue

The third one is the point of the whole exercise. Two lazy chunks means the boundary is not a naming convention — it is in the build output.

What you just built

- 3 projects — one shell, two features
- 2 public APIs, one per library
- 2 lazy boundaries, visible in the network tab
- 0 dependencies between the two features

Nx Architecture & The Intelligent Graph

Types, Scopes, and the Shell Pattern

In this lesson

- Project Types (The Building Blocks)
- Grouping by Scope (The Domain)
- The Shell Library Pattern
- Naming Conventions
- Metadata & Tags (project.json)
- The Intelligent Graph
- The AI side of things

Project Types

The building blocks

- Feature (type:feature)

  The "Smart" logic and routing.

- UI (type:ui)

  Dumb components. Pure presentation

- Data-Access (type:data-access)

  State (Signals), NGXS/NGRX, HTTP Services

- Utility (type:util)

  Pure functions, helpers, validators



Feature Libraries

type:feature

The role

Smart components that orchestrate the UI components.

Contains

- Domain-specific logic
- Page Layouts
- Services / Facades

GreenHeaven example

feature-catalogue

What it does

Orchestrates the entire page

Rules

It's the only library type allowed to have routes.

UI Libraries

type:ui

The role

"Dumb" components focused on pure presentation.

Contains

- Buttons
- Cards
- Business UI Components

GreenHeaven example

ui-product-card

What it does

A UI unit of the entire application

Rules

It just receives input(s) and emits output(s)

Data-Access Libraries

type:data-access

The role

Managing State and API communication.

Contains

- Angular Signals/State
- HTTP Services
- NGXS/NGRX

GreenHeaven example

catalogue-data-access

What it does

Communicates with the HTTP API

Rules

It has no UI components

Utility Libraries

type:util

The role

Low-level, pure helper functions

Contains

- Date formatters
- Math helpers
- Custom validators

GreenHeaven example

_

What it does

Increases the reusability

Rules

Has zero dependencies on other library types.

Do I need a library?

Q: "Where do I put this file?"

Which project **owns** this responsibility?

Do I need a library?

- Q: "I should develop a new pageable component"

  A: Yes -> create a **feature** library

- Q: "I should develop a reusable UI element

  A: Yes -> create a **UI** library

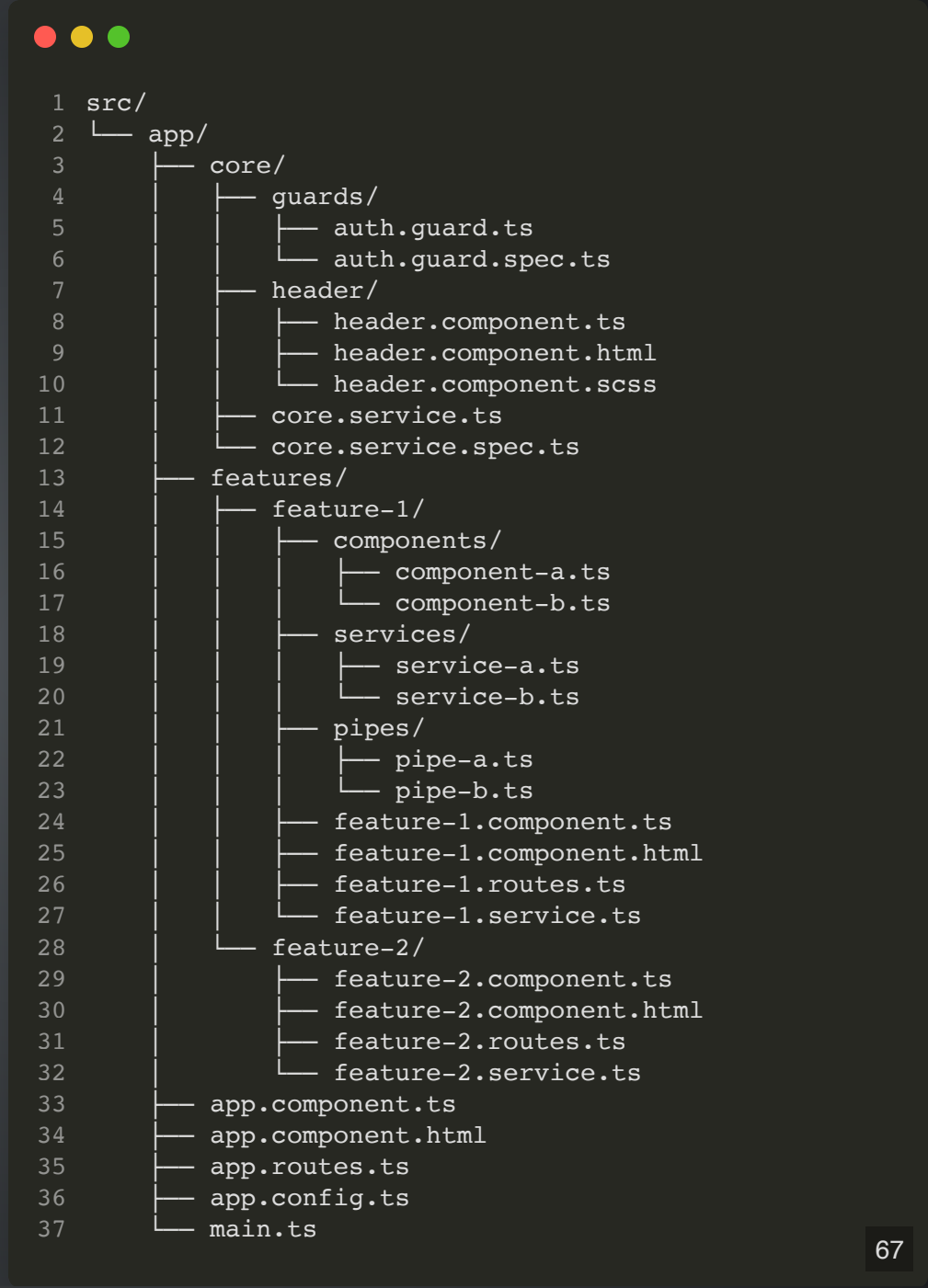
- Q: "I should develop a state management slice

  A: Yes -> create a **data-access** library

Q: "Do I need a library even for a component?"

A: No -> the component is an implementation detail

- The feature library is allowed to be "fat" internally
- Use the index.ts to hide the complexity



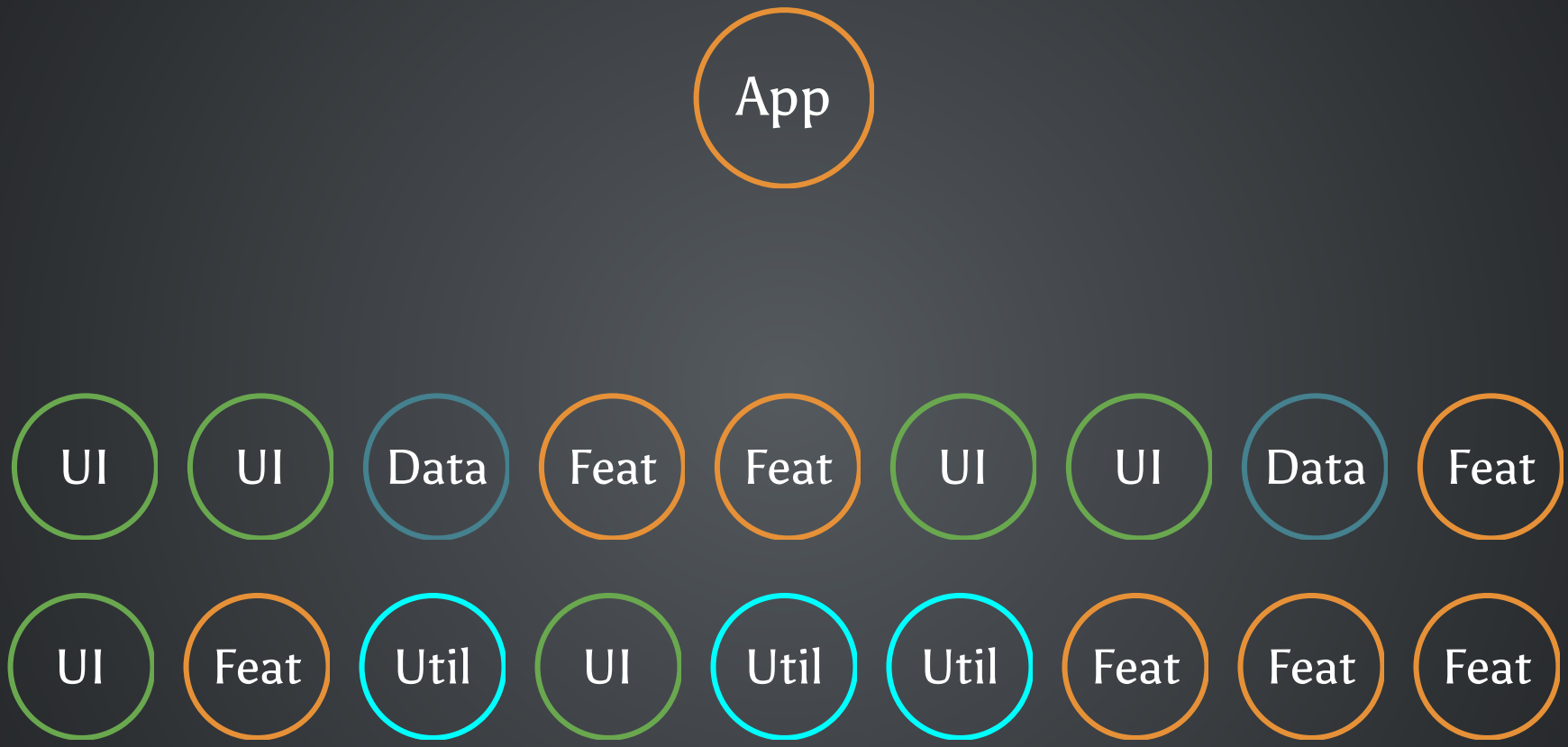
Scaling Up

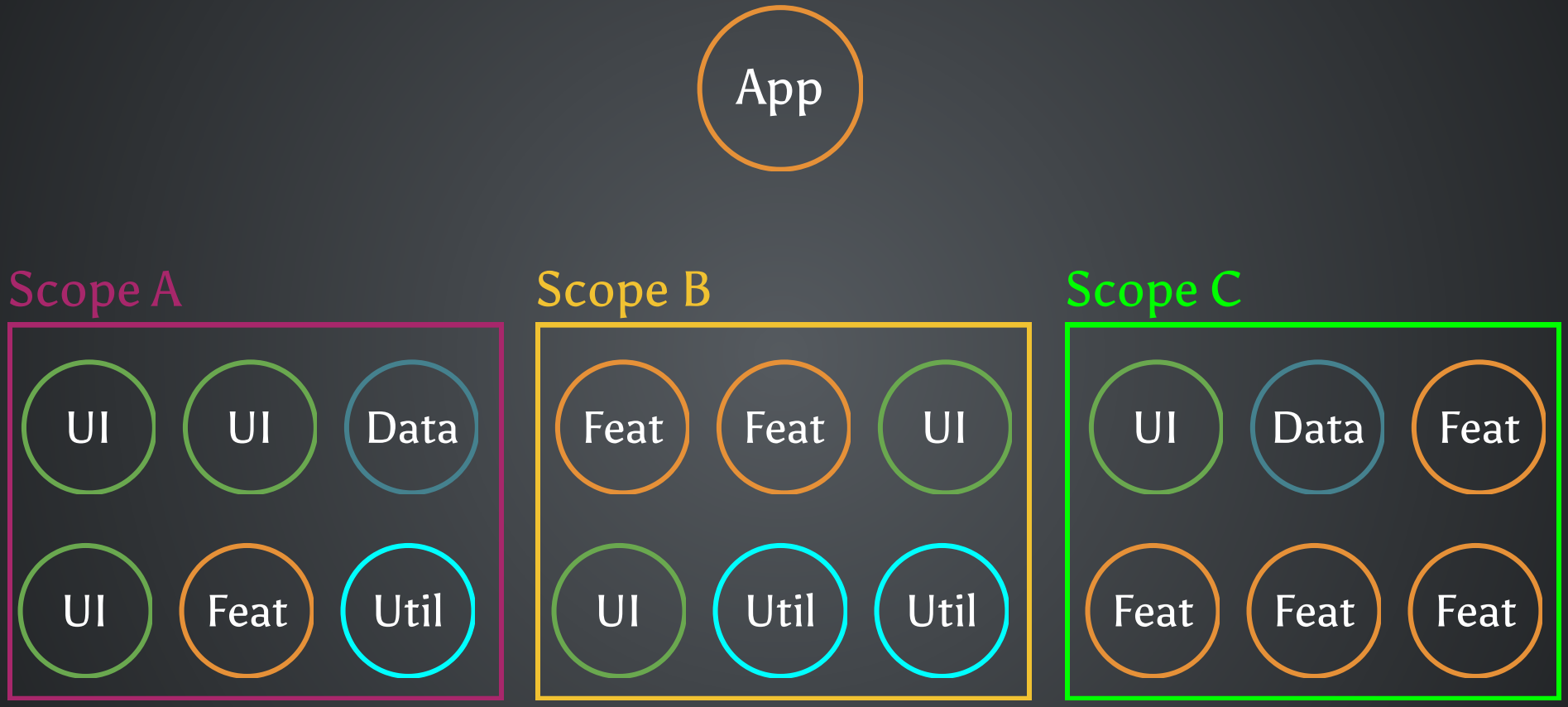
Organizing by Domain

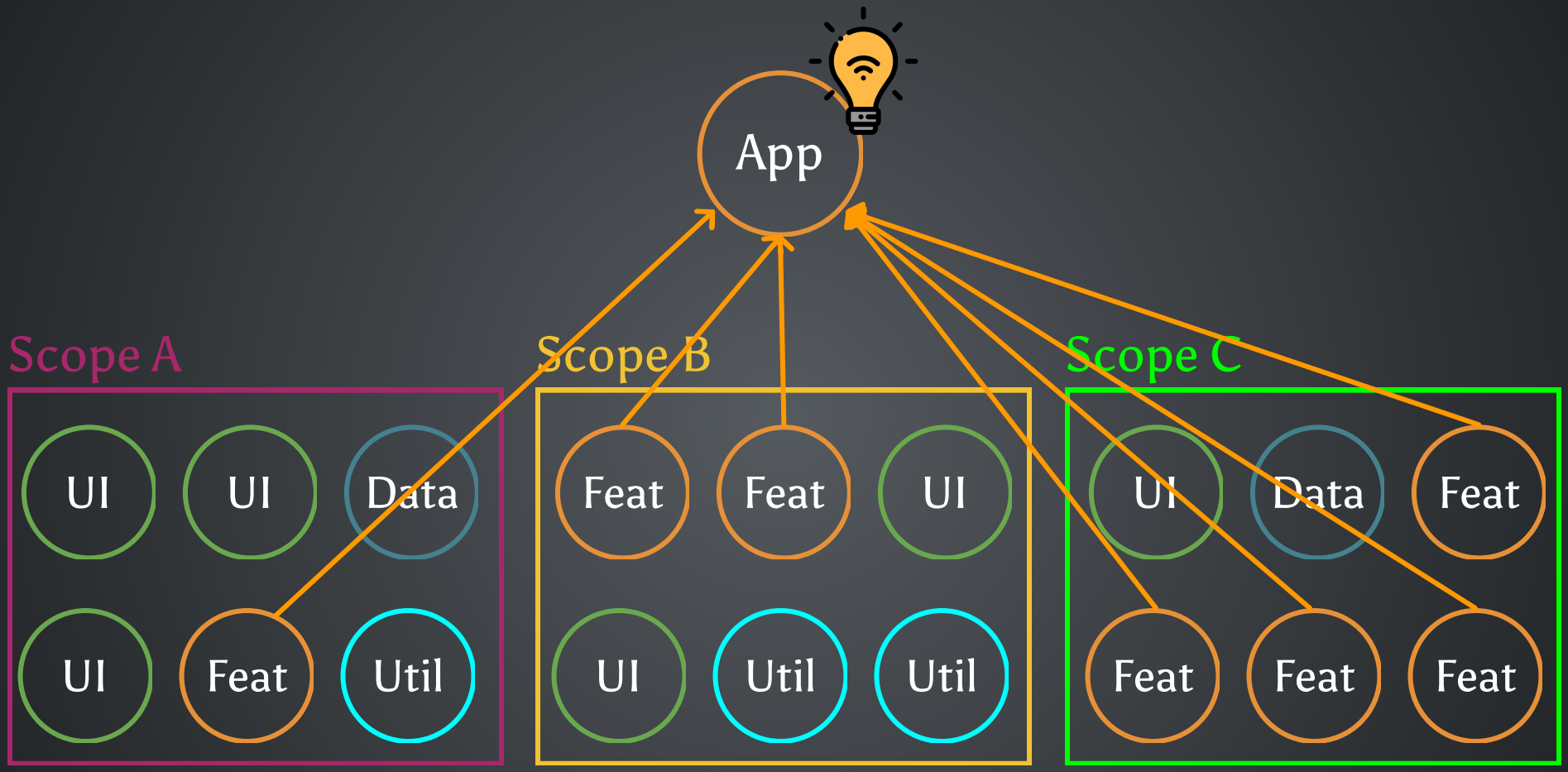
- **The problem**
  As the app grows, a flat **libs/** folder with **100+ libraries** becomes hard to find code and hard to see ownership

- **The solution**
  Group libraries by Scope (Domain)




App
UI
UI
Data
Feat
Feat
UI
UI
Data
Feat
UI
Feat
Util
UI
Util
Util
Feat
Feat
Feat


App
Scope A
UI
UI
Data
UI
Feat
Util
Scope B
Feat
Feat
UI
UI
Util
Util
Scope C
UI
Data
Feat
Feat
Feat
Feat


App
Scope A
UI
UI
Data
UI
Feat
Util
Scope B
Feat
Feat
UI
UI
Util
Util
Scope C
UI
Data
Feat
Feat
Feat
Feat

The Shell Library

The Entry Point for a domain

- The problem

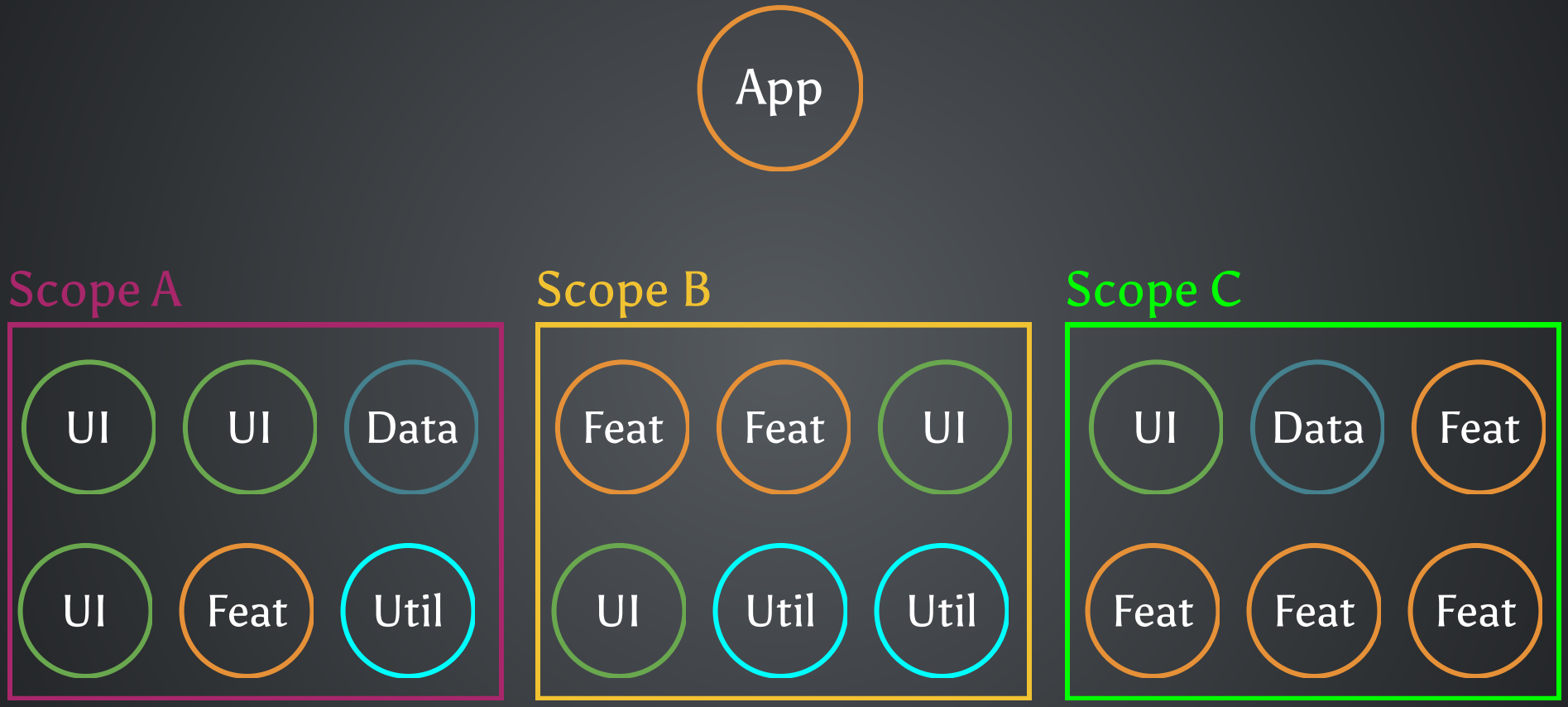
  If the App knows about "List" and "Details", the App is too smart

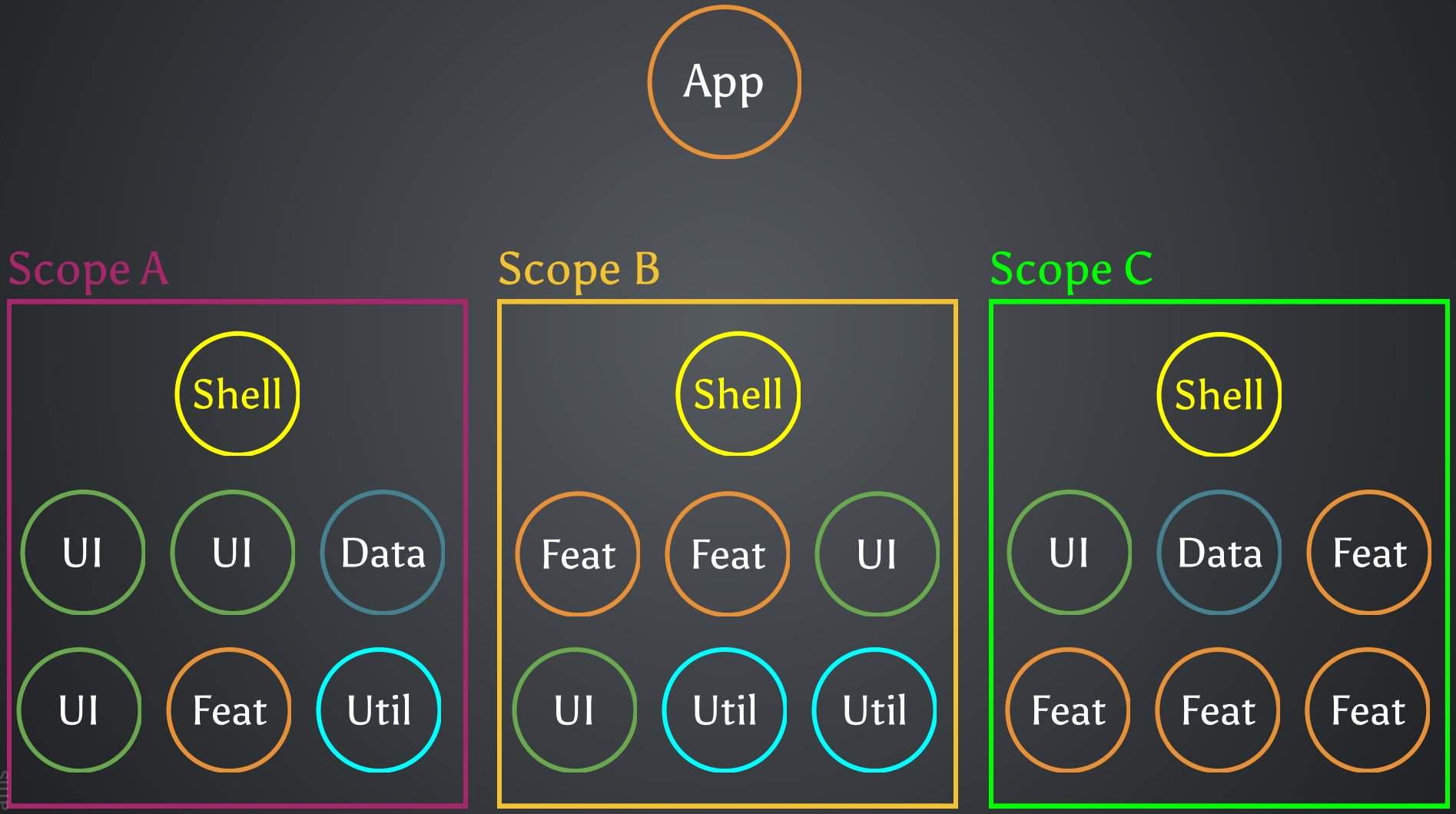
- The solution

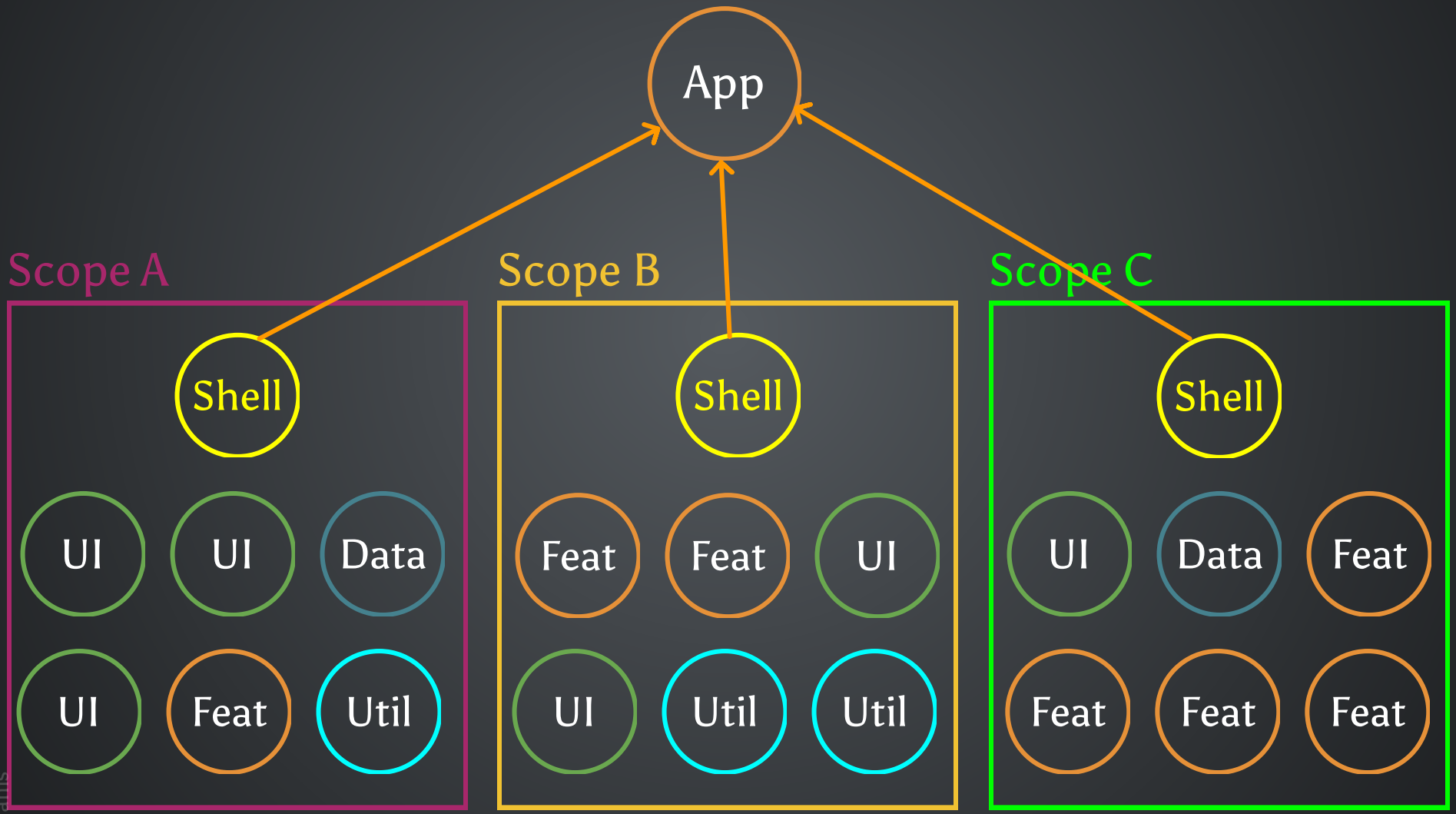
  The App knows the Shell.

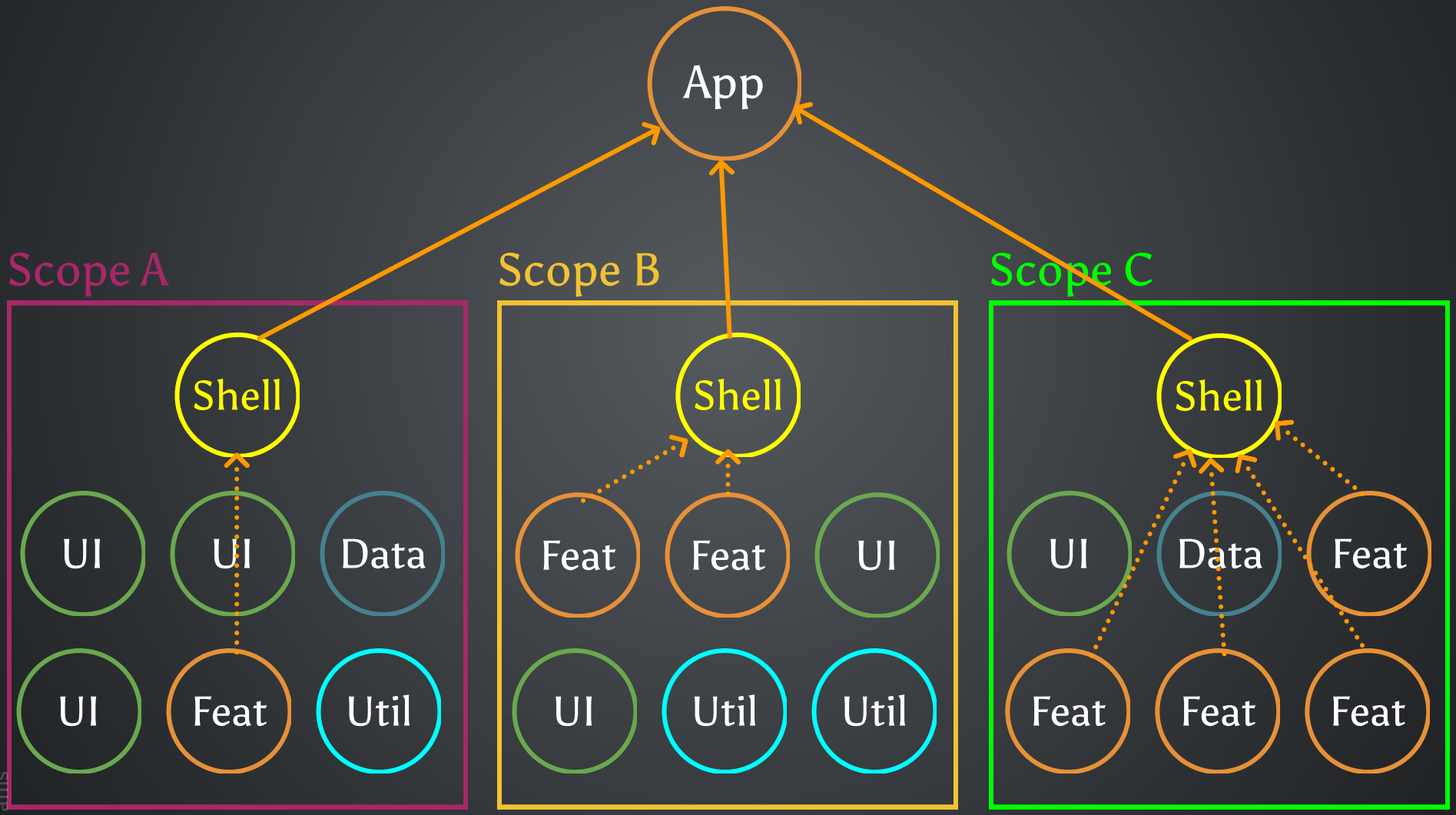
  The Shell sets up the child routes for the domain




App
Scope A
UI
UI
Data
UI
Feat
Util
Scope B
Feat
Feat
UI
UI
Util
Util
Scope C
UI
Data
Feat
Feat
Feat
Feat


App
Scope A
Shell
UI
UI
Data
UI
Feat
Util
Scope B
Shell
Feat
Feat
UI
UI
Util
Util
Scope C
Shell
UI
Data
Feat
Feat
Feat
Feat


App
Scope A
Shell
UI
UI
Data
UI
Feat
Util
Scope B
Shell
Feat
Feat
UI
UI
Util
Util
Scope C
Shell
UI
Data
Feat
Feat
Feat
Feat


App
Scope A
Shell
UI
UI
Data
UI
Feat
Util
Scope B
Shell
Feat
Feat
UI
UI
Util
Util
Scope C
Shell
UI
Data
Feat
Feat
Feat
Feat

The Shell Library

It orchestrates how the work is started and displayed.

- Routing & Lazy Loading

  Defines the top-level routes that the main app loads.

- Guards and Resolvers

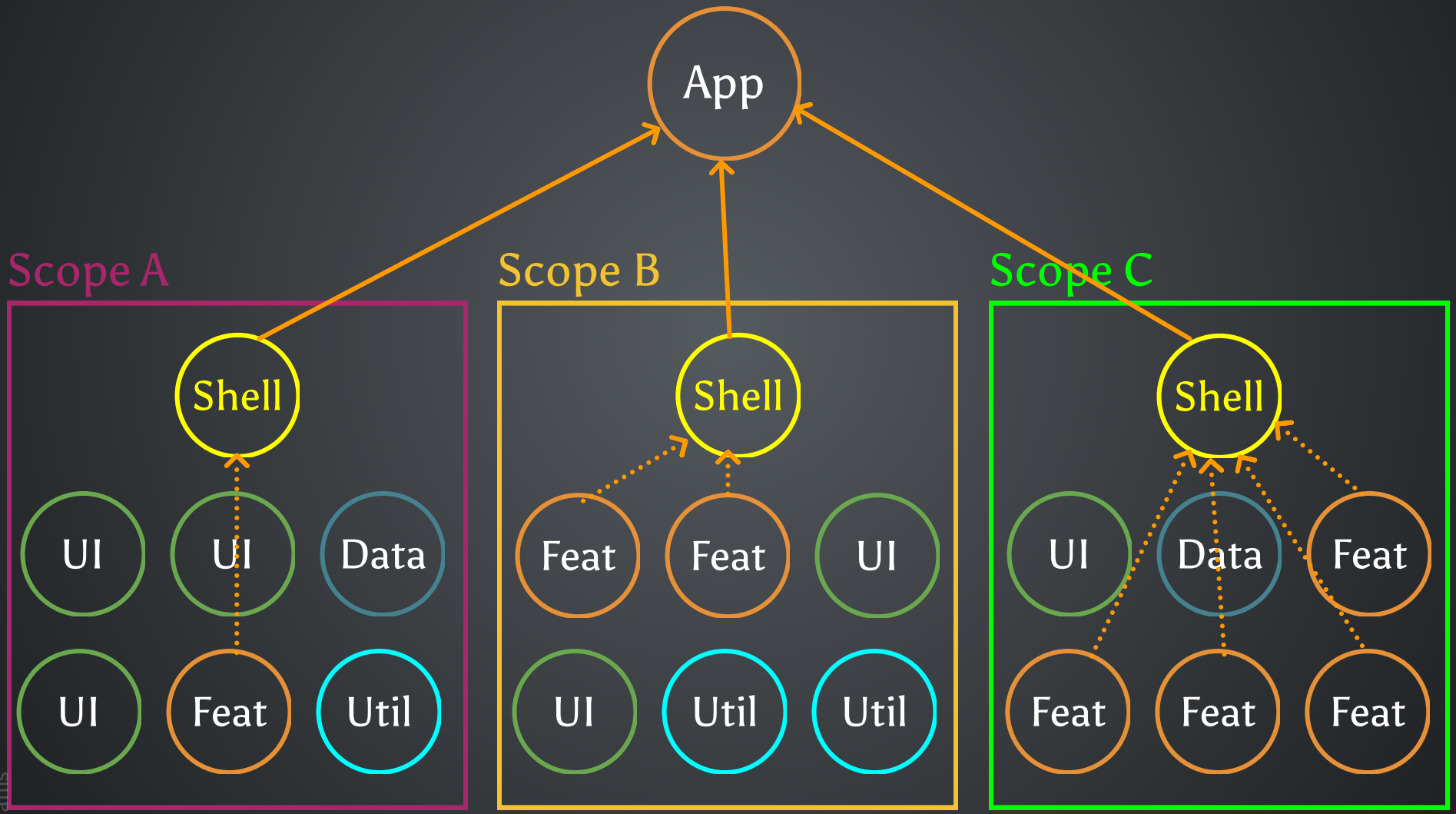
  Handle the auth guards and route resolvers

- State Bootstrapping

  Provides NGXS/NgRx state slices for the entire domain.

- Wrapper Layouts

  A specific sidebar, header, or footer that only appears within this feature module


App
Scope A
Shell
UI
UI
Data
UI
Feat
Util
Scope B
Shell
Feat
Feat
UI
UI
Util
Util
Scope C
Shell
UI
Data
Feat
Feat
Feat
Feat

The Shell Library

The Entry Point for a domain

```ts
// src/app/routes.ts

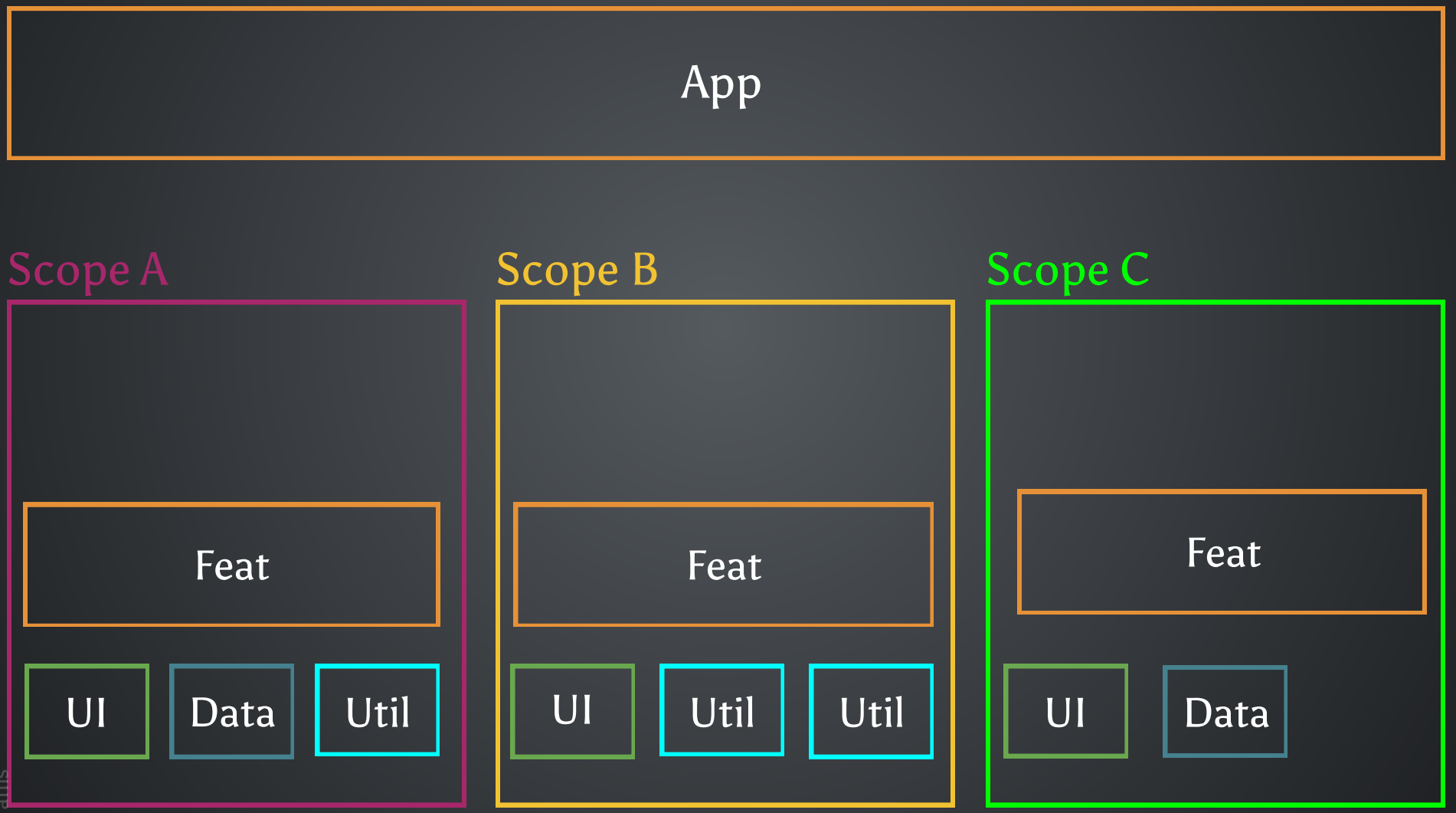
export const appRoutes: Route[] = [
  { path: '', pathMatch: 'full', redirectTo: 'home' },
  {
    path: 'catalogue',
    loadComponent: () =>
      import('@workshop/catalogue-feature-catalogue-list').then(
        (m) => m.CatalogueComponent
      ),
  },
  {
    path: 'catalogue/:id',
    loadComponent: () =>
      import('@workshop/catalogue-feature-catalogue-details').then(
        (m) => m.CatalogueDetailsComponent
      ),
  },
  { path: '**', redirectTo: '' },
];
```

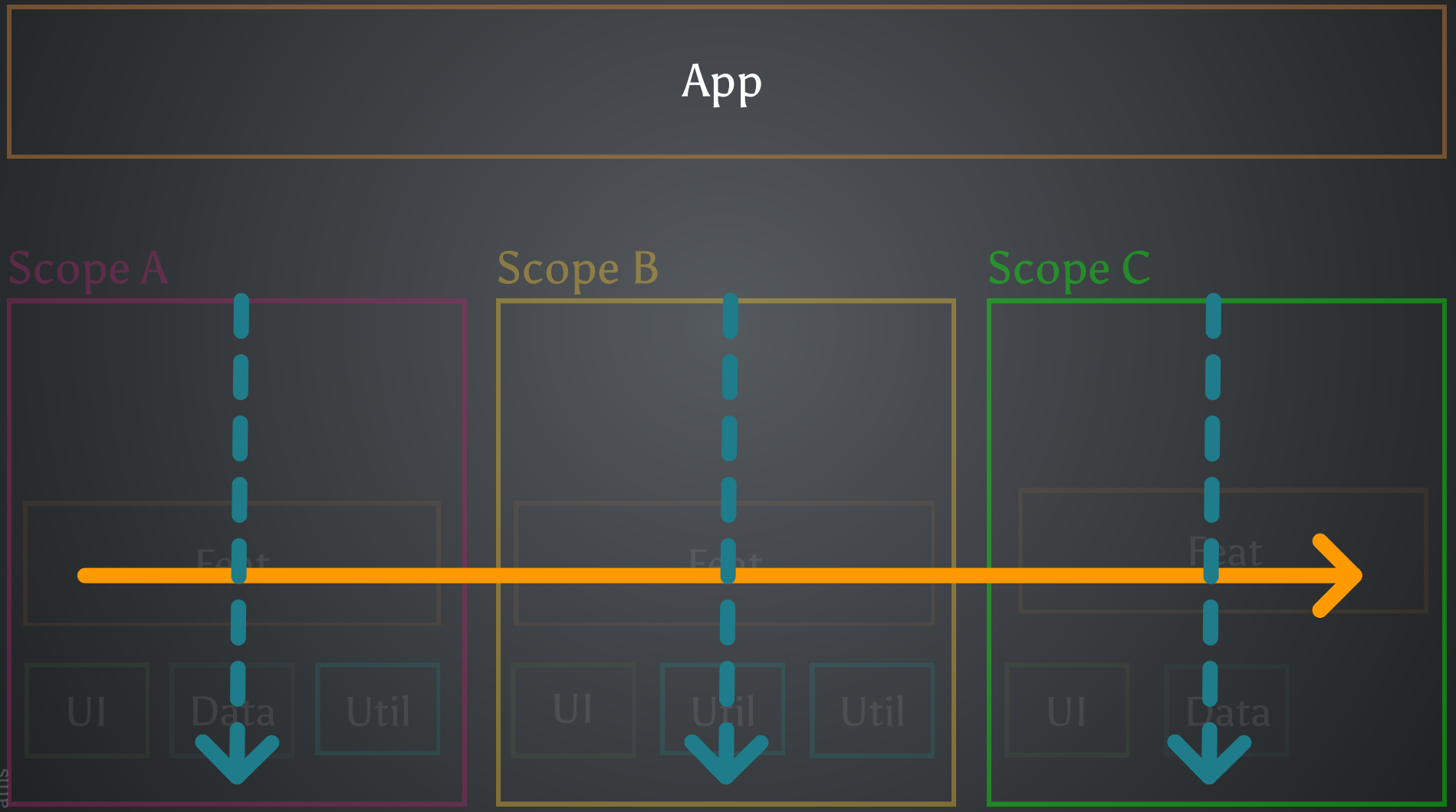
The Shell Library

The Entry Point for a domain

```ts
// src/app/routes.ts

export const appRoutes: Route[] = [
  { path: '', pathMatch: 'full', redirectTo: 'home' },
  {
    path: 'catalogue',
    loadChildren: () =>
      import('@workshop/catalogue-feature-shell').then(
        (m) => m.catalogueRoutes
      ),
  },
  { path: '**', redirectTo: '' },
];
```


App
Scope A
Feat
UI
Data
Util
Scope B
Feat
UI
Util
Util
Scope C
Feat
UI
Data


App
Scope A
Scope B
Scope C
Feat
Feat
Feat
UI
Data
Util
UI
Util
Util
UI
Data

Metadata and Tags

How does Nx know that feature-list is a "Feature"? We tell it.



type: segregates the **technical** responsibility



scope: segregates the **functional** responsibility

Metadata and Tags

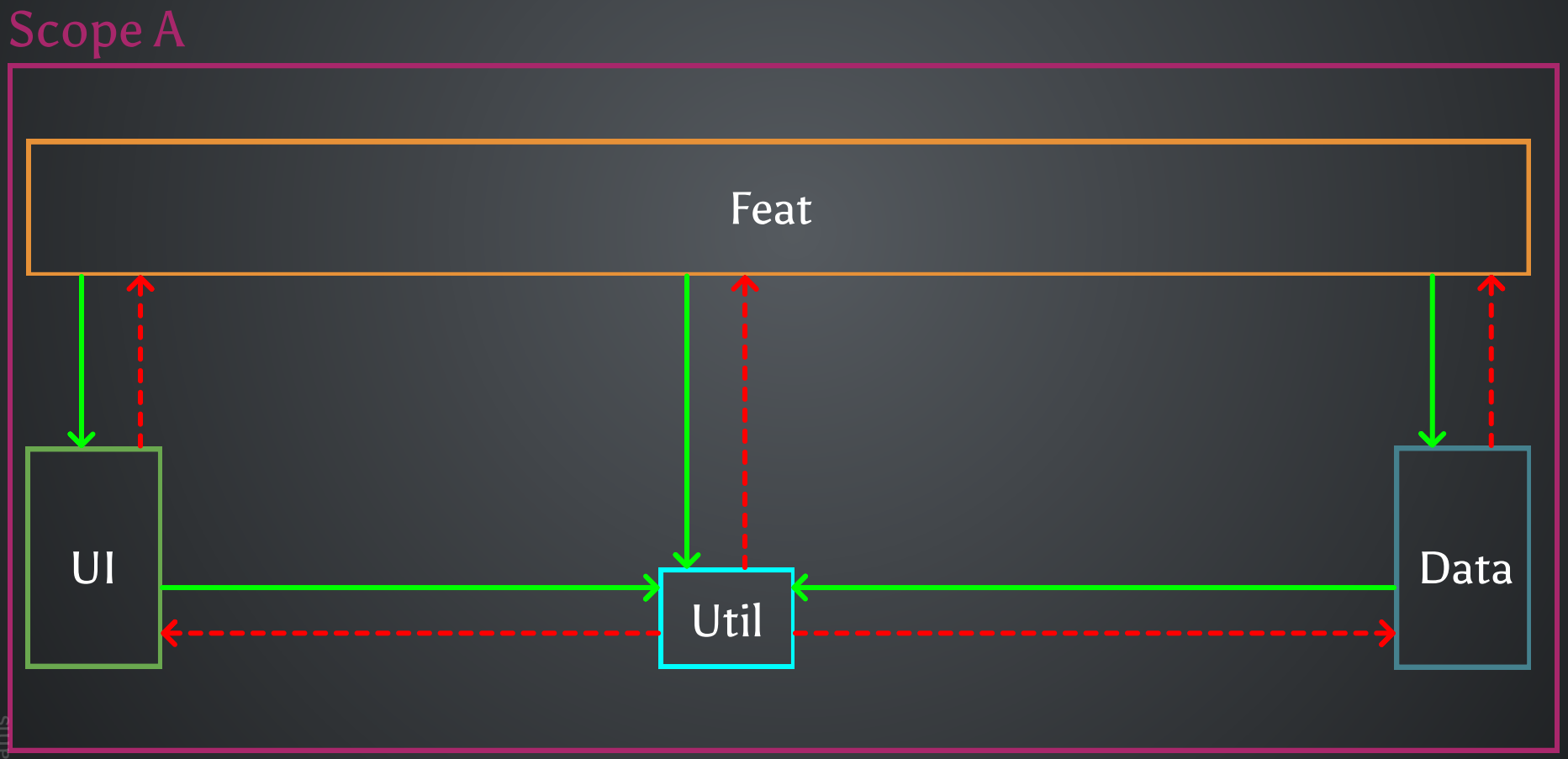
How does Nx know that feature-list is a "Feature"? We tell it.

```
// project.json

{
  "name": "catalogue-feature-catalogue-details",
  "$schema": "../../../node_modules/nx/schemas/project-schema.json",
  "sourceRoot": "libs/catalogue/feature-catalogue-details/src",
  "prefix": "lib",
  "projectType": "library",
  "tags": ["scope:catalogue", "type:feature"],
  "targets": {
    "test": { ... },
    "lint": { ... }
  }
}
```

Metadata and Tags

How does Nx know that feature-list is a "Feature"? We tell it.



Metadata and Tags

How does Nx know that feature-list is a "Feature"? We tell it.

```
// eslint.config.mjs

{
    "sourceTag": "type:feature",
    "onlyDependOnLibsWithTags": [
        "type:ui",
        "type:util",
        "type:feature",
        "type:data-access"
    ]
},
{
    "sourceTag": "type:ui",
    "onlyDependOnLibsWithTags": [
        "type:util",
        "type:ui",
    ]
}
```

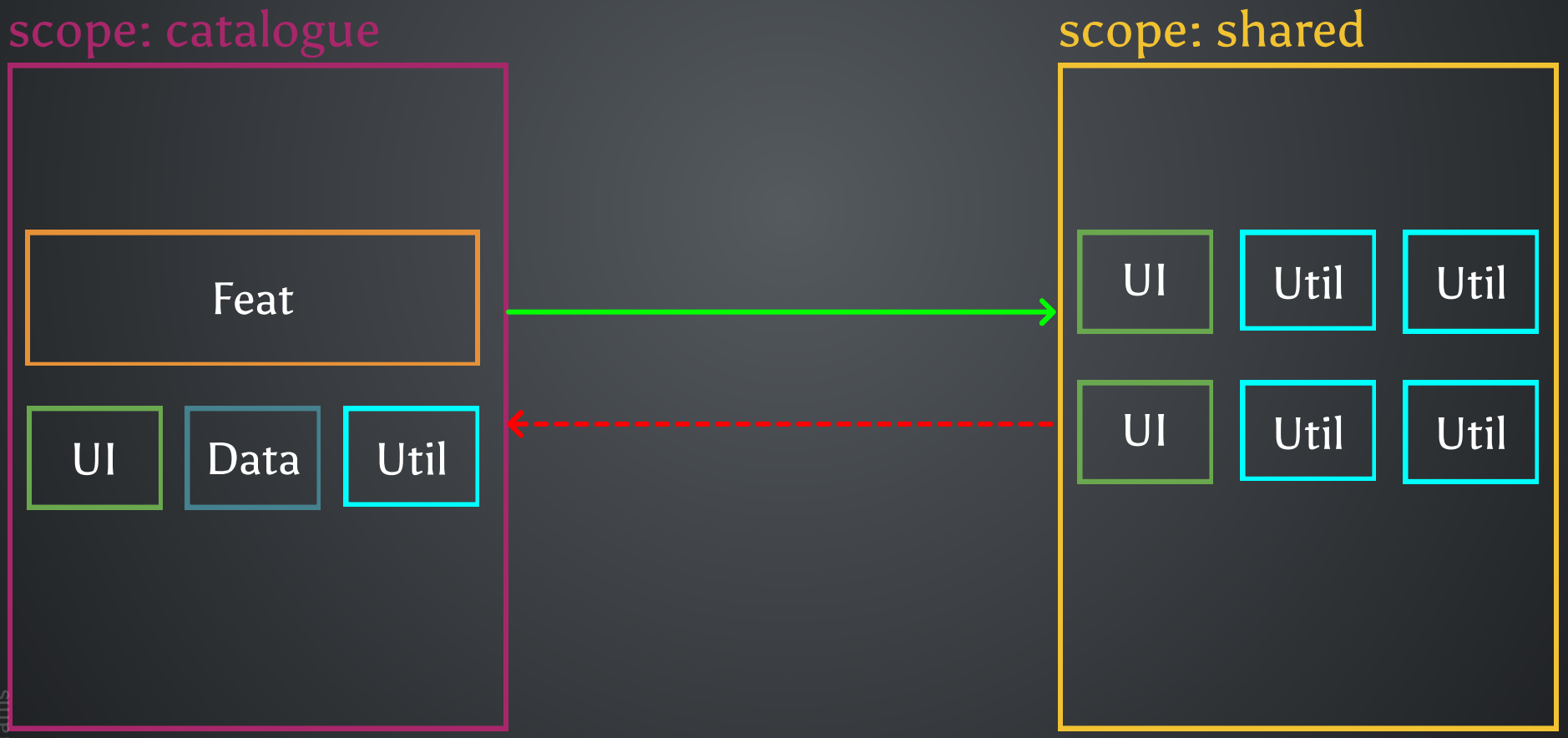
Metadata and Tags

How does Nx know that feature-list is a "Feature"? We tell it.

```
A project tagged with "type:ui" can only depend on libs tagged with "type:model", "type:util",
"type:ui" eslint(@nx/enforce-module-boundaries)
+ module "/Users/profanis/projects/temp/angular.love/workshop/libs/catalogue/feature-catalogue-list/src/index"
View Problem (⌥F8)  Quick Fix... (⌘.)  Fix (⌘I)
import { CatalogueLocalState } from '@workshop/catalogue-feature-catalogue-list';
You, 8 minutes ago | 1 author (You)
@Component(obj: {
  selector: 'lib-ui-header',
```

Metadata and Tags

How does Nx know that feature-list is a "Feature"? We tell it.



Scaling Up

Organizing by Domain

Within a single domain, we apply our Taxonomy to keep the internal structure clean

```
libs/
└── catalogue/                          <-- The Domain Boundary
    ├── data-access/                    🧠 State & API
    ├── feature-catalogue-details/      📄 Catalogue Details Page
    ├── feature-catalogue-list/         📄 Catalogue List Page
    ├── feature-shell/                  🐚 Entry Point & Internal Routing
    └── types/                          🐚 Domain specific types
```

Naming Convention

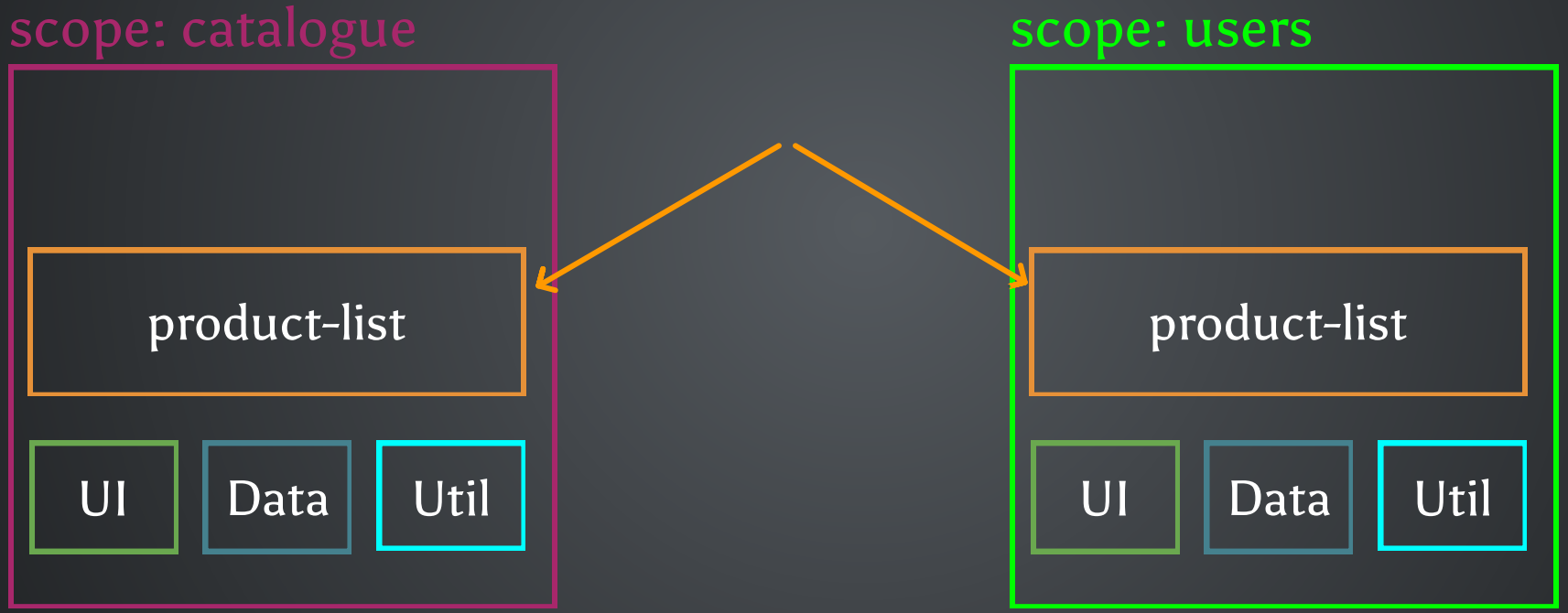
Naming consistency is key for tools and humans

```
// project.json

{
  "name": "catalogue-feature-catalogue-details",
  "$schema": "../../../node_modules/nx/schemas/project-schema.json",
  "sourceRoot": "libs/catalogue/feature-catalogue-details/src",
  "prefix": "lib",
  "projectType": "library",
  "tags": ["scope:catalogue", "type:feature"],
  "targets": {
    "test": { ... },
    "lint": { ... }
  }
}
```

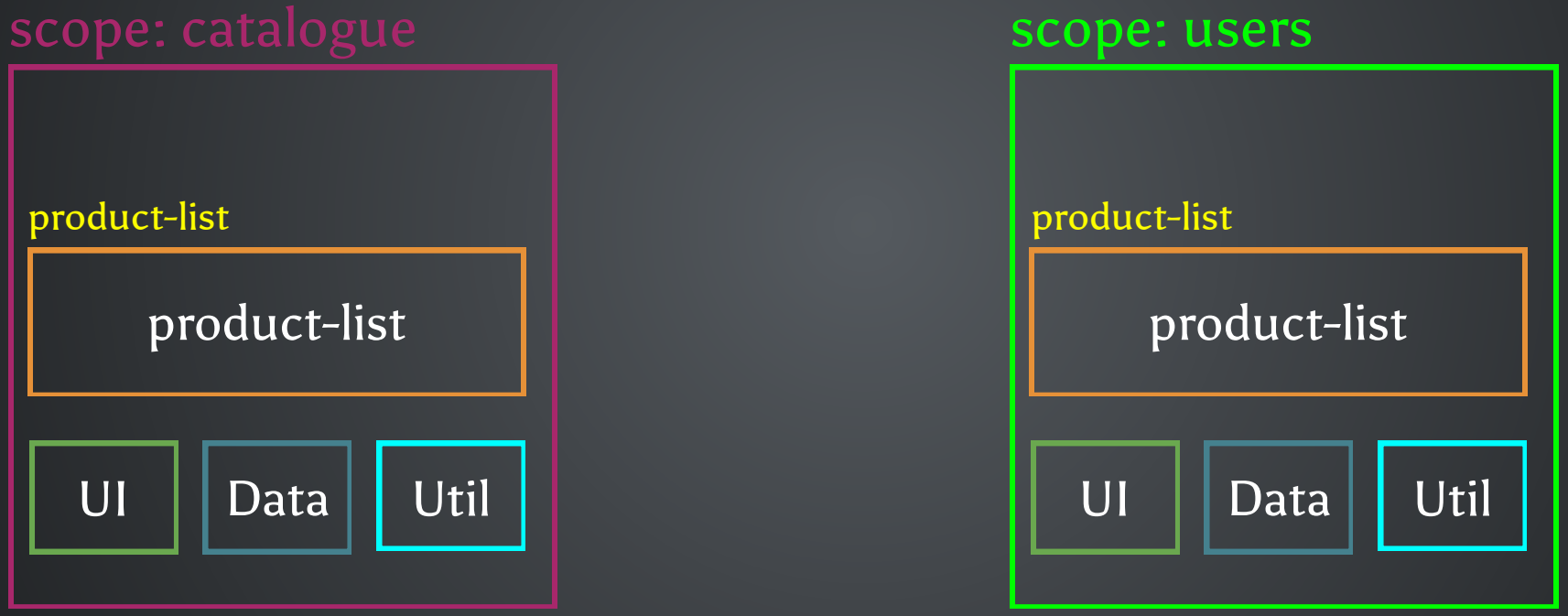
Naming Convention

Naming consistency is key for tools and humans



Naming Convention

Naming consistency is key for tools and humans



Naming Convention

`scope-type-identifier*`

catalogue	feature	catalogue-list
shared	ui	header
auth	data-access	users*

Naming Convention

`scope`-`type`-identifier*

catalogue-feature-catalogue-list

shared-ui-header

auth-data-access-users

Naming Convention

scope-type-identifier*

catalogue-feature-catalogue-list

shared-ui-header

auth-data-access-users

```
// project.json

{
  "name": "catalogue-feature-catalogue-list",
  "$schema": "../../../node_modules/nx/schemas/project-schema.json",
  "sourceRoot": "libs/catalogue/feature-catalogue-list/src",
  "prefix": "lib",
  "projectType": "library",
  ....
}
```

Naming Convention

`scope`-`type`-identifier*

catalogue-feature-catalogue-list → @org/catalogue-feature-catalogue-list

shared-ui-header → @org/shared-ui-header

auth-data-access-users → @org/auth-data-access-users

Naming Convention

scope-type-identifier*

catalogue-feature-catalogue-list → @org/catalogue-feature-catalogue-list

shared-ui-header → @org/shared-ui-header

auth-data-access-users → @org/auth-data-access-users

```
// tsconfig.base.json

{
  "compilerOptions": {
    "paths": {
      "@org/catalogue-feature-catalogue-list": [
        "libs/catalogue/feature-catalogue-list/src/index.ts"
      ],
      "@org/shared-ui-header": [
        "libs/shared/ui/ui-header/src/index.ts"
      ],
      "@org/auth-data-access-users": [
        "libs/auth/data-access/src/index.ts"
      ]
    }
  }
}
```

The Intelligent Graph

Visualize your architecture

- `nx graph`

  Defines the top-level routes that the main app loads.

- `nx graph --affected`

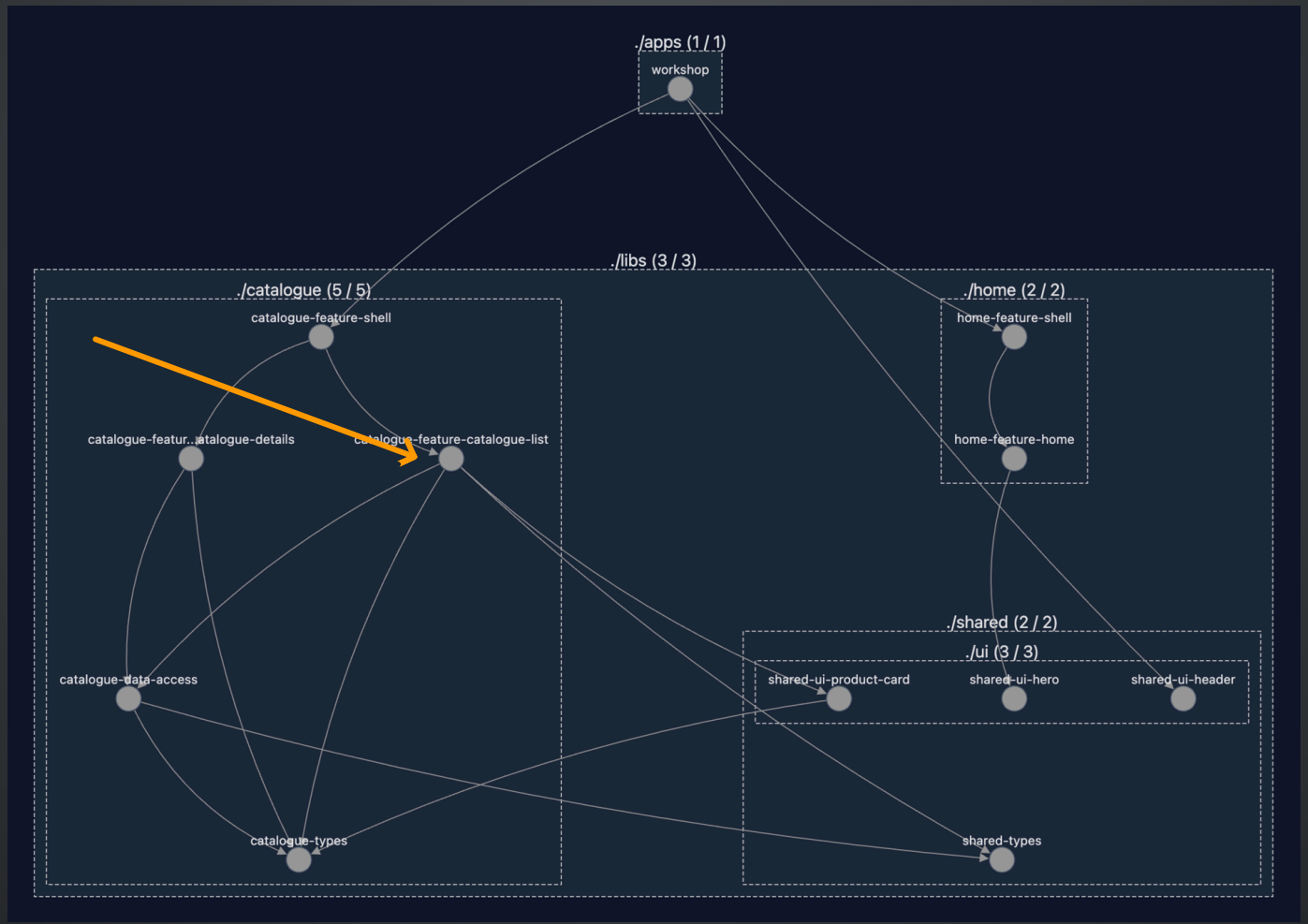
  Visualize only the affected by your changes projects

- `nx affected -t build --base=origin/main~1 --head=origin/main`

  Visualize only the affected by your changes projects - CI

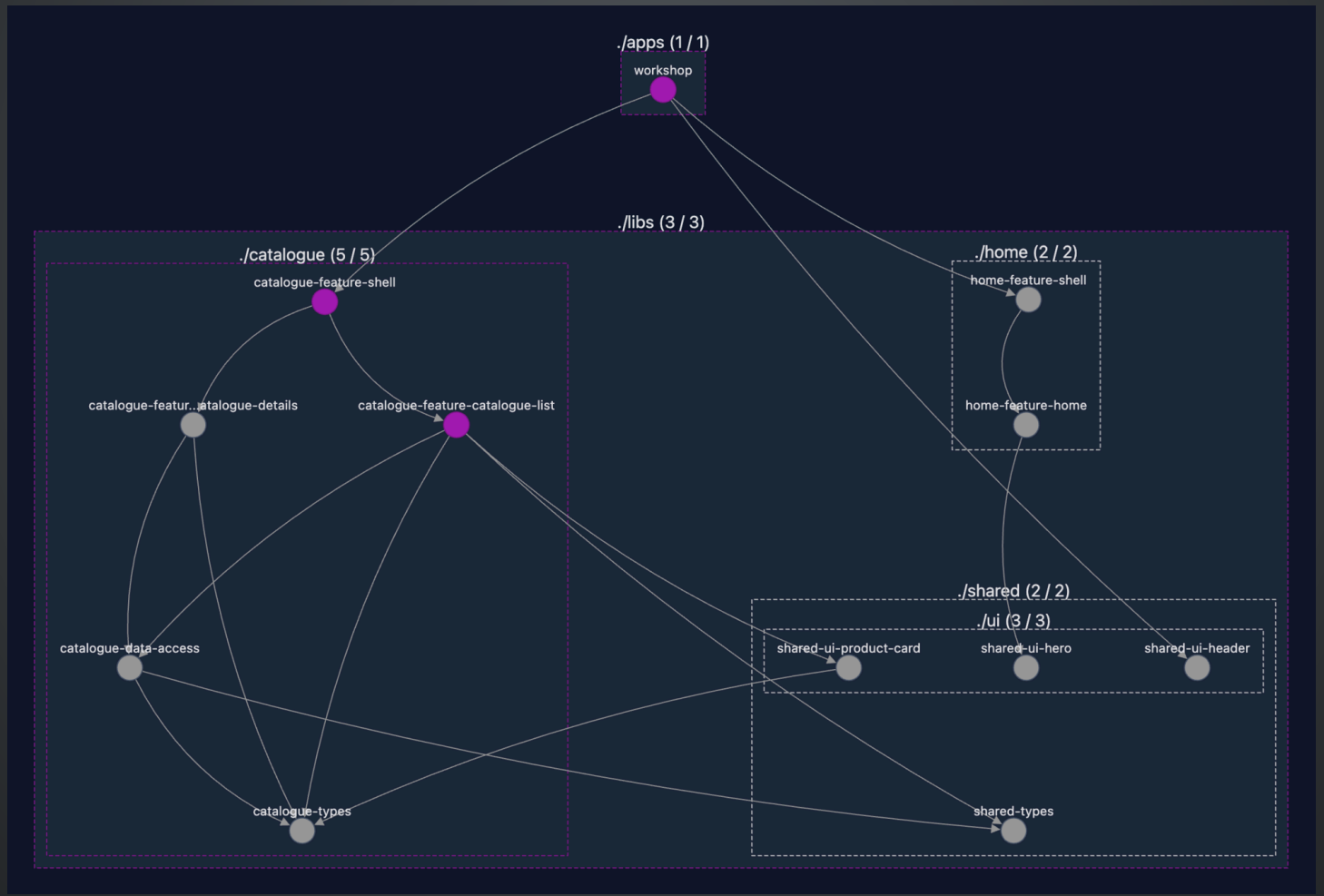
The Intelligent Graph

Visualize if your architecture is messy > nx graph



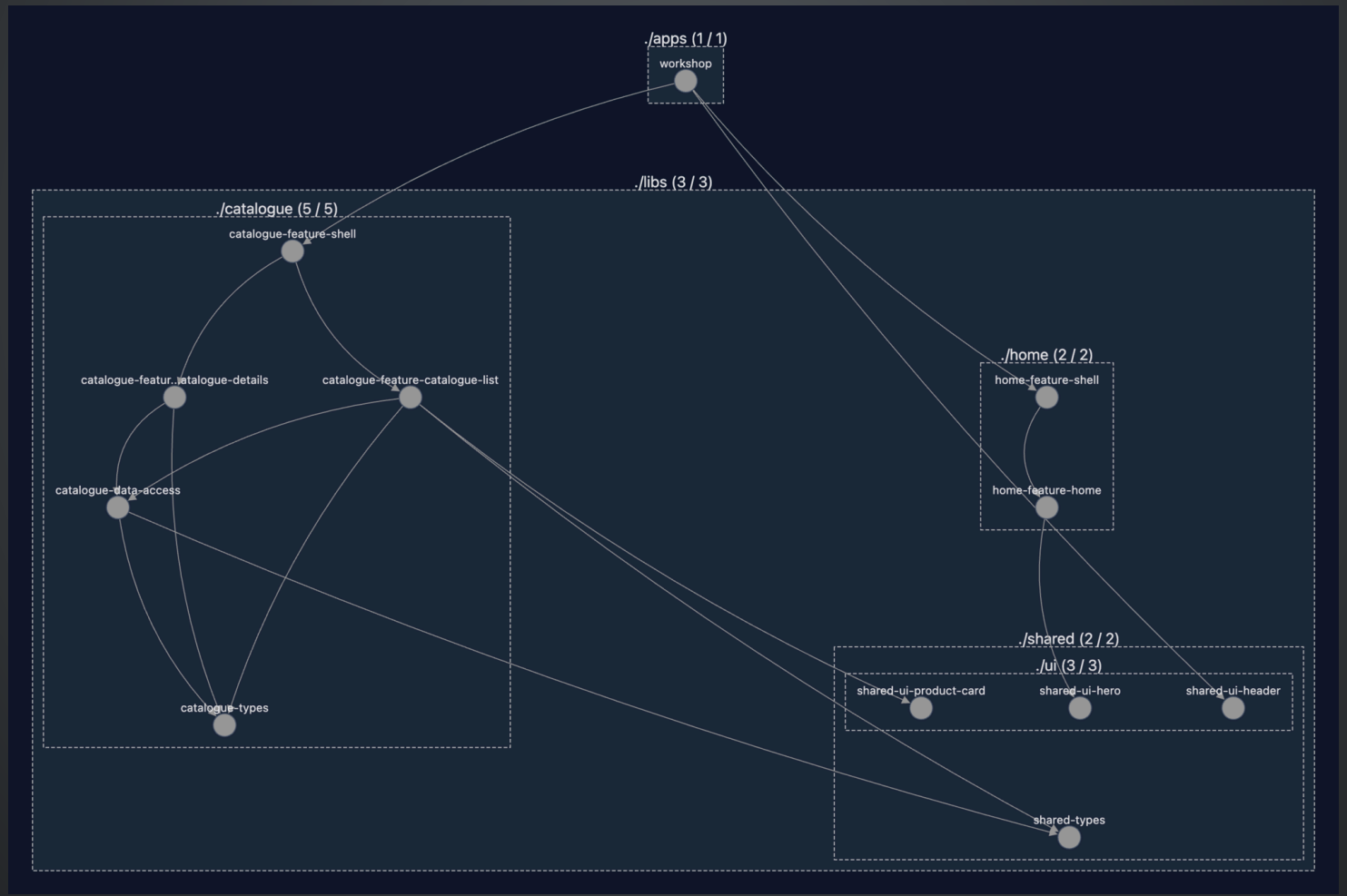
The Intelligent Graph

Visualize if your architecture is messy > nx graph --affected



The Intelligent Graph

Visualize if your architecture is messy > detect an error



Hands-on



Same shop, grown up

What changed while you were at the break

SECTION 1

flat libs/

2 libraries

no tags

you built it from nothing

SECTION 2

grouped by domain

7 libraries across 3 scopes — you make it 11

scope: and type: on every one

it exists — you extend it

Same product: the GreenHeaven plant shop. The Nx workspace is called workshop, so every import you write starts with @workshop/

Hands-on

Six steps

1. Shell library for the catalogue
2. Shell library for the home — on your own
3. Catalogue data-access library
4. Shared ui library for the product card
5. Tag everything, then let lint check your work
6. Find the boundary violation in the graph

Branch: lab/one · you keep this workspace for the rest of the section

Fell behind? The next lab starts from a branch that already contains all of this. Nobody gets stranded.

Hands-on

Get the workspace

TASK

```
git clone --branch lab/one https://github.com/profanis/nx-signals-workshop.git
cd nx-signals-workshop/
npm i
```

Already there: the workshop app and 7 libraries — catalogue list, catalogue details, catalogue types, home, shared types, ui-header, ui-hero.

You will add 4: two shell libraries, one data-access library, one ui library — plus the tags that hold the whole thing together.

Run npx nx graph once now. That picture is the "before".

Hands-on

A shell library for the catalogue

TASK

```
npx nx g @nx/angular:library \
  --directory=libs/catalogue/feature-shell \
  --name=catalogue-feature-shell
```

Then delete the generated component, and create lib.routes.ts next to it.

Name follows scope-type-identifier

Directory is the path, project name is flat and globally unique.

Hands-on

Fill in the two routes

HINT

```
// libs/catalogue/feature-shell/src/lib/lib.routes.ts

export const catalogueRoutes: Route[] = [
  {
    path: '',         // the list page
    // ??
  },
  {
    path: ':id',      // the details page
    // ??
  },
```

Both feature libraries already exist. You are only pointing at them.

Their import paths are in tsconfig.base.json — look there rather than guessing.

Hands-on

STEP 1 / 6

Hand the domain to the shell

HINT

```
// apps/workshop/src/app/app.routes.ts

{
  path: 'catalogue',
  loadChildren: () =>
    import('PATH TO FEATURE SHELL').then((m) => m.catalogueRoutes),
},
```

Find the real path in tsconfig.base.json → paths

loadChildren, not loadComponent. The app hands over the whole branch and stops caring what is inside it — that is the entire point of a shell.

Hands-on

The catalogue shell

CHECKPOINT

```ts
export const catalogueRoutes: Route[] = [
  {
    path: '',
    loadComponent: () =>
      import('@workshop/catalogue-feature-catalogue-list').then((m) => m.CatalogueComponent),
  },
  {
    path: ':id',
    loadComponent: () =>
      import('@workshop/catalogue-feature-catalogue-details')
        .then((m) => m.CatalogueDetailsComponent),
  },
];
```

- ✓ /catalogue shows the list, /catalogue/3 shows the details
- ✓ app.routes.ts no longer names any catalogue page

Hands-on

Now the home domain — on your own - A shell library for the home

TASK

Same four moves. No hand-holding this time.

1. Generate — use the Nx Console, and work out the project name yourself
2. Delete the generated component
3. Add lib.routes.ts exporting homeRoutes
4. Point app.routes.ts at it with loadChildren

Directory: libs/home/feature-shell · the name follows from scope-type-identifier

Hands-on

Two domains, two shells

CHECKPOINT

- ✓ Your project is named home-feature-shell
- ✓ /home and /catalogue both still work
- ✓ app.routes.ts contains exactly two loadChildren entries and nothing else

That third one is the win. The app no longer knows any page in the product — only two domains. Adding a third domain is now one line.

Hands-on

Get the API service out of the feature

TASK

```
npx nx g @nx/angular:library \
  --directory=libs/catalogue/data-access \
  --name=catalogue-data-access
```

move

libs/catalogue/feature-catalogue-list/src/lib/services/products.api.ts

into the new library, then export it from its index.ts

Anything that talks to HTTP or holds state is data-access — never a feature's private detail. Otherwise the details page can't reuse it without importing the list page.

Hands-on

Moving a file breaks three things

HINT

1. The new library's index.ts — nothing is public until you export it there
2. The old relative import in feature-catalogue-list — becomes @workshop/catalogue-data-access
3. The now-empty services/ folder — delete it

```
// libs/catalogue/data-access/src/index.ts
export * from './lib/products.api';   // wherever you dropped it
```

Hands-on

The product card belongs to everyone

TASK

```
npx nx g @nx/angular:library \
  --directory=libs/shared/ui/ui-product-card \
  --name=shared-ui-product-card
```

move

libs/catalogue/feature-catalogue-list/src/lib/components/product-card.component.ts

same three fixes as last time: barrel, import, empty folder

It takes inputs and emits outputs. That makes it type:ui, and it goes in scope:shared — not in the domain that happened to need it first.

Hands-on

STEPS 3–4 / 6 · CHECK

The feature got smaller

CHECKPOINT

- ✓ The app still serves and the catalogue still renders cards
- ✓ feature-catalogue-list has no services/ and no components/ folder left
- ✓ Every import it makes now starts with @workshop/

Relative imports hide connections. Package boundaries make them visible to the graph and to lint.

Which is exactly what happens next.

Hands-on

Ask the tooling what it thinks

TASK

```
1 npx nx run-many -t lint
```

it fails — linting home-feature-home

A project tagged type:feature can only depend on libs tagged

type:ui, type:util, type:model, type:feature, type:data-access

@nx/enforce-module-boundaries

Read it carefully. The library it is complaining about is not tagged at all.

Hands-on

Tell Nx what these libraries are

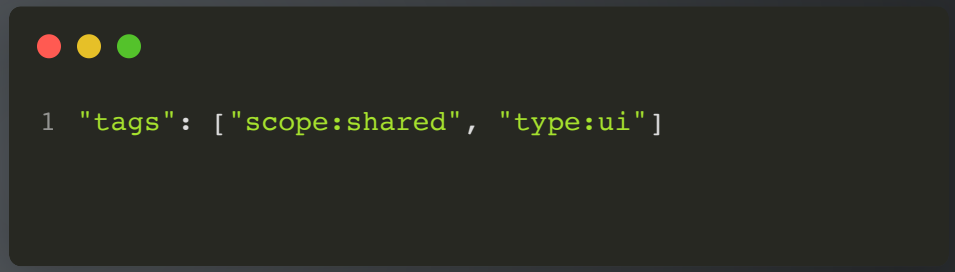
HINT

open the project.json of

shared-ui-header

shared-ui-hero

shared-ui-product-card



Run npx nx run-many -t lint again — it should pass.

Then open eslint.config.mjs and read depConstraints. That is the whole architecture, written down as rules.

type: is what a library is. scope: is who it belongs to. You need both.

Hands-on

Every library declares itself

CHECKPOINT

- ✓ `npx nx run-many -t lint` passes on every project
- ✓ The libraries you created in steps 1–4 all carry a `scope:` and a `type:`
- ✓ You can point at the rule in `eslint.config.mjs` that would have caught it

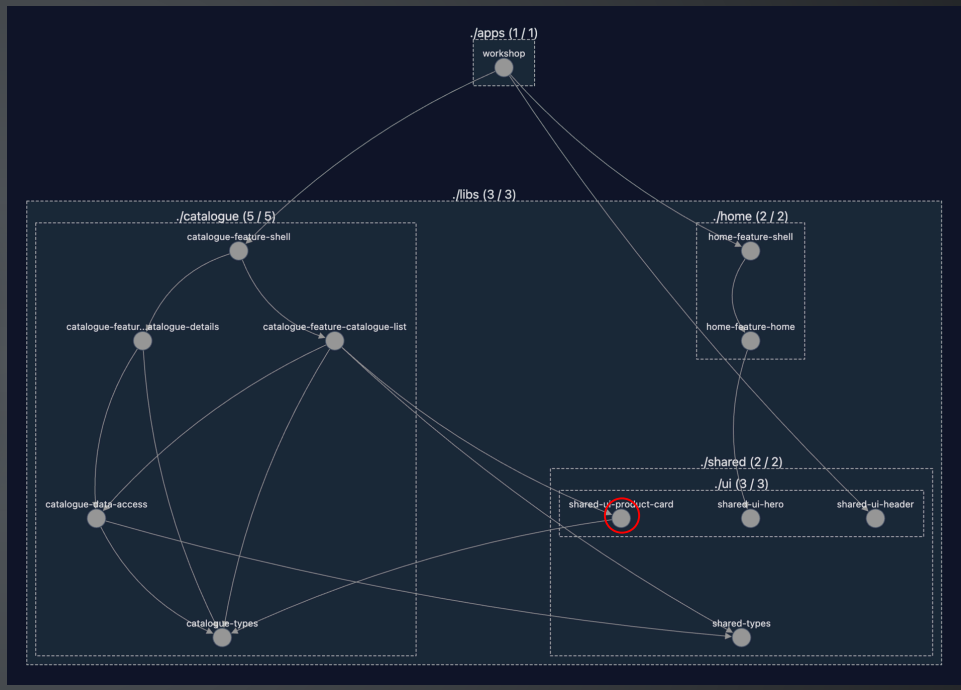
Hands-on

STEP 6 / 6

Lint is happy. The graph is not.

TASK

```
1  npx nx graph
```



Find the edge that leaves scope:shared and lands inside scope:catalogue.

One arrow. Everything else is fine.

Hands-on

STEP 6 / 6

Shared code cannot know about domains

HINT

A scope:shared library may depend on other shared libraries only. The moment it reaches into a domain, it stops being shared — it is a catalogue library sitting in the wrong folder.

The fix

Move the type the card needs into shared-types, and re-point both importers at it.

Ask first whether the type is really catalogue-specific. If two scopes need it, it was never domain knowledge.

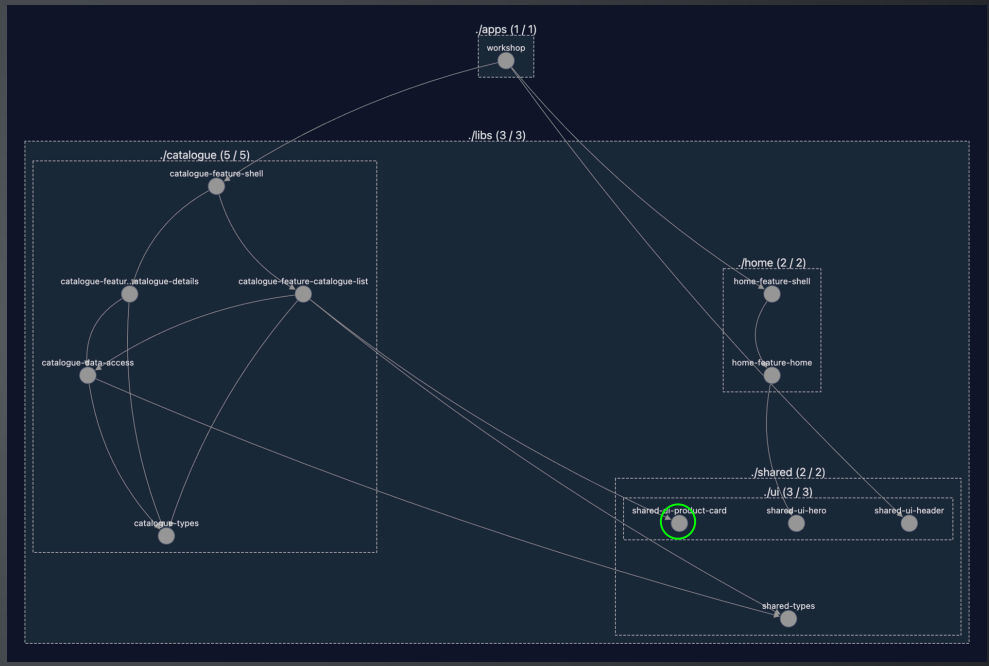
Hands-on

STEP 6 / 6 · CHECK

No arrow leaves shared

CHECKPOINT

- ✓ The card now imports from shared-types
- ✓ Lint still passes
- ✓ The app still serves



Lint never complained about this one. The graph found it.

Worth adding a scope: constraint so it can never come back.

What you just built

- 4 new libraries, each one a deliberate type
- 2 shells — the app knows domains, not pages
- 1 lint rule that now fails the build on a bad import
- 1 violation you found by looking at a picture

The structure is settled. Nothing you add from here can quietly break it.

Next: what actually goes inside these libraries — and why the way we manage state has to change too.

Context is King: Supercharging AI with Nx

Skills & MCPs

The "Context Gap"

The problem with Agents

- Linear View vs. Graph Map

  - AI sees a flat list of files.

- Logic vs. Boundaries

  - AI understands the code, but ignores the library limits.

- The "Spaghetti" Risk

  - Illegal imports and circular dependencies

The Cost of Standard AI

The problem with Agents

- Inconsistent Code

  - Breaks your workspace global standards

- The "Token Trap"

  Massive waste feeding 150+ libs

- Guesswork vs. Knowledge

  Broken refactors from "guessing" paths

The Nx Solution

Instant Architectural Understanding

- The Bridge
  - Nx Graph acts as an instant workspace "map"
- Deep Insights
  - AI moves from Autocomplete to Architect
- Efficient Discovery
  - Faster results using significantly fewer tokens

The Nx Solution

Instant Architectural Understanding

- Deep workspace architecture

  - What projects exist
  - How they relate
  - What tasks can be run



  - Faster discovery
  - Better understanding
  - Fewer tokens

The Nx Solution

Instant Architectural Understanding

nx graph  **instant** architectural understanding

```
nx graph --print
```


```
{
  "graph": {
    "nodes": {
      "catalogue-feature-catalogue-details": {
        "name": "catalogue-feature-catalogue-details",
        "type": "lib",
        "data": {
          "root": "libs/catalogue/feature-catalogue-details",
          "targets": {
            "eslint:lint": {
              "cache": true,
              "options": {
                "cwd": "libs/catalogue/feature-catalogue-details",
                "command": "eslint ."
              },
              "inputs": [
                "default",
                "^default",
                "{workspaceRoot}/eslint.config.mjs",
                "{workspaceRoot}/libs/catalogue/feature-catalogue-details/eslin
                "{workspaceRoot}/tools/eslint-rules/**/*",
                {
                  "externalDependencies": [
                    "eslint"
                  ]
                }
              ],
              "outputs": [
                "{options.outputFile}"
              ],
              "metadata": {
                "technologies": [
:
```

Agent Skills

Evolution of Nx AI

From Infrastructure to Intelligence

- The System-Configured Era
  - MCP servers lived in your local machine settings
- The Repo-Integrated Era
  - Both MCP servers AND skills live inside the repo
- The Modern Workflow Split
  - Skills tell the agent what to know. MCP tools tell the agent what to do.
- Versioned Intelligence
  - AI rules evolve alongside your code

Efficiency & Control

Stopping the "Token Tax"

- Contrast with the old way
  - Skills loaded when needed, not stuffed upfront

Your AI Coding Agent

Set it up

```
npx nx configure-ai-agents
```

Your AI Coding Agent

Set it up - claude

```
Fetching nx...
? Which AI agents would you like to configure? (space to select, enter to confirm) …
✔ Claude Code (update available)
✔ OpenAI Codex (update available)
✔ OpenCode (update available)
✔ GitHub Copilot for VSCode (update available)
✔ Cursor (update available)
✔ Gemini
  Installs Nx plugin (MCP + skills + agents). Updates CLAUDE.md.
```

Your AI Coding Agent

Set it up - claude

- Generates

```
// .claude/settings.json

{
  "extraKnownMarketplaces": {
    "nx-claude-plugins": {
      "source": {
        "source": "github",
        "repo": "nrwl/nx-ai-agents-config"
      }
    }
  },
  "enabledPlugins": {
    "nx@nx-claude-plugins": true
  }
}
```

...and CLAUDE.md file

Your AI Coding Agent

Set it up - claude



prompt ⟶ "list the Nx related skills you see"



There are **no** Nx-specific skills in the currently available skill list.

Your AI Coding Agent

Set it up

```
claude plugin install nx@nx-claude-plugins
```

Install Claude Code: https://code.claude.com/docs/en/quickstart

Your AI Coding Agent

Set it up - claude

prompt ⟶ "list the Nx related skills you see"

- Here are the Nx-related skills available:
  - **nx:nx-workspace** — Explore and understand Nx workspaces (projects, tasks, configuration)
  - **nx:link-workspace-packages** — Link workspace packages in monorepos to resolve import/dependency issues
  - **nx:nx-run-tasks** — Run tasks in an Nx workspace (build, test, lint, serve, etc.)
  - **nx:monitor-ci** — Monitor Nx Cloud CI pipeline and handle self-healing fixes
  - **nx:nx-plugins** — Find and add Nx plugins to the workspace
  - **nx:nx-import** — Import/merge repositories into an Nx workspace using `nx import`

> ls -la ~/.claude/plugins/marketplaces/nx-claude-plugins/skills/

Your AI Coding Agent

Set it up - claude - skills in repo

```
1 npx skills add nrwl/nx-ai-agents-config
```

Key Concepts

Skills

Reusable AI behaviors invoked via slash commands or automatically

Key Concepts

Skills - how to run them

- You explicitly run /skill-name
  - /monitor-ci or /nx-workspace
- Claude invokes them automatically
  - guided by instructions in CLAUDE.md

Bottom line: You don't need to manually run /skill-name for most things.

Examples

Skills - how to run them

- Using the /nx-workspace skill, analyze the code in @libs/catalogue/types/src/lib/product.ts and the current project graph. Based on our library type definitions (Feature, UI, Data-Access, Util), identify the correct architectural home for this file. Explain why the current location is a violation and suggest the target library.

Examples

Skills - how to run them

- Show me the dependency graph for the shared-ui-product-card lib and list any circular dependencies.

- What are the available targets for the shared-ui-product-card library?

Examples

multi-step prompt

- 1. Use /nx-workspace to find all projects tagged type:ui. For shared-ui-product-card, report its directory, project name, import alias, and tags. Stop and show me before continuing.

  2. Use /nx-generate to create an Angular library for "product-mini-card", following exactly the conventions you reported in step 1 (directory, naming, alias, tags).

  3. Use /angular-developer to build the component, using ProductCardComponent as the template. Make it display-only: remove the favorite button, its output and handler, and any imports or styles that become unused. Keep the card-level routerLink so it still navigates.

  4. Use /nx-run-tasks to run lint and test for the new project.

Examples

Skills - how to run them

1. /nx-workspace: list all libs and check them against:
   - name = <scope>-<type>-<name>
   - one scope:* + one type:* tag (per eslint depConstraints)
   - type:ui libs live in libs/shared/ui/

   Show the violations and stop.

2. Once confirmed, create a branch, then for each lib:
   a. /nx-generate: move/rename it (@nx/workspace:move)
   b. Fix its tags
   c. /nx-run-tasks: lint, test, build (affected)
   d. Green → commit. Red → stop and show me.

3. Summarise what changed.

Introducing Signals

Bridge the Structure (Nx) and State (Signals)

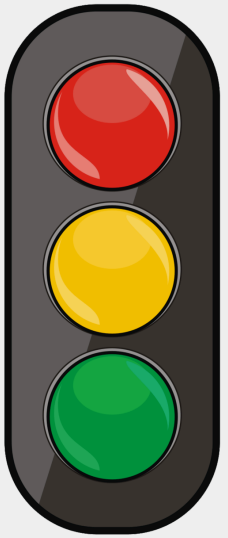
In this lesson

- The Signals: What and Why
- Signals API

What are Signals?

- smart variables that notify anyone who's interested when their value changes.

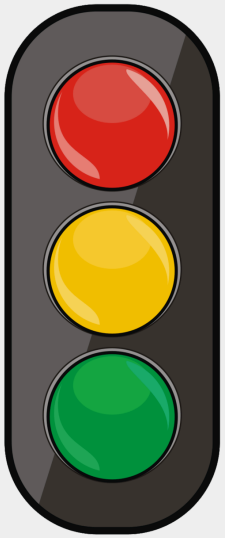
- Traditional change detection checks everything. Signals allow tracks specific values



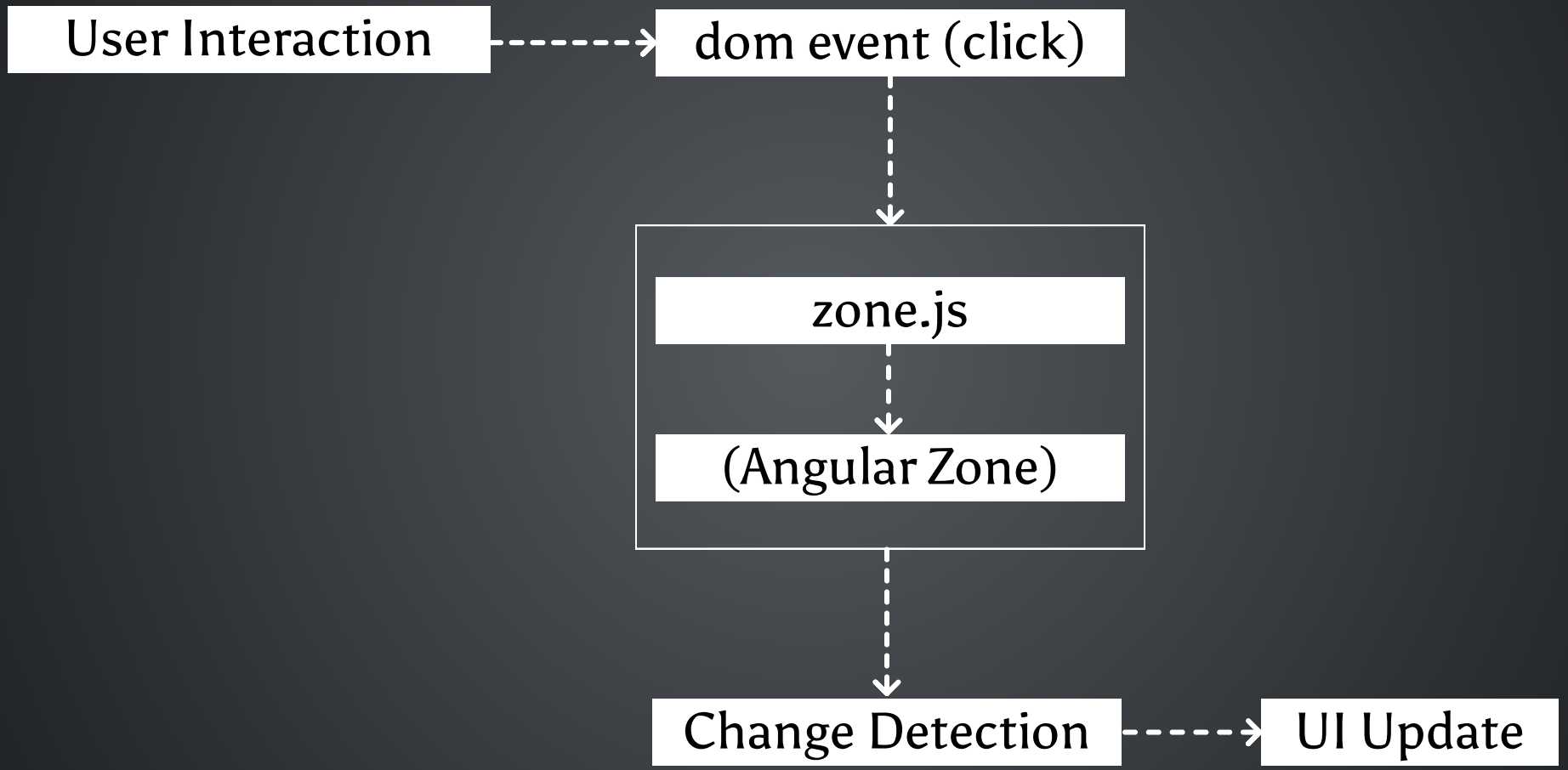
Why Signals?

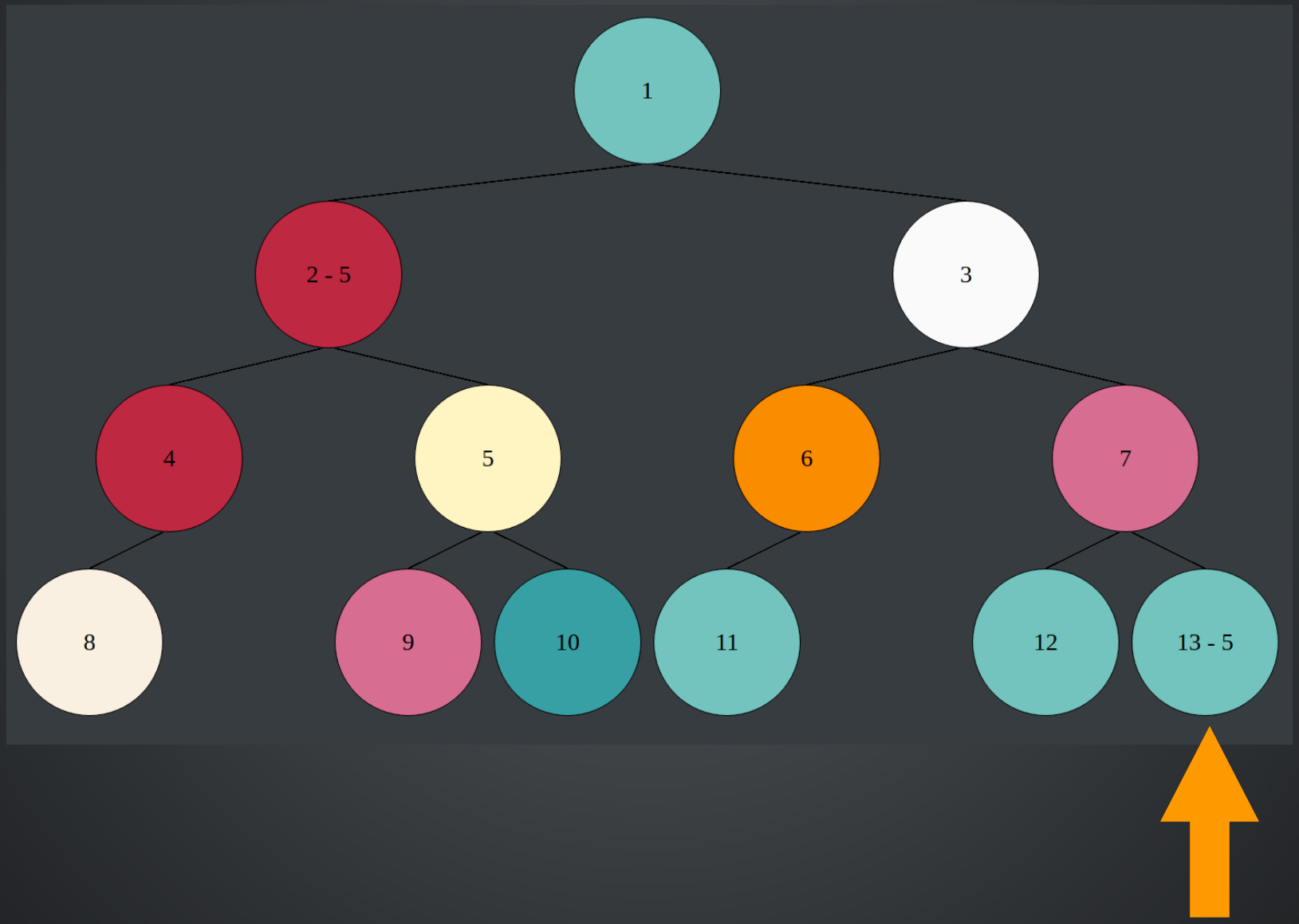
- Traditional Change Detection can be slow on complex applications

- zone.js is great but triggers the CD multiple times



Zone.js


User Interaction
dom event (click)
zone.js
(Angular Zone)
Change Detection
UI Update

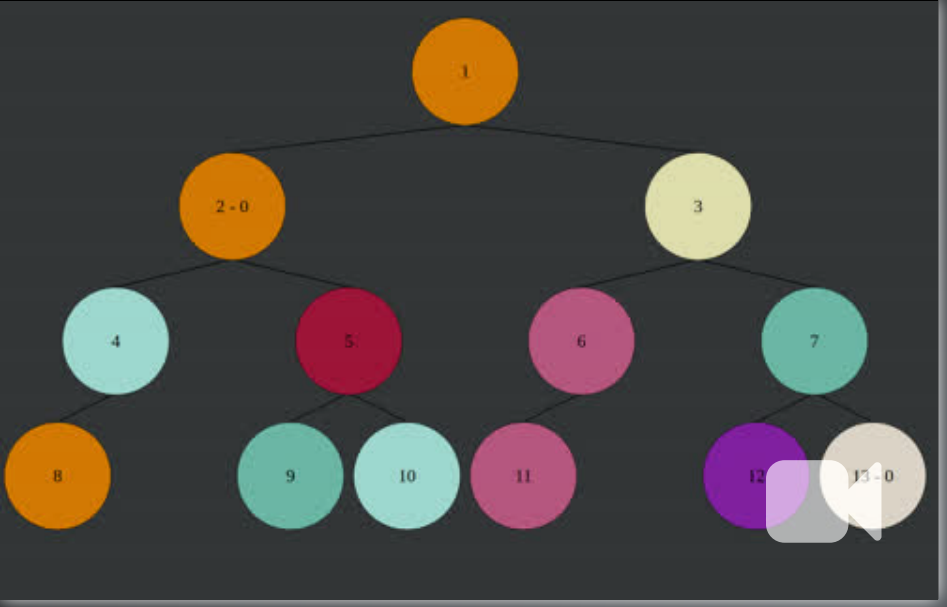

1
2 - 5
3
4
5
6
7
8
9
10
11
12
13 - 5

Angular checks the entire component tree
when the micro-task queue is empty

Default + Observable

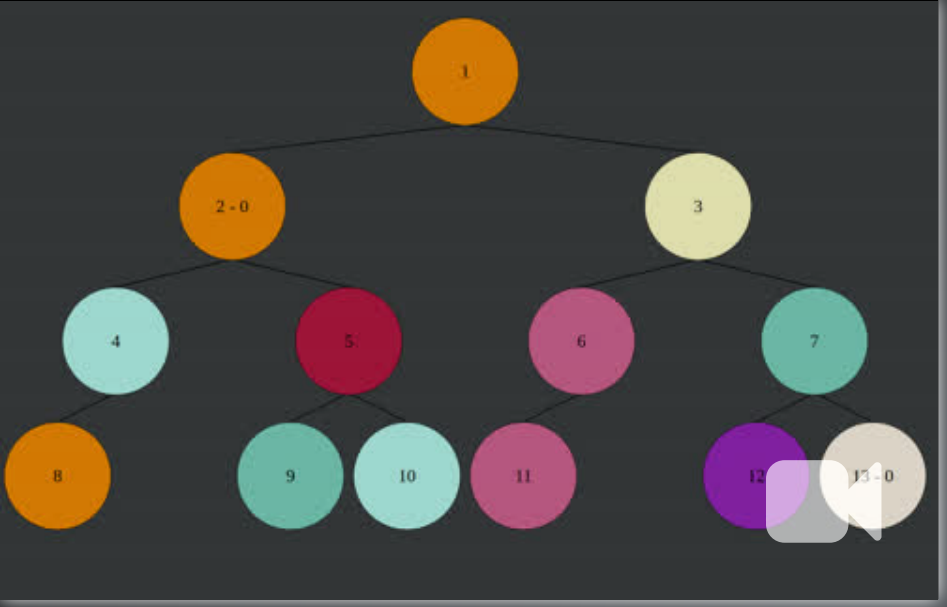
OnPush + Observable

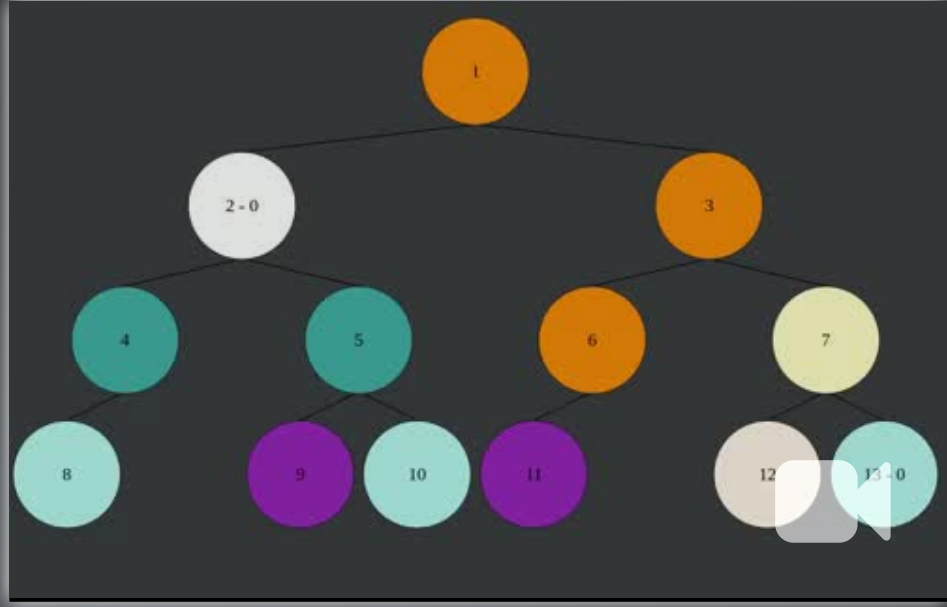
OnPush + Signals



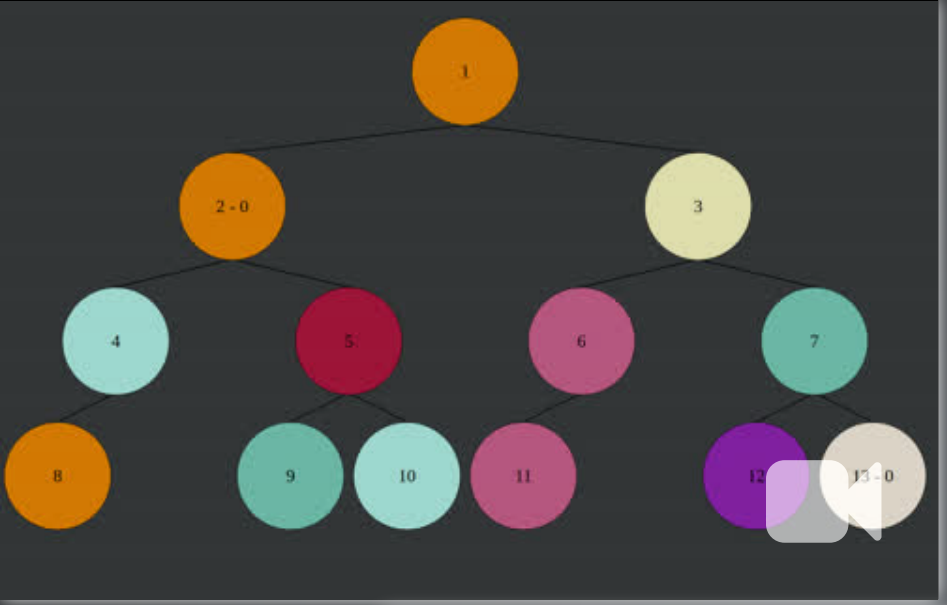
OnPush + Observable

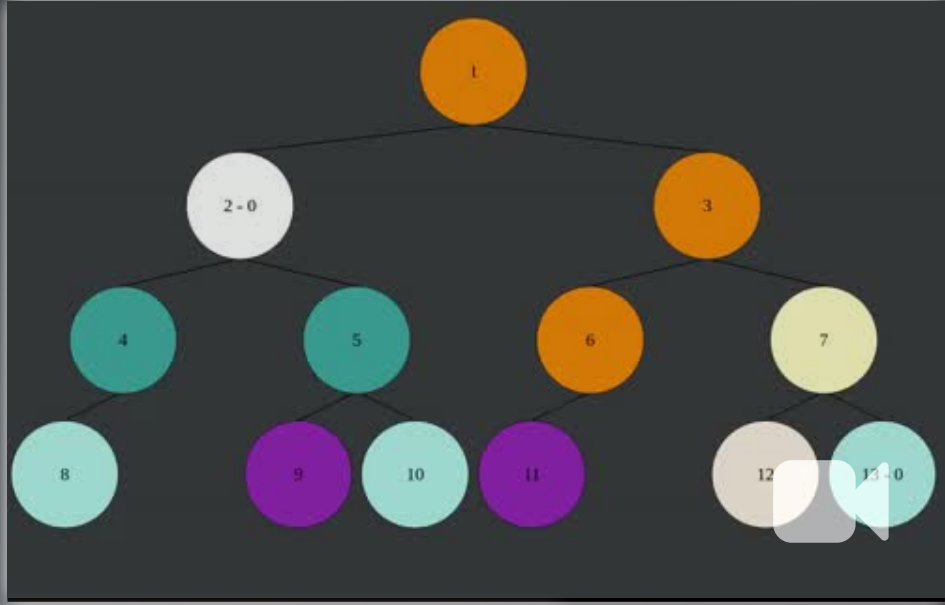
OnPush + Signals

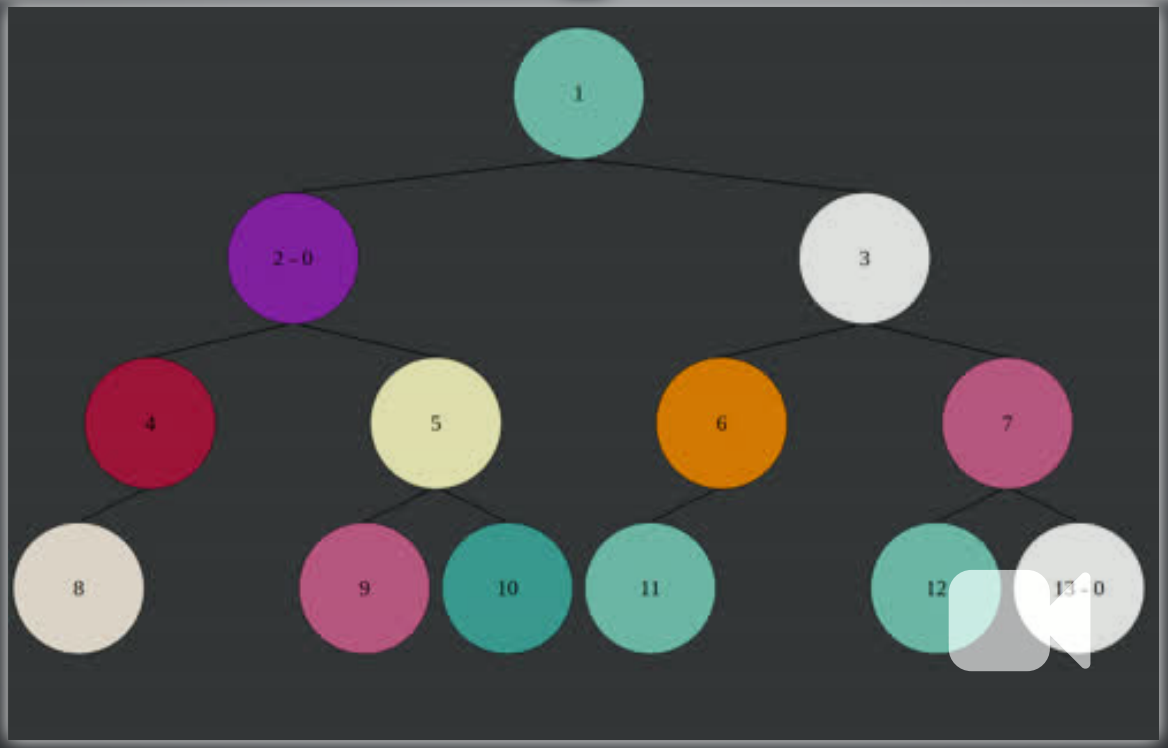




OnPush + Signals


1
2 - 0
3
4
5
6
7
8
9
10
11
12
13 - 0

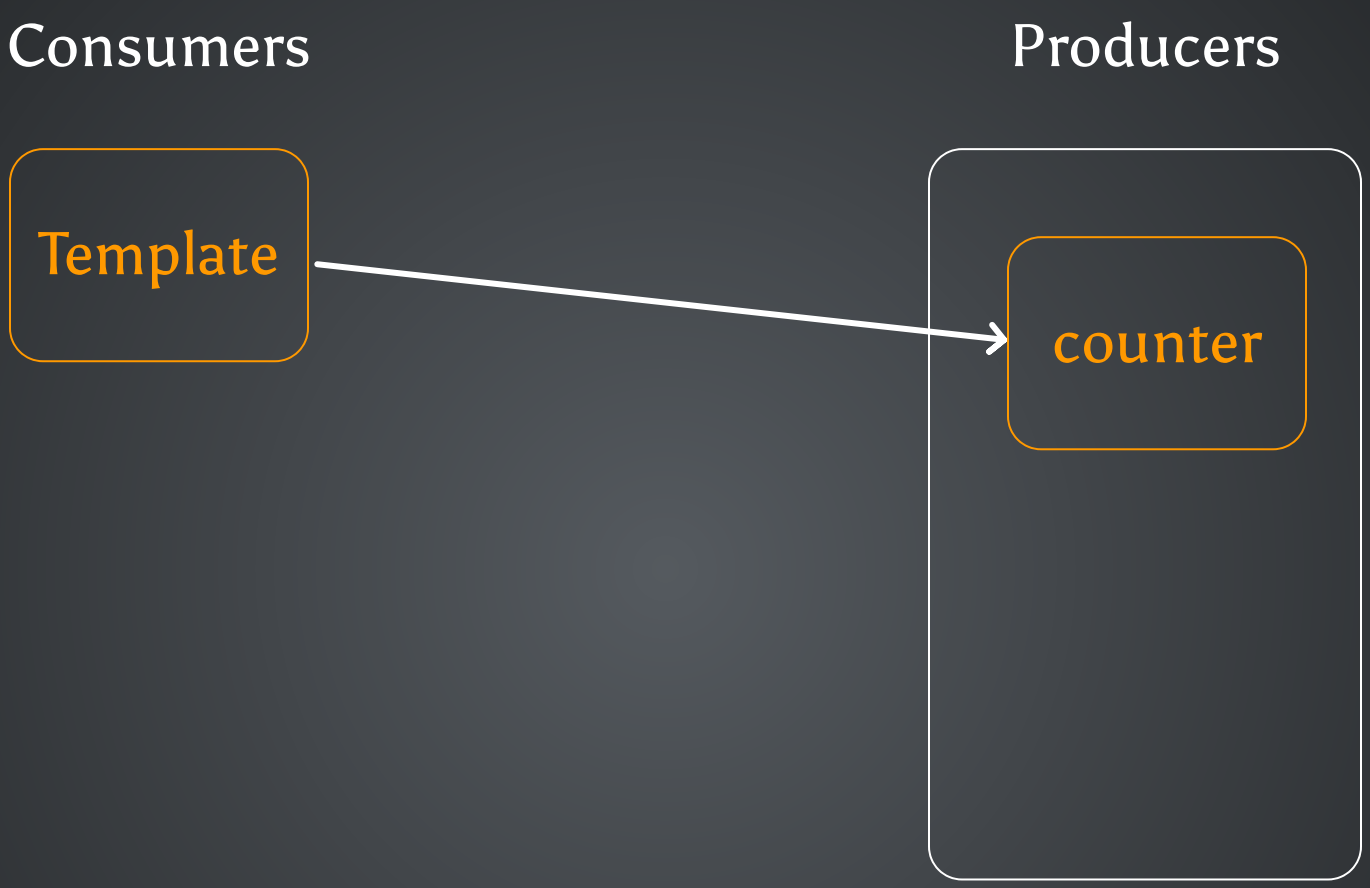

1
2 - 0
3
4
5
6
7
8
9
10
11
12
13 - 0


1
2 - 0
3
4
5
6
7
8
9
10
11
12
13 - 0

profanis

~~Consumers~~
Observers

~~Producers~~
Subject


Consumers
Producers
Template
counter

Producer

```
1 counter = signal<number>(0);
```

Producer

```
1 counter = signal<number>(0);
```



Returns a WritableSignal

Producer

```
1 counter = signal<number>(0);
```



Define the type

Producer

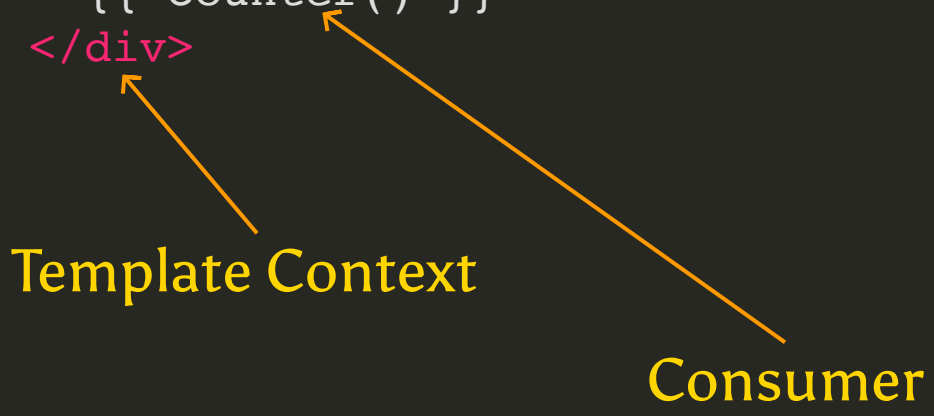
```
1 counter = signal<number>(0);
```



Default value

Consumer

```
1 <div>
2   {{ counter() }}
3 </div>
```



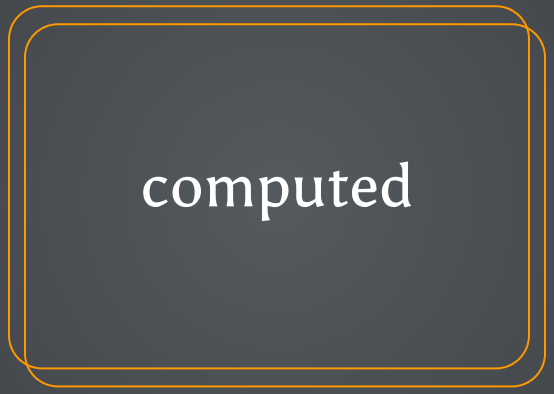
Consumer & Producer



Producer

```
1 const evenOrOdd =
2       computed(() => counter() % 2 === 0 ? 'even' : 'odd');
```

Consumer


computed

Computed

- The derivedState is getting updated when the source signal has a change

** source signal = mySignalArray()*

```
1 const derivedState =
2           computed(() => mySignalArray().length);
```


effect

Effect

- - The effect runs synchronously

  - The effect registers the referenced signals as dependencies.

```
1 constructor() {
2    effect(() => {
3       console.log(mySignalArray().length)
4    })
5 }
```

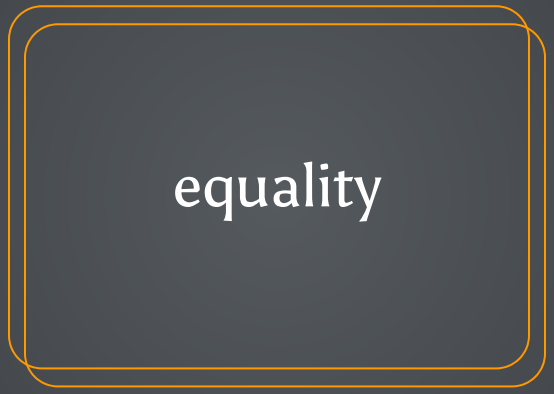
- Use for

  - Logging

  - DOM Manipulation

  - Storage handling (e.g. localStorage)

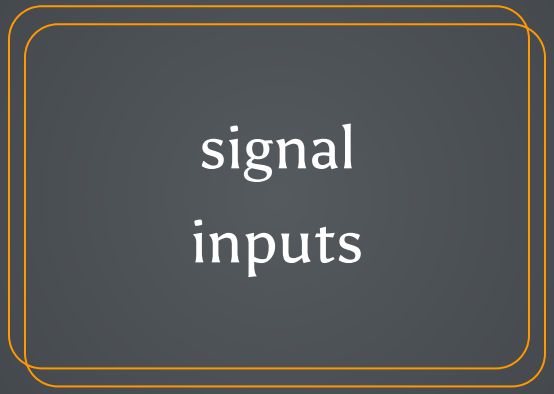
  - Update other signals*


equality

Equality

- by default signals use the `Object.is()` comparison
- optionally provide an equality function

```
1 mySignalArray = signal([], { equal: _.isEqual })
2
3 constructor() {
4    effect(() => {
5       console.log(mySignalArray().length)
6    })
7 }
```


signal
inputs

```
1 @Component({...})
2 export class MyComponent {
3   @Input() isChecked = false;
4 }
```

```
1 @Component({...})
2 export class MyComponent {
3   isChecked = input(false);
4 }
```



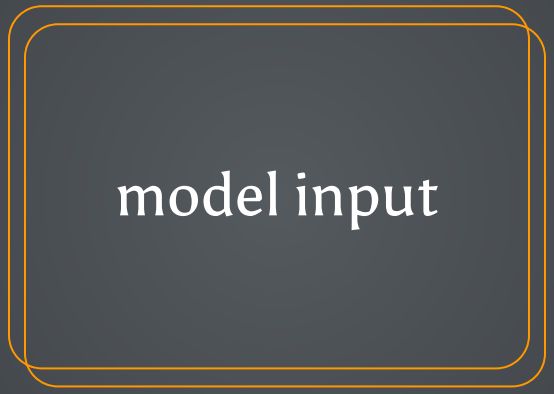
```typescript
export interface UserModel {
  name: string;
  age: number;

  /*Social*/
  address: string;
  twitter: string;
  linkedin: string;
  github: string;
  instagram: string;
  facebook: string;
  website: string;
  email: string;
}
```

```typescript
@Component({...})
export class MyComponent implements OnChanges {
  @Input({ required: true }) user!: UserModel;
  userSocials: string[] = [];

  ngOnChanges(changes: SimpleChanges): void {
    if (changes.user) {
      const { name, age, ...userSocials } = this.user;
      this.userSocials = Object.values(userSocials);
    }
  }
}
```

```ts
@Component({...})
export class MyComponent {
  user = input.required<UserModel>();
  userSocials = computed(() => {
    const { name, age, ...userSocials } = this.user();
    return Object.values(userSocials);
  });
}
```


model input

```
1 @Component({...})
2 export class ChildComponent {
3   @Input({ required: true }) name!: string;
4   @Output() nameChange = new EventEmitter<string>();
5 }
```

```
1  @Component({
2    selector: 'app-parent',
3    standalone: true,
4    imports: [ChildComponent],
5    template: `
6      <app-child  [name]="username"
7                  (nameChange)="changeHandler($event)" />
8    `,
9  })
10 export class ParentComponent {
11    username = 'profanis';
12 }
```

```
1  @Component({
2    selector: 'app-parent',
3    standalone: true,
4    imports: [ChildComponent],
5    template: `
6      <app-child  [name]="username"
7                  (nameChange)="changeHandler($event)" />
8    `,
9  })
10 export class ParentComponent {
11    username = 'profanis';
12 }
```

```ts
@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [ChildComponent],
  template: ` <app-child [(name)]="username" /> `,
})
export class ParentComponent {
  username = 'profanis';
}
```

```ts
@Component({...})
export class ChildComponent {
  // @Input({ required: true }) name!: string;
  // @Output() nameChange = new EventEmitter<string>();
  name = model<string>();
}
```

```ts
@Component({...})
export class ChildComponent {
  // @Input({ required: true }) name!: string;
  // @Output() nameChange = new EventEmitter<string>();
  name = model.required<string>();
}
```

```ts
@Component({...})
export class ChildComponent {
  name = model<string>(); // writable signal

  addTitle() {
    this.name.update((name) => `Mr. ${name}`);
  }
}
```

```ts
@Component({...})
export class ChildComponent {
  name = model<string>(); // writable signal

  titleExists = computed(() => this.name().startsWith('Mr.'));
}
```

```
1 @Component({
2   selector: 'app-parent',
3   standalone: true,
4   imports: [ChildComponent],
5   template: ` <app-child [(name)]="username" /> `,
6 })
7 export class ParentComponent {
8   username = 'profanis';
9 }
```


new output

```
1 @Component({...})
2 export class ChildComponent {
3   name = input.required<string>();
4   @Output() nameChange = new EventEmitter<string>();
5 }
```

```
@Component({...})
export class ChildComponent {
  name = input.required<string>();
  nameChange = output<string>()
}
```

```
1 @Component({...})
2 export class ChildComponent {
3   @Output() formIsValid = this.form.statusChanges.pipe(
4      map((status) => status === 'VALID'),
5   );
6 }
```

```ts
import { outputFromObservable } from '@angular/core/rxjs-interop';

@Component({...})
export class ChildComponent {
  formIsValid = outputFromObservable(
    this.form.statusChanges.pipe(
            map((status) => status === 'VALID'))
  );
}
```


view queries

```
@Component({
  selector: 'app-parent',
  template: ` <div #child>Child wrapper</div> `,
})
export class ParentComponent {
  child = viewChild('child', { read: ElementRef<HTMLElement> });

  constructor() {
    effect(() => {
      const child = this.child();
      if (child) {
        console.log('Child element:', child.nativeElement);
      }
    });
  }
}
```

```
@Component({
  selector: 'app-parent',
  template: ` <div #child>Child wrapper</div> `,
})
export class ParentComponent {
  child = viewChild.required('child', { read:
                                      ElementRef<HTMLElement> }

  constructor() {
    effect(() => {
      const child = this.child();
      console.log('Child element:', child.nativeElement);
    });
  }
}
```

```html
<app-parent>
    <app-child></app-child>
</app-parent>
```

```ts
@Component({
  selector: 'app-parent',
  imports: [ChildComponent],
})
export class ParentComponent {
  child = contentChild(ChildComponent);

  constructor() {
    effect(() => {
      const childComponent = this.child();
    });
  }
}
```

profanis

```html
<app-parent>
    <app-child></app-child>
</app-parent>
```

```ts
@Component({
  selector: 'app-parent',
  imports: [ChildComponent],
})
export class ParentComponent {
  child = contentChild.required(ChildComponent);

  constructor() {
    effect(() => {
      const childComponent = this.child();
    });
  }
}
```

profanis

```typescript
@Component({ })
export class ChildComponent {
  buttonClick = output<void>();

  onButtonClick() {
    this.buttonClick.emit();
  }
}
```

```typescript
export class ParentComponent {
  child = contentChild(ChildComponent);

  constructor() {
    effect(() => {
      const childComponent = this.child();

      if (childComponent) {
        const buttonClick$ = outputToObservable(childComponent.buttonClick);

        buttonClick$.subscribe(() => {
          console.log('Child button was clicked!');
        });
      }
    });
  }
}
```

profanis

Hands-on



From structure to behaviour

The libraries are in place. Now what goes inside them.

Three exercises, one idea:

stop recomputing, start deriving

1. A value the template keeps asking for → derive it once
2. Two components that need to talk → signal in, signal out
3. Shared state → one signal, many readers

Hands-on

Three exercises

1. Get the function out of the template — use computed
2. A filter component with signal input and output
3. Favourite products — shared state in a service

Branch: lab/two — it already contains every answer from the last section, so nobody starts behind.

Hands-on

SETUP

Move to the next branch

TASK

```
git stash push -u -m "my lab one work"
git checkout lab/two
npm i
```

lab/two already has the two shells, the data-access library, the ui library and every tag you added. Your own version is safe in the stash if you want it later.

The exercises below are marked TODO in the code. Search the workspace for it if you lose your place.

Hands-on

EXERCISE 1 / 3

Get the function out of the template

TASK

libs/catalogue/feature-catalogue-details/…/product-details.component.ts

1. Replace getStarArray() with a computed called commentsView
2. Each comment carries its own stars array
3. Update the template, then delete the function

Why it matters: a function in a template runs on every change detection pass.

A computed runs once and then only when something it reads actually changes.

Hands-on

EXERCISE 1 / 3

Shape of the answer

HINT

```
commentsView = computed(() =>
  this.comments().map((comment) => ({
    ...comment,
    stars: Array.from({ length: 5 }, (_, i) => i < comment.rating),
  }))
);
```

```
<!-- template: iterate commentsView(), read comment.stars -->
@for (comment of commentsView(); track comment.user) { ... }
```

Hands-on

Derived, not recomputed

CHECKPOINT

- ✓ The stars still render on the product details page
- ✓ `getStarArray` is gone — not unused, gone
- ✓ There is no `()` call in the template except reading a signal

Worth noticing: the shape of the data changed to fit the view. That is what a view model is, and it belongs next to the component — not in the type library.

Hands-on

EXERCISE 2 / 3

Make the filter a real component

TASK

libs/catalogue/feature-catalogue-list/…/plant-filter.component.ts

1. Turn searchTerm from a local signal into an input
2. Add a search output and emit from the template
3. Delete emitAnEvent — it was scaffolding
4. Use the component in catalogue.component.html#25

The component owns no state. It is handed a value and reports an intent. Everything else stays where it was.

Hands-on

EXERCISE 2 / 3

Inside the filter

HINT

```ts
export class PlantFilterComponent {
  searchTerm = input<string>();
  search = output<string>();
}
```

```html
<!-- in the component's own template -->
<input matInput [value]="searchTerm()"
       (input)="search.emit($event.target.value)" />
```

Both are functions you call. input() gives you a read-only signal; output() gives you something with .emit() — and no RxJS in sight.

Hands-on

EXERCISE 2 / 3

Use it in the catalogue

HINT

```
<!-- catalogue.component.html — replaces the whole mat-form-field -->
<lib-plant-filter
  [searchTerm]="state.searchTerm()"
  (search)="onSearchChange($event)"
/>
```

Easy to forget: add PlantFilterComponent to the imports array of CatalogueComponent.

onSearchChange already exists and already does the right thing. You are only changing who calls it.

Hands-on

Same behaviour, one less responsibility

CHECKPOINT

- ✓ Typing in the box still filters the list
- ✓ Clearing the search from the empty state still works
- ✓ PlantFilterComponent holds no state of its own

The second check is the interesting one. The clear button calls onSearchChange(''), the state updates, and the input follows — because the value flows down as a signal rather than being held in two places.

Hands-on

EXERCISE 3 / 3

Favourite products — the state

TASK

libs/catalogue/data-access/src/lib/favorites.state.ts

1. A private writable signal holding the favourite IDs
2. count — a computed, not a signal you maintain by hand
3. isFavorite(id) returning a boolean
4. toggleFavorite(id) adding or removing

The signal is private on purpose. Nobody outside this service is allowed to write to it — they go through the two methods. That is the whole discipline.

Hands-on

EXERCISE 3 / 3

Shape of the answer

HINT

```ts
private favorites = signal<Set<string>>(new Set());

count = computed(() => this.favorites().size);

isFavorite(productId: string): boolean {
  return this.favorites().has(productId);
}

toggleFavorite(productId: string): void {
  this.favorites.update((current) => {
    const next = new Set(current);          // a NEW Set — this matters
    if (next.has(productId)) next.delete(productId);
    else next.add(productId);
    return next;
  });
}
```

Mutate the existing Set and nothing will re-render. Signals compare by reference — same object, no change, no notification.

Hands-on

EXERCISE 3 / 3

Two readers, one source

TASK

5 catalogue.component.ts — a computed that pairs each product with its favourite flag, and onToggleFavorite calling the service

6 app.ts — read count and feed the header badge

Two parts of the app, in different libraries, reading the same state. Neither one knows the other exists.

Nothing subscribes. Nothing unsubscribes. Nothing needs to be told to refresh.

Hands-on

EXERCISE 3 / 3

Shape of the answer

HINT

```
// catalogue.component.ts
productsWithFavorites = computed(() =>
  this.state.products().map((product) => ({
    ...product,
    isFavorite: this.favoritesState.isFavorite(product.id),
  }))
);
```

```
// app.ts
private readonly favoritesState = inject(FavoritesState);
favoriteCount = this.favoritesState.count;

// app.html
<lib-ui-header [favoriteCount]="favoriteCount()" />
```

profanis

Hands-on

One click, two places update

CHECKPOINT

- ✓ Clicking a heart fills it in and the header count goes up
- ✓ Clicking it again reverses both
- ✓ Search for something, clear it — your favourites survive
- ✓ Nothing in the codebase says subscribe

Ask yourself

The app now imports from catalogue-data-access. Does that break the rules you set up in the last section?

What you just built

A value the template stopped asking for

A component that owns nothing and reports everything

Shared state in data access

Next: what happens when the value has to come from a server — and has not arrived yet.

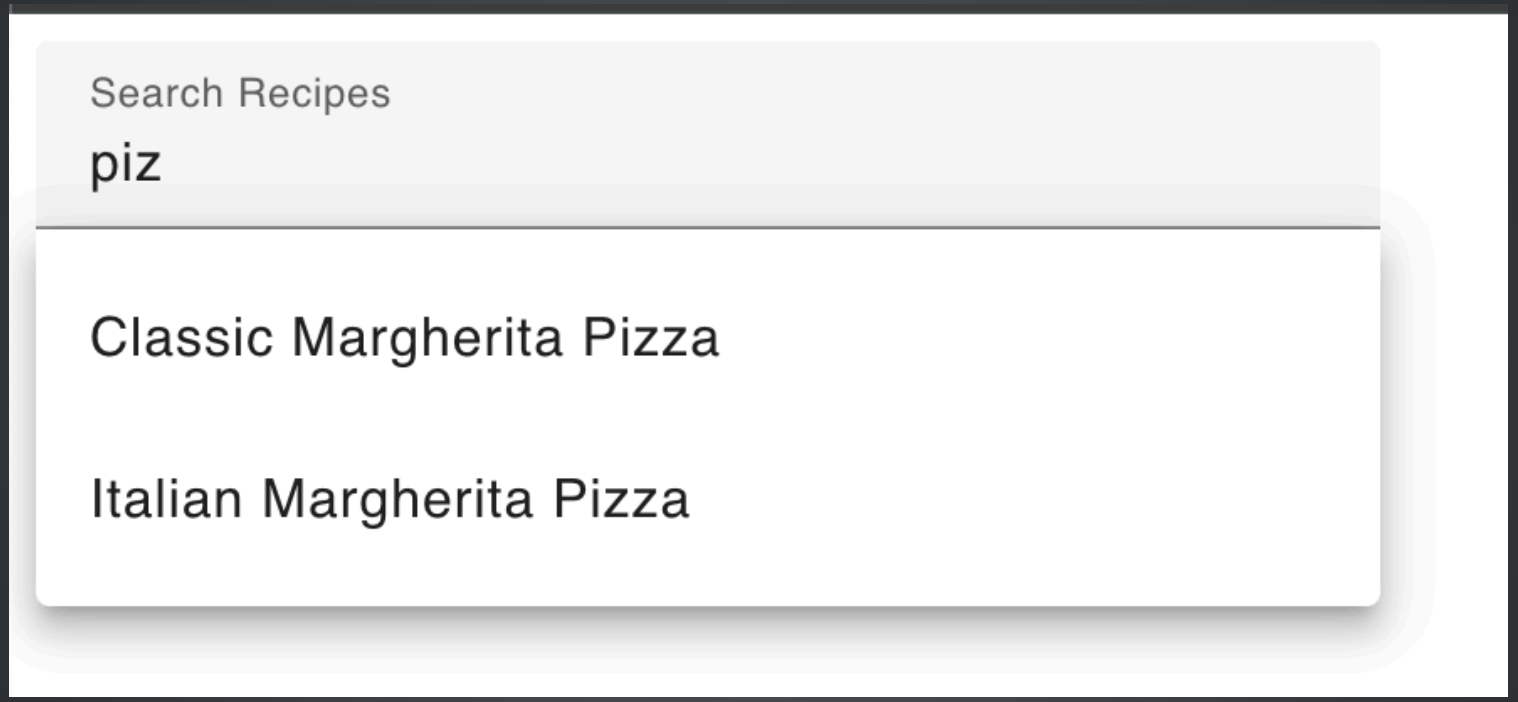
Advanced Reactivity

Resources, I/O

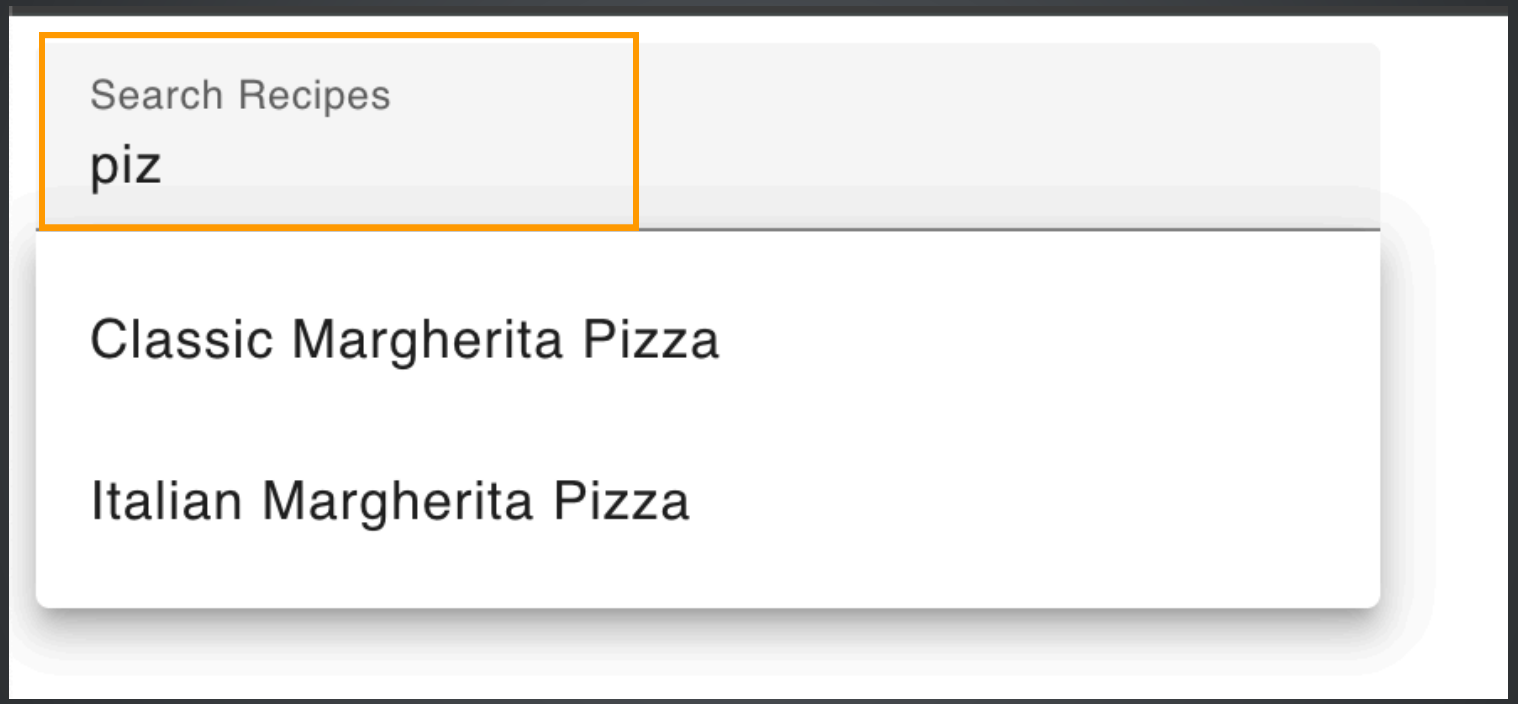
In this lesson

- rxResource and httpResource
- LinkedSignal
- Signals Graph
- Signals Pull Push algorithm
- Agentic Coding with AI

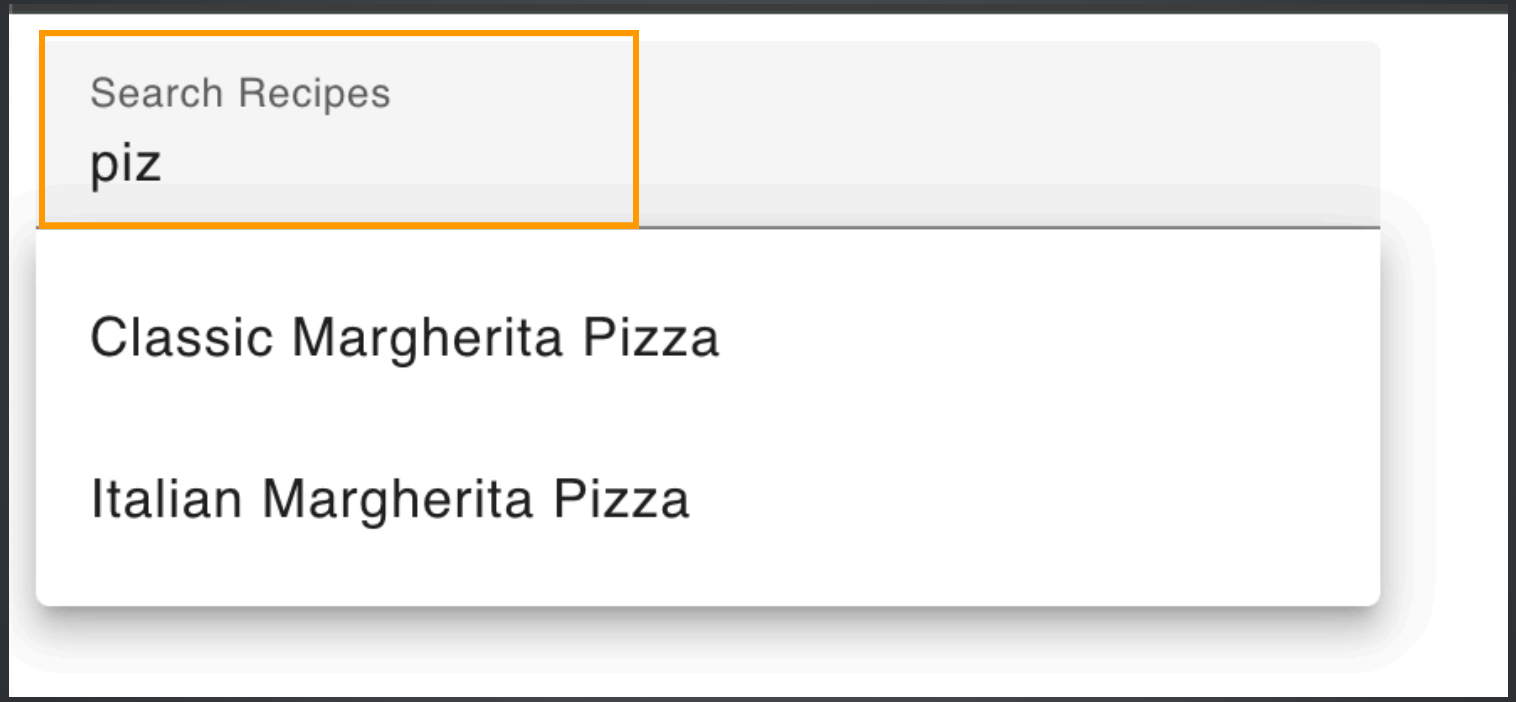

data fetching


Search Recipes
piz
Classic Margherita Pizza
Italian Margherita Pizza

signal<string>



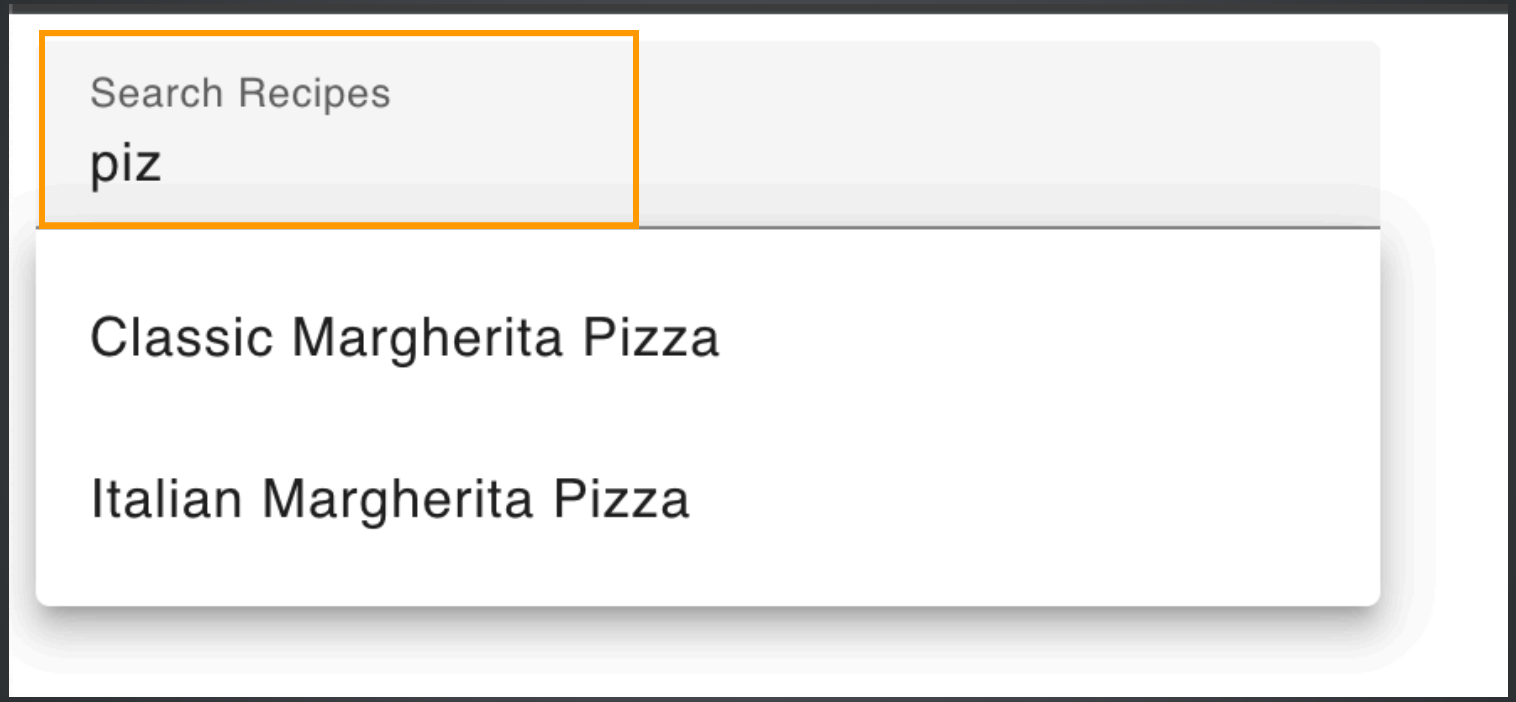
signal<string>



Name	Path
{:} search?q=p	/recipes/search
{:} search?q=pi	/recipes/search
{:} search?q=piz	/recipes/search

profanis

signal<string>



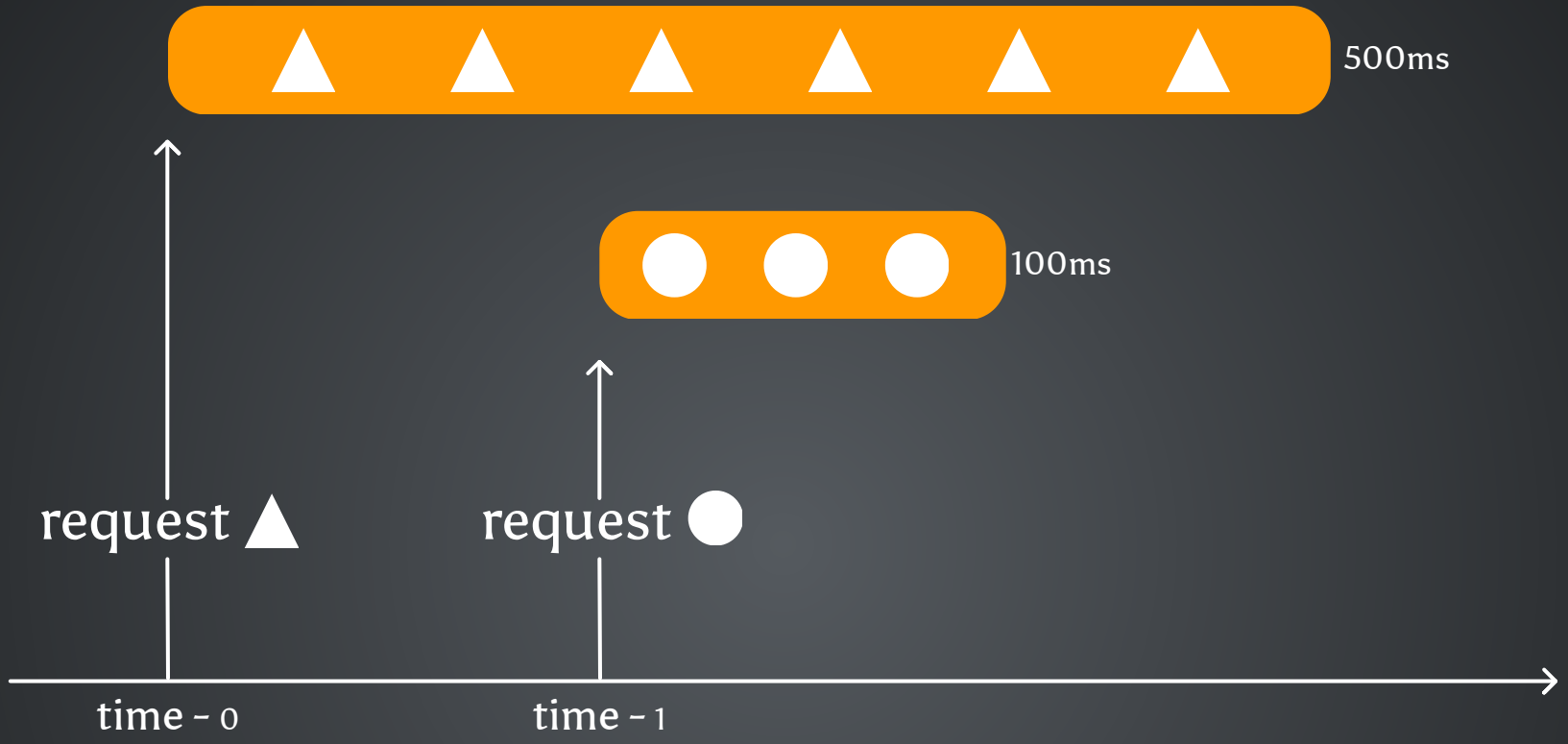
Name	Path
{:} search?q=p	/recipes/search
{:} search?q=pi	/recipes/search
{:} search?q=piz	/recipes/search

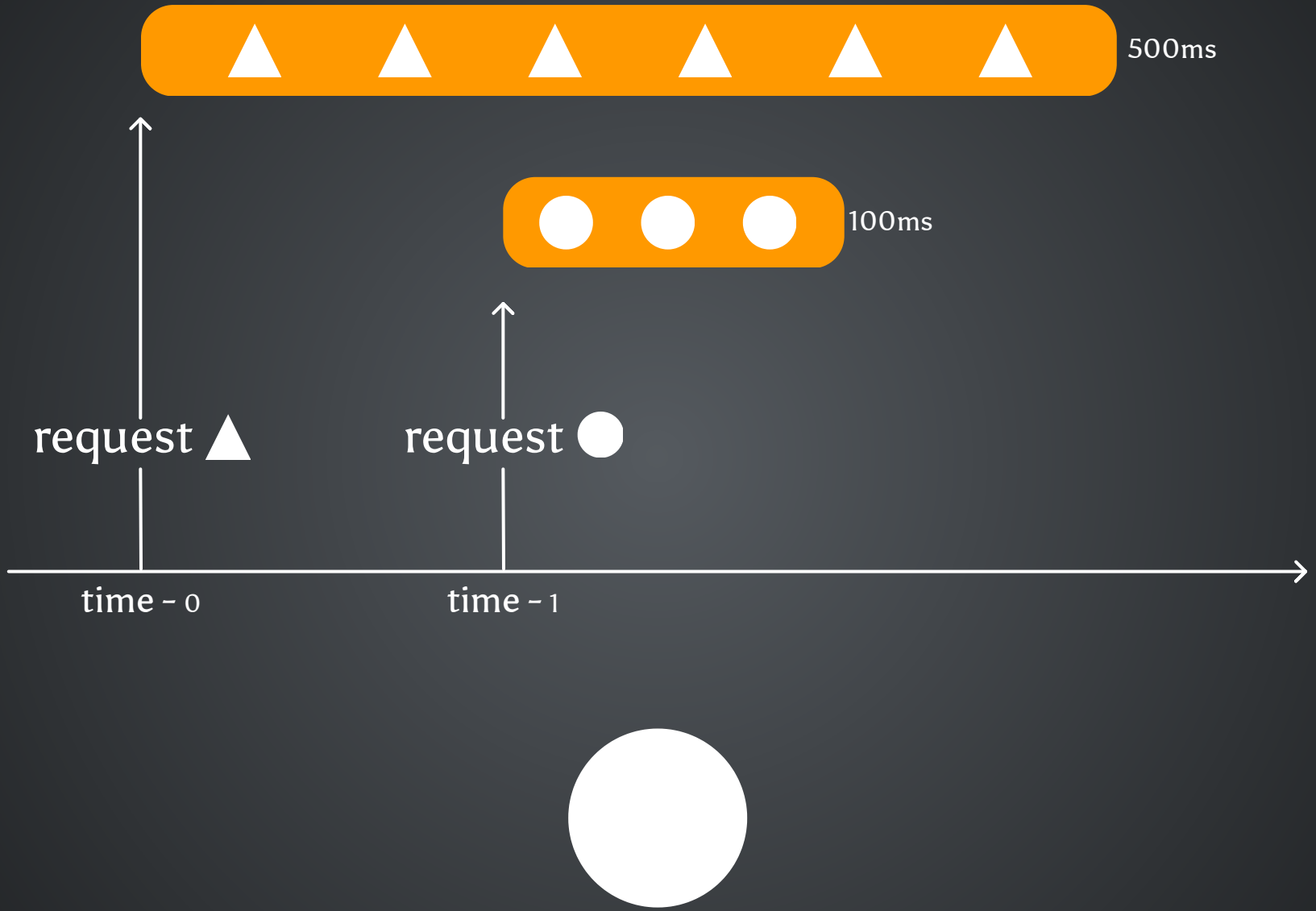
cancel

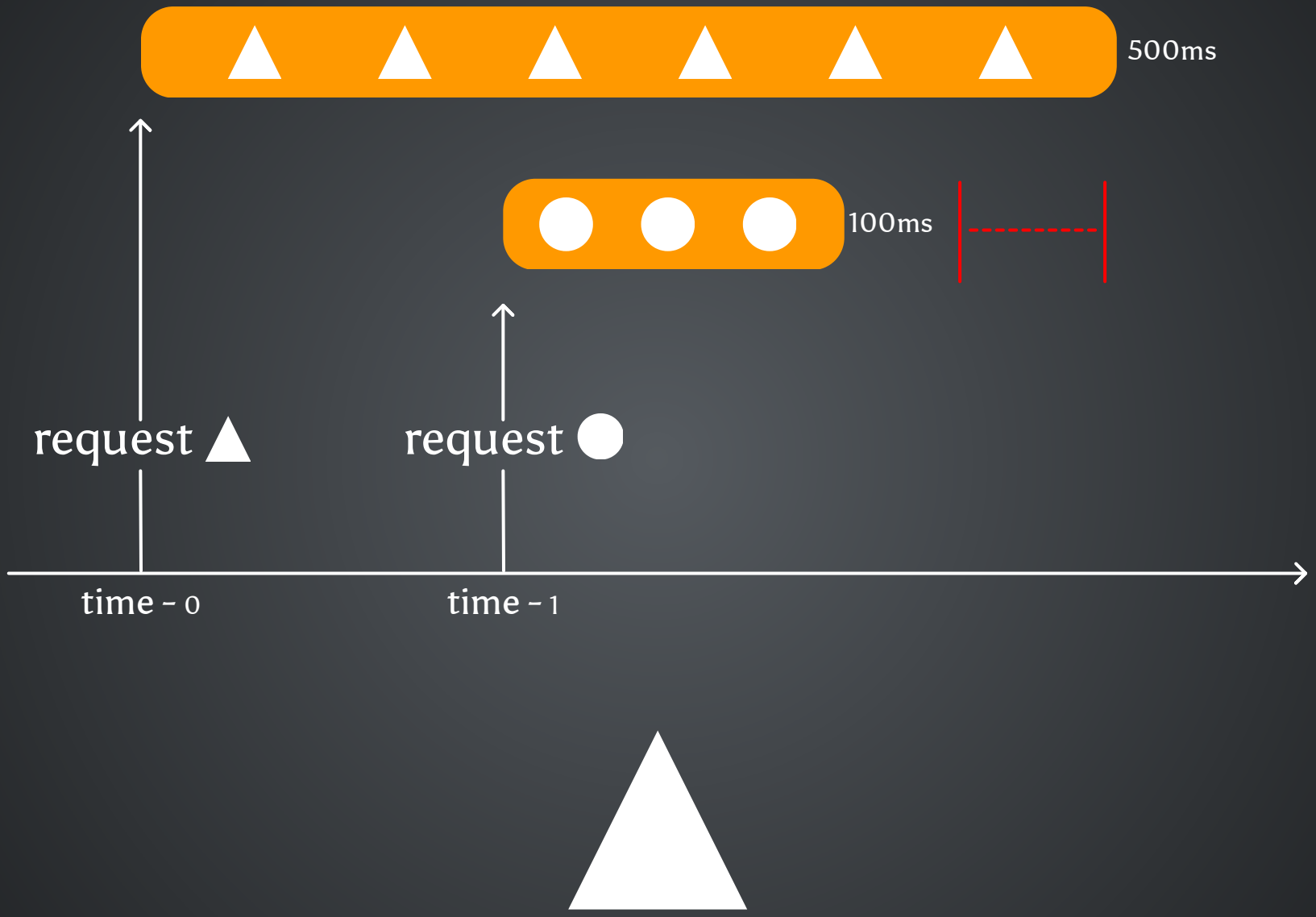
cancel

profanis

“ How about effect?


500ms
100ms
request
request
time - 0
time - 1


500ms
100ms
request
request
time - 0
time - 1


500ms
100ms
request
request
time - 0
time - 1


WRONG
WRONG
WRONG

Sync



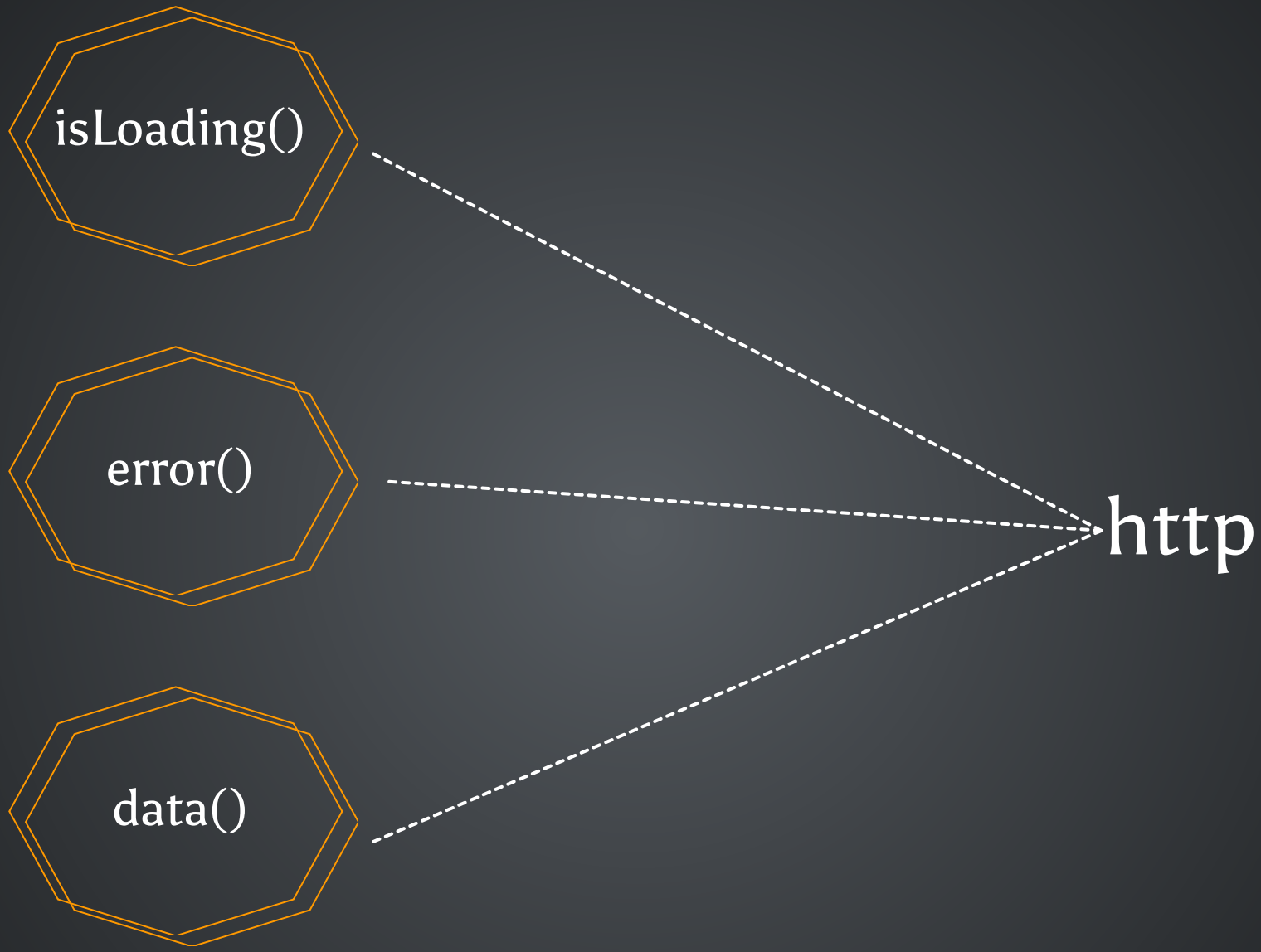
Async

“ At some point we will have the data

“ ~~*At some point we will have the data*~~

“ *We will always have data*


isLoading
error
data
http


isLoading()
error()
data()
http


httpResource

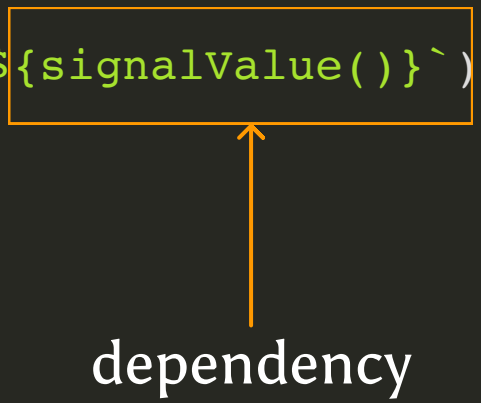
httpResource makes a reactive HTTP request and exposes the **request status** and **response value**

```
1 httpResource(
2     ?,
3     ?
4 )
```

```
1 httpResource(
2     string | object | function ,
3     ?
4 )
```

```
1 httpResource(
2     string | object | function ,
3     options
4 )
```

```
1 // String
2 httpResource(`https://api.com/${signalValue()}`)
```





```
1 // Object
2 httpResource(
3     {
4         url: `https://api.com/${signalValue()}`,
5         method: 'GET',
6         params: { type: `${queryParamSignalValue()}` }
7     }
8 )
```

dependency


dependency
// Object
httpResource(
{
url: `https://api.com/${signalValue()}`,
method: 'GET',
params: { type: `${queryParamSignalValue()}` }
}
)

```
1 // Object
2 httpResource(
3     {
4         url: `https://api.com/${signalValue()}`,
5         method: 'GET',
6         params: { type: `${queryParamSignalValue()}` }
7     }
8 )
```

Http verb

```
1 // Function
2 httpResource(() => `https://api.com/${signalValue()}`)
```

```
1 // Function
2 httpResource(() =>
3               signalValue() ?
4               `https://api.com/${signalValue()}` :
5               undefined
6 )
```

```ts
// String with Options
httpResource(`https://api.com/${signalValue()}`, {
    defaultValue: {},
    parse: (response) => zodSchema.parse(response),
})
```

```
// String
resource = httpResource(`https://api.com/${signalValue()}`)
```

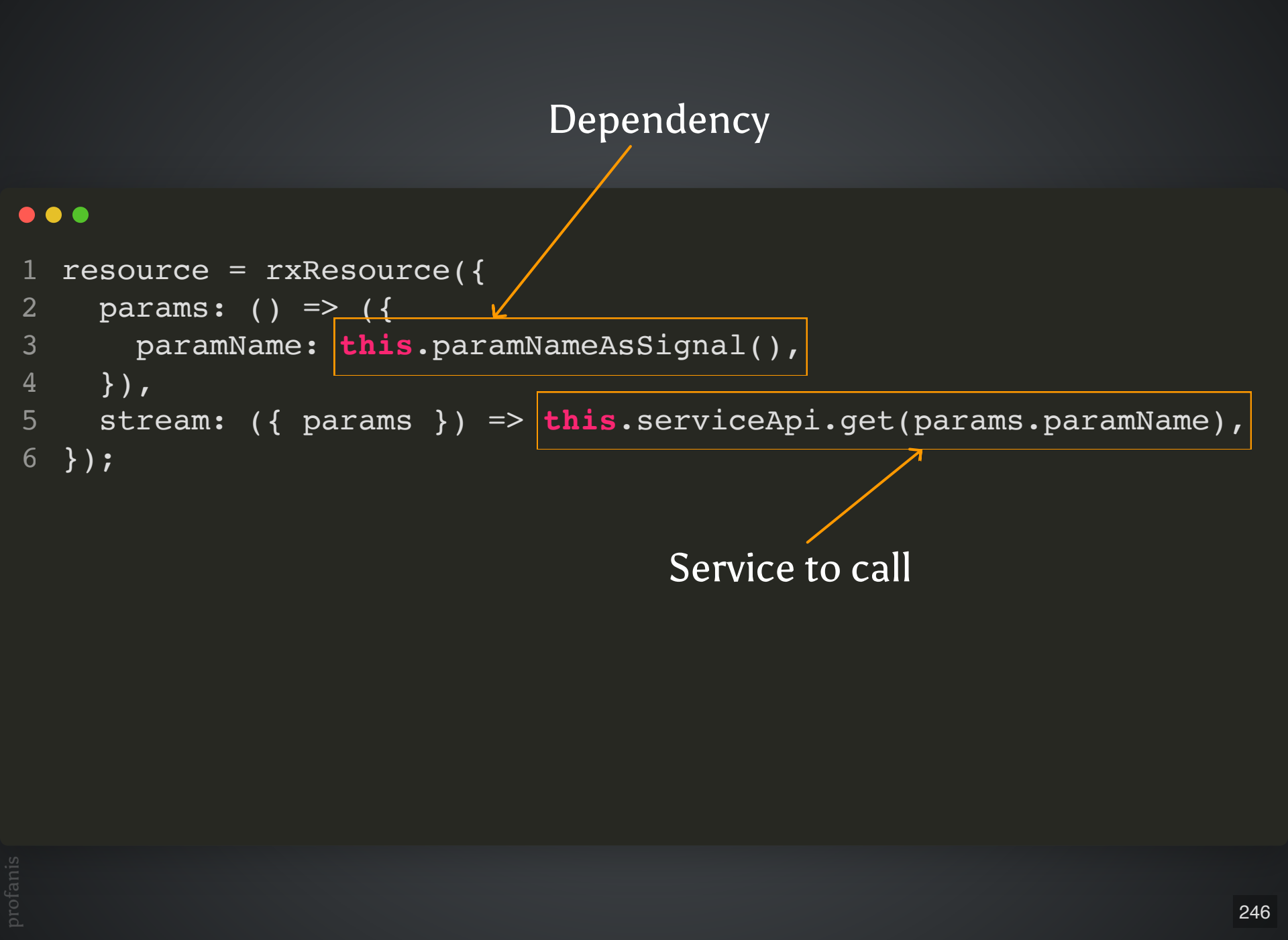
```
@if (resource.isLoading()) {
    <div>Loading...</div>
}

@if (resource.error()) {
    <div>Oops...</div>
}

@if (resource.value()) {
    <div>{{ resource.value() }}</div>
}
```

```
// String
resource = httpResource(`https://api.com/${signalValue()}`)
```

```
derivedState = computed(
  () => resource.value().map(it => it.title)
)
```


Dependency
resource = rxResource({
params: () => ({
paramName: this.paramNameAsSignal(),
}),
stream: ({ params }) => this.serviceApi.get(params.paramName),
});
Service to call


linkedSignal

```
1 listOfItems = signal(['item1', 'item2', 'item3']);
2
3 countOfItems = linkedSignal(() => this.listOfItems().length);
4 countOfItems.set(0)
```



```
1 listOfItems = signal(['item1', 'item2', 'item3']);
2
3 countOfItems = computed(() => this.listOfItems().length);
4 countOfItems.set(0)
```



```
1 listOfItems = signal(['item1', 'item2', 'item3']);
2
3 countOfItems = linkedSignal(() => this.listOfItems().length);
4 countOfItems.set(0)
```

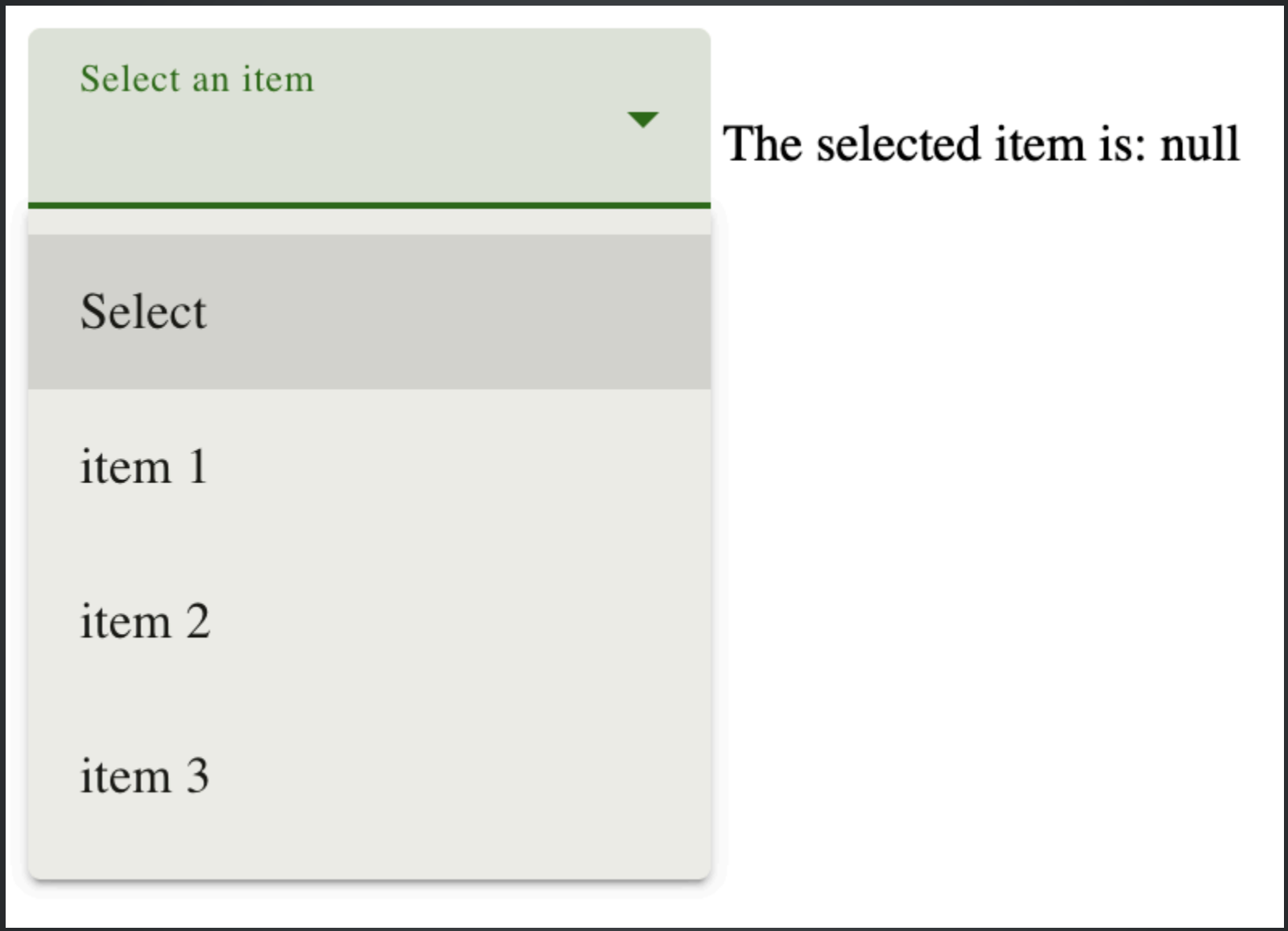
```
1 listOfItems = signal(['item1', 'item2', 'item3']);
2
3 countOfItems = linkedSignal({
4     source: this.listOfItems,
5     computation: (items) => items.length,
6 });
7 countOfItems.set(0)
```

```
1 listOfItems = signal(['item1', 'item2', 'item3']);
2
3 countOfItems = linkedSignal({
4     source: this.listOfItems,
5     computation: (items) => items.length, // 3
6 });
7 countOfItems.set(0)
```

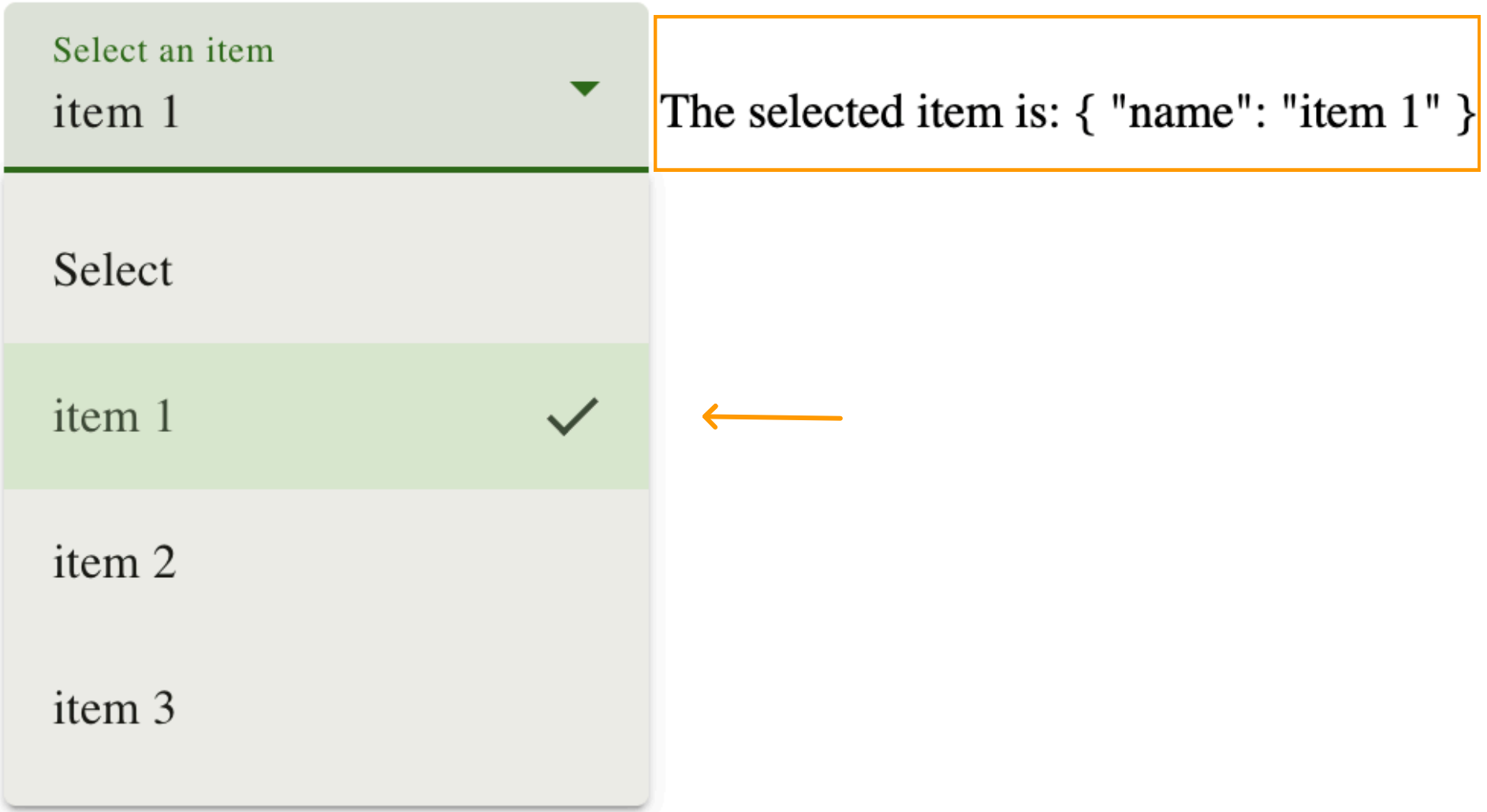


```
1 listOfItems = signal(['item1', 'item2', 'item3', 'item4']);
2
3 countOfItems = linkedSignal({
4     source: this.listOfItems,
5     computation: (items) => items.length, // 4
6 });
7 countOfItems.set(0)
```


Select an item
The selected item is: null


Select an item
The selected item is: null
Select
item 1
item 2
item 3

profanis


Select an item
item 1
The selected item is: { "name": "item 1" }
Select
item 1
item 2
item 3

```
1 listOfItems = signal([
2   { name: 'item 1' },
3   { name: 'item 2' },
4   { name: 'item 3' }
5 ]);
6
7
8 // keeps the selected item
9 selectedItem = listOfItems[0];
```

```
listOfItems = signal([
  { name: 'item 1' },
  { name: 'item 2' },
  { name: 'item 3' }
]);



// an HTTP call is happening
http.get().subscribe(data =>
    this.listOfItems.set([...data])
)
```



The selected item is: { "name": "item 1" }

```
1  listOfItems = signal([
2    { name: 'item 1' },
3    { name: 'item 2' },
4    { name: 'item 3' }
5  ]);
6
7
8
9
10
11 selectedItem = signal<Item | null>(null);
```

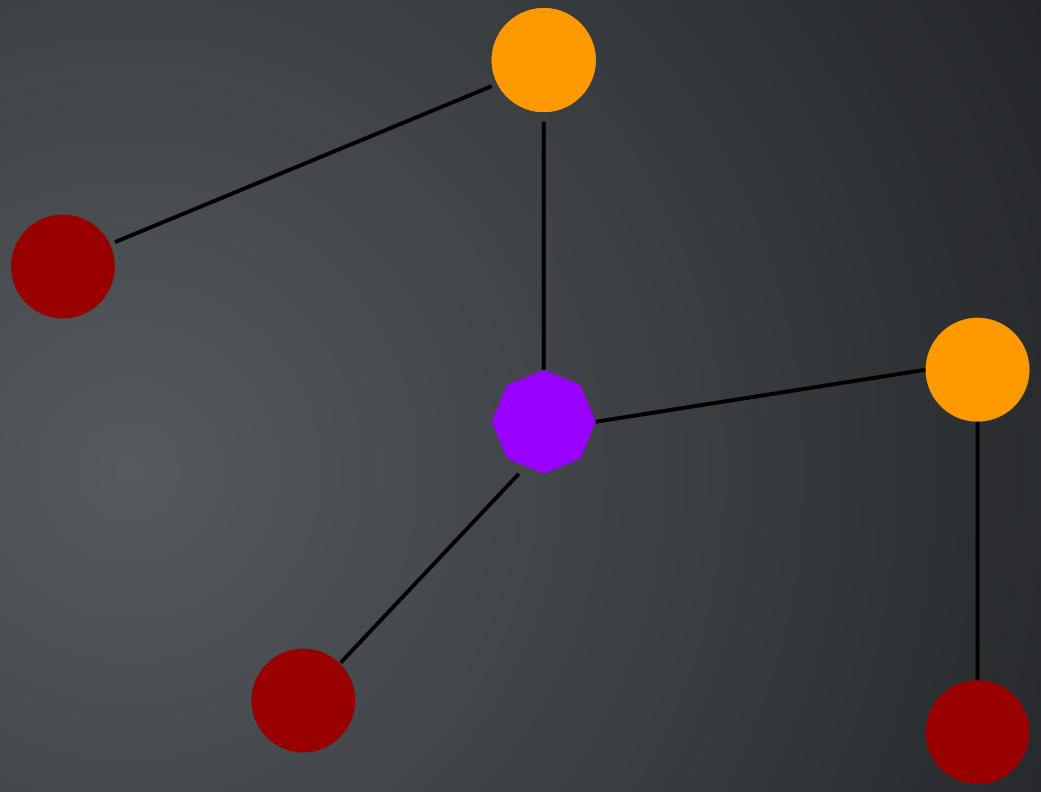


```
1  listOfItems = signal([
2    { name: 'item 1' },
3    { name: 'item 2' },
4    { name: 'item 3' }
5  ]);
6
7
8
9
10 selectedItem = linkedSignal({
11   source: this.listOfItems,
12   computation: (items, previous) =>
13     items.find((item) => item.name === previous?.value.name),
14 });
```

```
1  listOfItems = signal([
2    { name: 'item 1' },
3    { name: 'item 2' },
4    { name: 'item 3' }
5  ]);
6
7
8
9
10 selectedItem = linkedSignal({
11   source: this.listOfItems,
12   computation: (items, previous) =>
13     items.find((item) => item.name === previous?.value.name),
14 });
```



The Graph



```
1 const counter = signal(0);
```





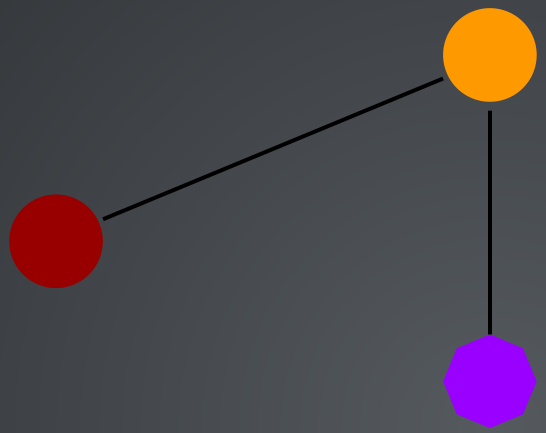
```
1 <div> {{ counter() }} </div>
```

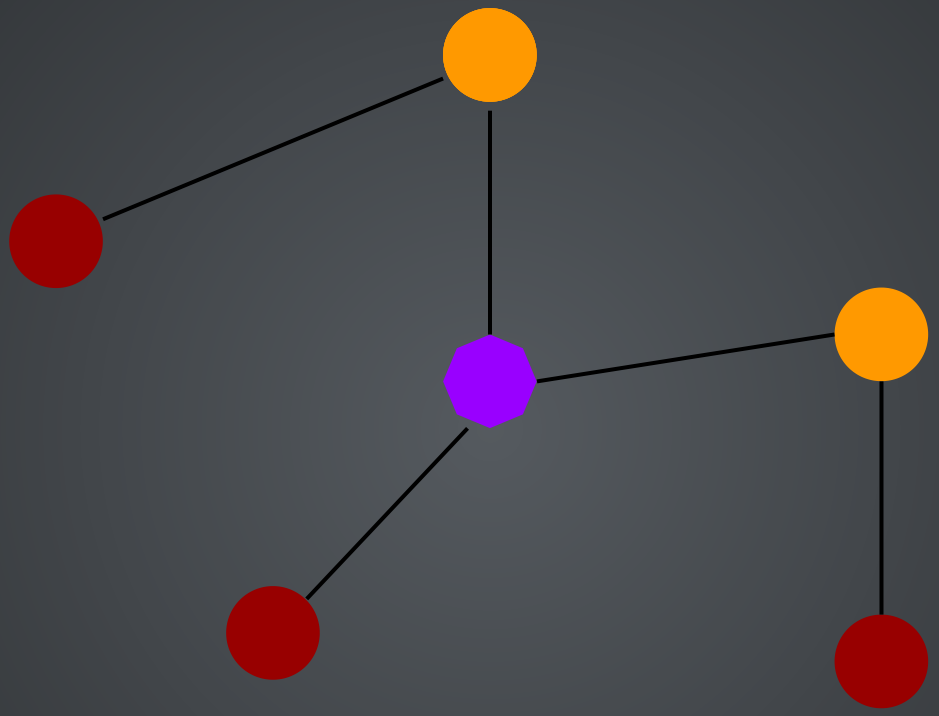






```
1 const evenOrOdd =
2       computed(() => counter() % 2 === 0 ? 'even' : 'odd');
```





"Since Angular knows how the data flows, can have a more fine-grained change detection"

Pull - Push

```
1  const isValid = signal(true);
2  const username = signal('profanis');
3
4  effect(() => {
5    if (isValid() === true) {
6      console.log(username());
7    }
8  });
9
10 // Update signal values
11 isValid.set(false);
12 username.set('profanis2');
```

Consumers

effect

Push (notification)

Producers

isValid

username

```
1 effect(() => {
2   if (isValid() === true) {
3     console.log(username());
4   }
5 });
6
7 // Update signal values
8 isValid.set(false);
9 username.set('profanis2');
```

Consumers

effect

Producers

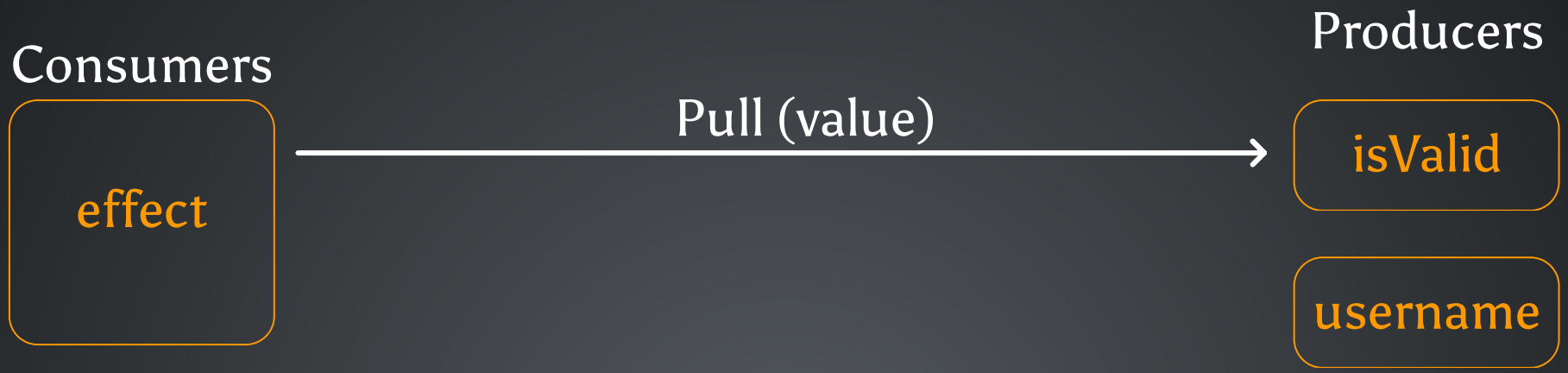
isValid

username

Push (notification)



```
1 effect(() => {
2   if (isValid() === true) {
3     console.log(username());
4   }
5 });
6
7 // Update signal values
8 isValid.set(false);
9 username.set('profanis2');
```




```
1 effect(() => {
2   if (isValid() === true) {
3     console.log(username());
4   }
5 });
6
7 // Update signal values
8 isValid.set(false);
9 username.set('profanis2');
```

Consumers

Producers




```
1 effect(() => {
2   console.log(`${isValid()} - ${username()}`);
3 });
4
5 isValid.set(false);
6 username.set('profanis2');
```

Consumers

Producers

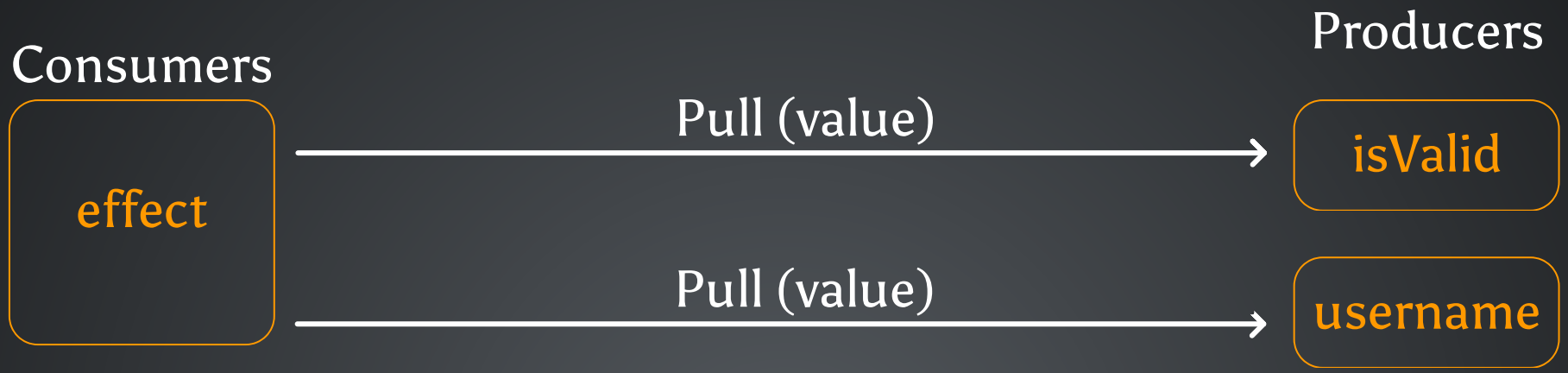
effect

isValid

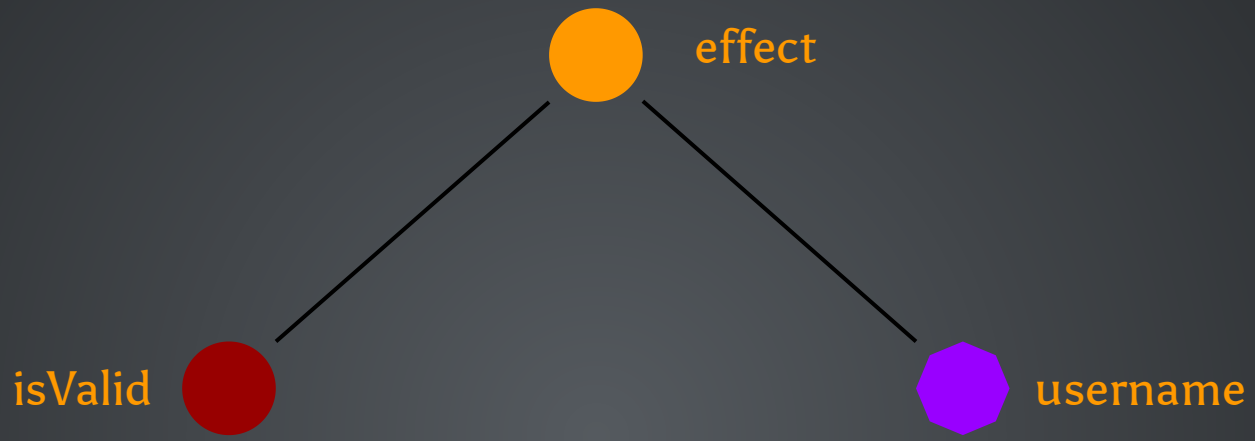
Push (notification)

username

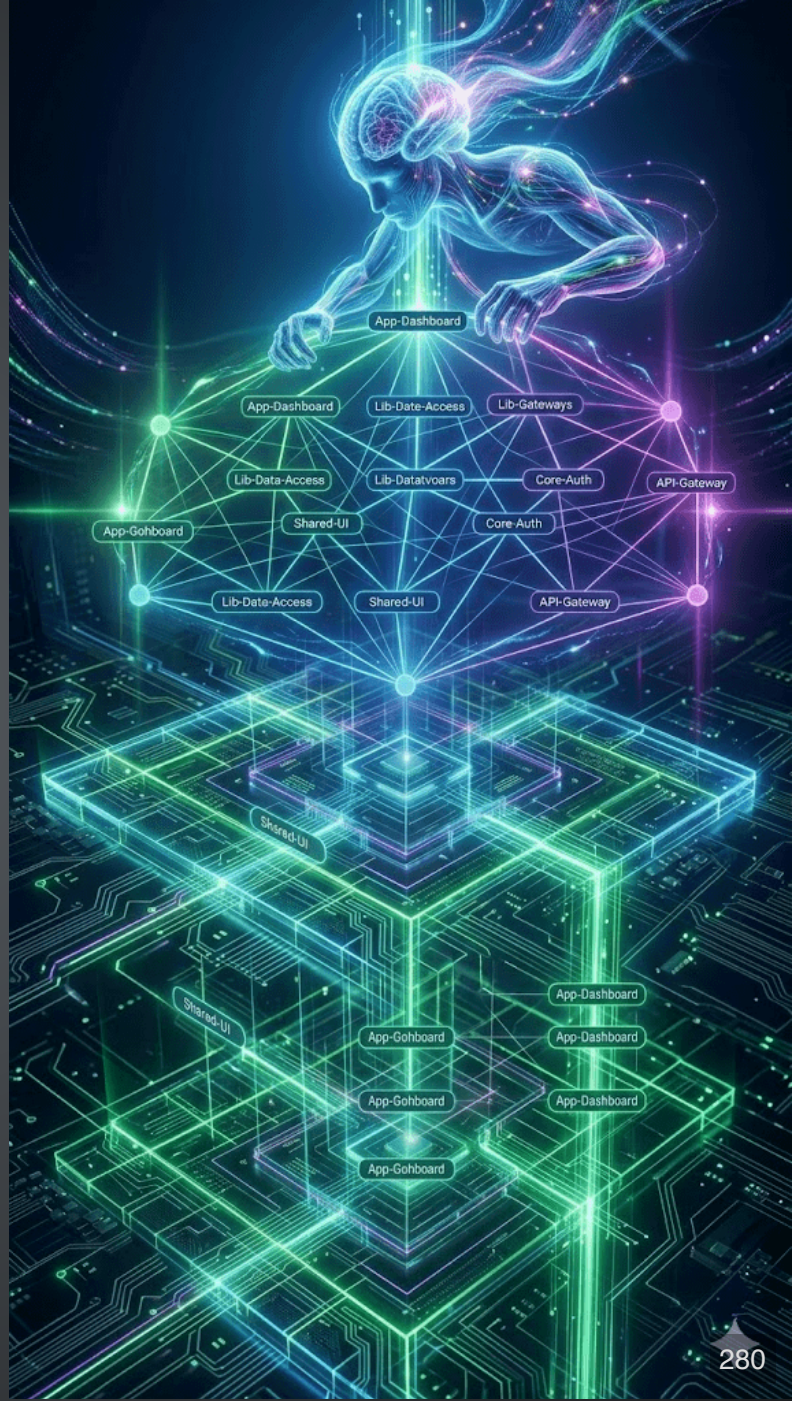
```
1 effect(() => {
2   console.log(`${isValid()} - ${username()}`);
3 });
4
5 isValid.set(false);
6 username.set('profanis2');
```




```
1 effect(() => {
2   console.log(`${isValid()} - ${username()}`);
3 });
4
5 isValid.set(false);
6 username.set('profanis2');
```


effect
isValid
username

Agentic Coding with AI



Modern Angular Expert

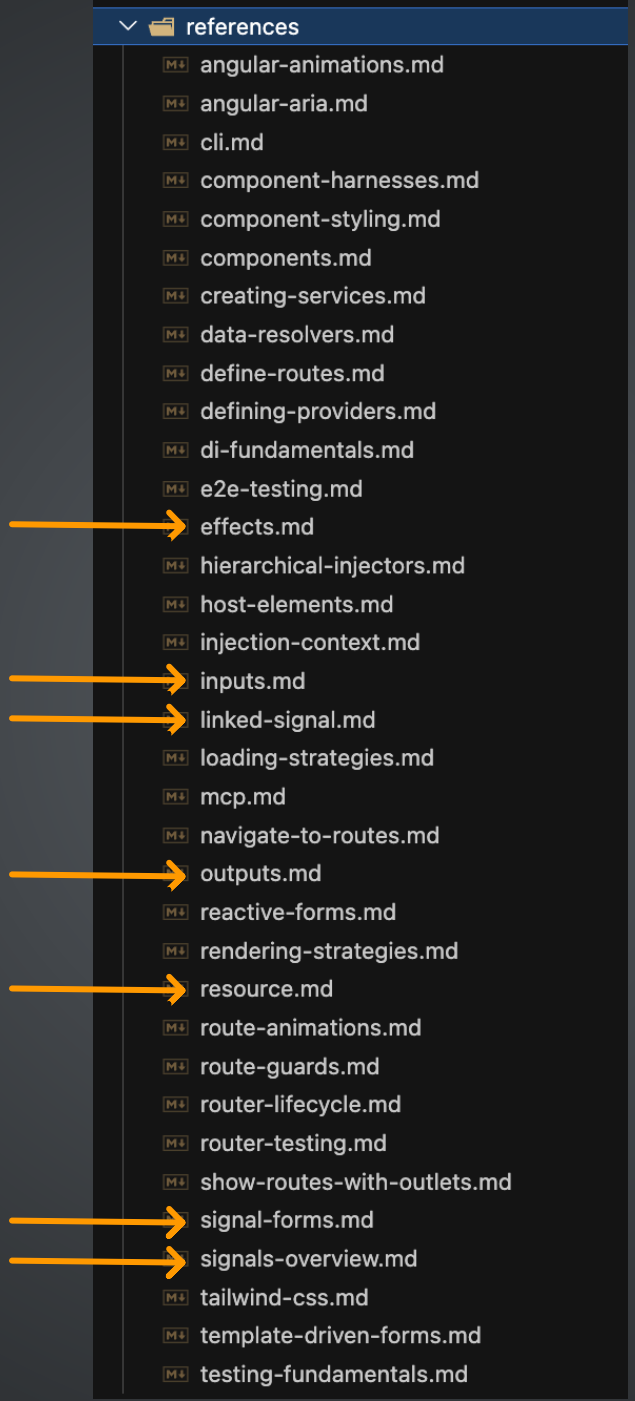
Bridging the Framework Gap

- The Legacy Trap
  - LLMs often default to NgModules and decorators
- Agent Skills (The Brain)
  - In-repo rules for signals and modern angular
- MCP Servers (The Hands)
  - Get best practices, search documentation

Angular Skills

Set it up

```
npx skills add https://github.com/angular/skills
```


references
angular-animations.md
angular-aria.md
cli.md
component-harnesses.md
component-styling.md
components.md
creating-services.md
data-resolvers.md
define-routes.md
defining-providers.md
di-fundamentals.md
e2e-testing.md
effects.md
hierarchical-injectors.md
host-elements.md
injection-context.md
inputs.md
linked-signal.md
loading-strategies.md
mcp.md
navigate-to-routes.md
outputs.md
reactive-forms.md
rendering-strategies.md
resource.md
route-animations.md
route-guards.md
router-lifecycle.md
router-testing.md
show-routes-with-outlets.md
signal-forms.md
signals-overview.md
tailwind-css.md
template-driven-forms.md
testing-fundamentals.md

Angular MCP

Set it up

```
ng mcp
```

Angular MCP

Example

```
# To start using the Angular CLI MCP Server,
# add this configuration to your host:

{
  "mcpServers": {
    "angular-cli": {
      "command": "npx",
      "args": ["-y", "@angular/cli", "mcp"]
    }
  }
}
```

`.mcp.json`

Angular MCP in Claude

```
{
  "enabledMcpjsonServers": ["angular-cli"], // <-- you need this line
  "extraKnownMarketplaces": {
    "nx-claude-plugins": {
      "source": {
        "source": "github",
        "repo": "nrwl/nx-ai-agents-config"
      }
    }
  },
  "enabledPlugins": {
    "nx@nx-claude-plugins": true
  }
}
```

.claude/settings.json

Angular MCP in an Nx Workspace

Framework Intelligence inside the Modulith

- The Synergy

  - Nx defines the where (Architecture); Angular MCP ensures the how (Standards).

- Angular CLI MCP tools

  - list_projects -> requires an angular.json
  - get_best_practices -> useful
  - search_documentation -> useful
  - find_examples -> useful
  - onpush_zoneless_migration -> useful
  - ai_tutor -> useful (teach me angular signal forms)

Workflows

workflow 1 -> Find & Migrate Legacy Inputs

- Search all libs in this Nx workspace for components still using the @Input() decorator. For each one found, use the angular-cli MCP modernize tool with the signal-input-migration transformation to convert them to the signals-based input() function.

Workflows

workflow 2 -> Modernize → Targeted Fix

- Find the affected changes using Nx skills. For each project, use the angular-cli MCP search_documentation to look up the relevant breaking change, then use find_examples to show the correct modern pattern. Apply the fix.

Workflows

workflow 3 -> Agentic Migration Audit

- Using the /nx-workspace skill, list all projects in this workspace and identify which are applications vs libraries — including their source root paths.

  Then use the angular-cli MCP onpush_zoneless_migration on each library's source root and report back a summary of what needs to change in each project before we can go zoneless.

Hands-on



From memory to the network

Everything so far was already in the browser

The catalogue has been showing you eight hardcoded plants all day.

Time to delete them.

1 A request that refires when its inputs change → rxResource

2 State that follows a source but keeps its history → linkedSignal

3 A view over all of it → computed, same as this morning

Hands-on

Three exercises

1. Fetch the products with rxResource
2. Accumulate pages with linkedSignal
3. Filter the result with computed

Branch: lab/three — everything from this morning is already in it.

All three live in one file: catalogue.state.ts. Do them in order — each one keeps the app running.

Hands-on

Last branch of the day

TASK

```
git stash push -u -m "my lab two work"
git checkout lab/three
```

One file does almost all the work today:

libs/catalogue/feature-catalogue-list/src/lib/catalogue/catalogue.state.ts

Open it now and read it top to bottom before you change anything. Two of the three answers are already half-written in there.

Hands-on

Fetch the products

TASK

1. Create a private productsResource that reloads whenever page changes
2. Make totalProducts read the real total instead of the hardcoded 70

You are not calling the API. You are declaring which signals the request depends on. Angular decides when to run it.

Hands-on

Shape of the answer

HINT

```
1 private productsResource = rxResource({
2   params: () => ({ page: this.page() }),
3   stream: ({ params }) => this.productsApi.getProducts(params.page),
4 });
5
6 totalProducts = computed(() => this.productsResource.value()?.total || 0);
```

params is the dependency list. Read a signal in there and the request refires when it changes.

stream is the call itself. It receives what params produced — it does not read signals of its own.

.value() is undefined until the first response lands. That is why the ?. is there.

Hands-on

The request fires by itself

CHECKPOINT

- ✓ DevTools → Network shows one products request on load — nobody called it
- ✓ "Showing 8 of … plants" now reads a number that came from the server
- ✓ The grid still shows the same 8 plants. It should — you have not switched it over yet

No ngOnInit. No subscribe. No takeUntilDestroyed. The request exists because something reads its value.

Hands-on

EXERCISE 2 / 3

Keep the pages you already loaded

TASK

1 combinedProducts is already written — except for its source. Give it one.

2 Read the computation underneath it. Be ready to say out loud what previous.value is for.

Why not a computed? A computed throws its previous value away every time. Page 2 would replace page 1 instead of joining it.

A linkedSignal follows a source and can see what it held before. Read the computation that is already there — previous.value is the whole point.

Hands-on

EXERCISE 2 / 3

One line. That is the whole exercise.

HINT

```
// catalogue.state.ts
private combinedProducts = linkedSignal({
  source: this.productsResource.value,   // <- the only line you write
  computation: (source, previous) => { /* already written for you */ },
});
```

Hands-on

Still eight plants on screen

CHECKPOINT

- ✓ The app compiles and still shows the mock plants
- ✓ Clicking Load More fires a second request, for the next page
- ✓ No error in the console — the linkedSignal now has a real source to call

Nothing new appears in the grid yet, because the grid still reads the mock list. The plumbing is done; the tap is closed.

Watch the Network tab instead of the page. That is where exercise 2 is visible.

Hands-on

EXERCISE 3 / 3

Open the tap

TASK

1 Turn products into a computed over combinedProducts

2 Filter it by searchTerm — empty term means everything

This is the one line that connects everything: the resource feeds the linkedSignal, the linkedSignal feeds this, and this feeds the grid.

Nothing downstream changes. productsWithFavorites in the component keeps working, untouched, because it only ever asked for products().

Hands-on

EXERCISE 3 / 3

Shape of the answer

HINT

```
products = computed(() => {
  const term = this.searchTerm().toLowerCase().trim();

  if (!term) {
    return this.combinedProducts();
  }

  return this.combinedProducts().filter((product) =>
    product.name.toLowerCase().includes(term)
  );
});
```

Delete mockProducts now if you have not already. TypeScript will tell you if anything still points at it.

Hands-on

Seventy plants, no subscriptions

CHECKPOINT

- ✓ Real plants from the server, not the eight you knew by heart
- ✓ Load More appends — the grid grows, it does not swap
- ✓ Search still filters, and your favourites survive both

Now break it

Search for a plant you have not scrolled to yet. Nothing comes back.

You are filtering the pages you happen to have loaded, not the catalogue. What would you change — and which signal would the search term have to reach?

Where you started

An empty folder and a CLI command

Then boundaries a linter could enforce

Then state that derives itself

Then a catalogue that fills itself from a server

Not one subscription. Not one ngOnInit. Not one manual refresh.

The architecture told you where code goes. The signals told you when it runs. Neither works as well without the other.